

AI Problem Framing for AI Practitioners

Course book. October 2026 edition.

© 2026 Rajiv Shah. Licensed under [CC BY-NC 4.0](#). You are free to share and adapt this book for non-commercial use, in any medium, as long as you credit Rajiv Shah and link to aiframer.dev.

Table of contents

Introduction	3
--------------------	---

Part 1: The method

Chapter 1: First Principles	6
The first principles map	6
The cost of bad framing.....	6
Why it's hard	12
Three thinking techniques	14
What good looks like	17
First principles in the enterprise.....	19
Judgment, the part agents don't have.....	24
Preview of the GOATS Loop.....	25
The chapter summarized.....	26
Chapter 2: Framing the Agent Task.....	27
Two loops, not one	27
The GOATS Loop.....	28
Step 1: Goal	29
Step 2: Operating Assumptions	37
Step 3: Alternatives	44
Step 4: Trade-offs	51
Step 5: Signals	54
The complete GOATS Loop.....	60
The chapter summarized.....	61
Chapter 3: Knowing If It's Working.....	62
The diagnostic mindset	62
Is there signal?	67

Is there cheating?	78
Is the metric real?	85
From diagnosis to decision	95
The chapter summarized.....	98
Chapter 4: The Decision Point.....	100
The decision map	100
The skill most courses won't teach	101
Name the failure mode.....	102
The three options	108
The reframing moves	115
The shipping matrix	127
Communicating the decision	129
The chapter summarized.....	135

Part 2: Reference module

Diagnostics in Depth.....	137
General diagnostics (any AI system)	137
ML-specific diagnostics.....	146
GenAI / RAG-specific diagnostics	154
Agent-specific diagnostics.....	160
Failure modes by modality.....	162
Diagnostic techniques summary.....	166
Afterword: The arc, end to end	168
Appendix: The four artifacts.....	170
Artifact 1: The Framing Brief	170
Artifact 2: The GOATS One-Pager	171
Artifact 3: The Diagnostic Readout.....	173
Artifact 4: The Decision Memo	174

Introduction

JP Morgan Chase built one hundred fine-tuned models to help customers understand privacy policies. Every accuracy metric was green. Nobody used it. The team spent close to a year trying to tune their way out before asking a different question. Customers didn't want a chatbot that explained a policy. They wanted help deciding whether to accept one.

That is the kind of failure this book is about.

Over the last few years I've worked hands-on with more than a hundred AI use cases across enterprise, startups, and research. The successes fade quickly, but the failures stayed with me. Almost none of them were failures of the model. It was because someone had chosen the wrong problem, or defined success so loosely that nobody could tell we were solving it wrong.

I've seen the same pattern across every era I've worked in. Traditional ML, generative AI, now agents. Pilots with strong results that never make it to production. The popular explanation is that the technology is immature. I think that's mostly wrong. The technology is plenty mature for what most teams are trying to do. What's immature is the framing: setting yourself up for success by thinking through the typical pitfalls of AI projects.

This book is about the framing layer that sits above evals, frameworks, and architecture choices. Evals are the inner loop, whether the model produced a good output. The outer loop, the harder one, is whether you framed the right problem at all. Most teams have a great inner loop and no outer loop.

The backbone of the book is something I call the GOATS Loop. Five questions, in order, to ask about any AI problem before you build it and again every time something breaks.

1. **Goal.** What decision is the system supporting, and how will we measure it?
2. **Operating Assumptions.** What would have to be true for this approach to work?
3. **Alternatives.** What approaches could solve this, and do we even need AI?
4. **Trade-offs.** How do we choose between candidates?
5. **Signals.** How will we know if we're wrong?

It looks simple on the page and is not simple in practice. Each question hides assumptions teams routinely skip past, and the cost of skipping them compounds the further into a project you go. Run it before you build, then go back to the assumptions the moment a signal tells you something is off.

The book is in two parts. Part 1 is the method: the four chapters, in order. First Principles, on why projects fail and how to think about them. Framing the Agent Task, which walks the GOATS Loop end to end. Knowing If It's Working, the diagnostic tests for a system that's already running. The Decision Point, on when to persist, pivot, or stop. Everyone reads Part 1. Part 2 is one reference, Diagnostics in Depth, split into ML, GenAI, and agent sections, so an ML practitioner can dig into ML diagnostics and

an agent builder into trajectories. General practice first, then specialty, in the same way medicine works. An appendix closes the book filled with worksheets you can apply right away. If you want a deeper dive into additional examples or hands-on practice, I offer a course which you can participate in.

A note on what reading this should feel like. If you finish a chapter feeling slower, less confident, or a little annoyed, you're doing it right. The instincts that got you here are trained on years of "I've seen this before, I know what to build." That pattern matching is fast and usually right. The trouble is that AI keeps producing problems that look like ones you've solved before but aren't. One of my goals is to encourage you to slow down and use first principles thinking for new projects.

This book isn't a coding guide. The how is well-served elsewhere, by frameworks, vendor docs, and the long tail of AI tutorials that appear every week. The more valuable material is thinking more deeply about the structure of your approach to an AI project.

This edition comes with two skills you run on your own project. `frame-use-case` walks the framing before you build. `diagnose-use-case` runs the diagnosis once it's live. Point either one at your project and it drafts the whole frame in minutes: the goal, the alternatives, the trade-offs, the signals. The drafting used to be the slow part of this work, and it isn't anymore. What the skill can't do is the judgment. It can't tell you whether your seventy-five percent target is ambitious or sandbagged, it doesn't know what happened to the last team that shipped your exact problem, and it's trained to agree with you more than a skeptical colleague would. That judgment is what these chapters teach. Read them to learn it, then run the skill and put your attention on the part it hands back to you. Both skills are free and open source at github.com/rajshah4/ai-framing-skills, with install instructions at aiframer.dev/setup.

By the end of this book, you should be able to do four things. Frame any AI problem as a clear decision the system is supporting. Diagnose a live AI system by asking, in order, whether there is signal, whether it is cheating, and whether the metric is real, and trace the answer back to the assumption that broke. Make the persist-pivot-stop call early enough to matter. And know which questions to ask, in which order, every time you start something new.

The first chapter starts with what bad framing costs.

Part 1: The method

The four chapters of the method, in order. Each one runs through a section of the GOATS Loop and ends with a summary.

Chapter 1: First Principles

Why do agent projects fail, and what do you do about it?

The first principles map

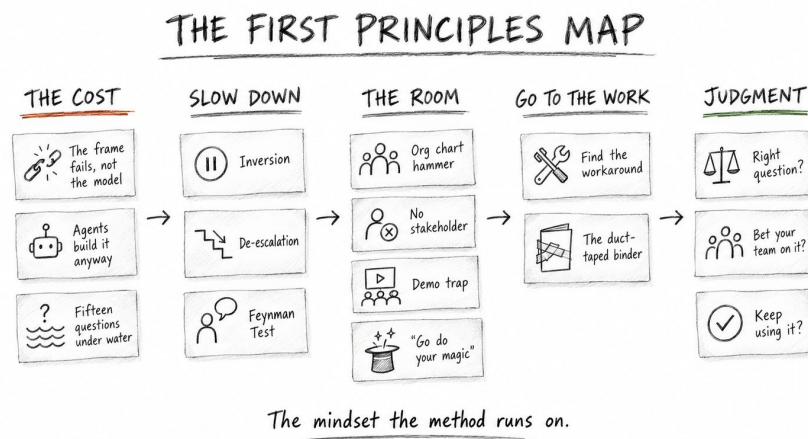


Figure: The chapter runs left to right. Why the frame fails rather than the model, three ways to slow the first answer, four traps in the room, going to the work, and the judgment that catches what the metrics miss.

The cost of bad framing

MD Anderson set out to use AI to help cure cancer. They partnered with IBM Watson on an Oncology Expert Advisor, and over four years the contract stretched from a couple million dollars to sixty-two million across a dozen extensions. Then a University of Texas audit found the system had never been used on a single patient. It couldn't read the hospital's medical records, it covered only a handful of cancers, and no clinician ran it in real care. The audit was about procurement and oversight, and it never had to rule on the model, because the question the model would have answered was never pinned down: what decision the thing was supposed to make, for which patient, at what point in treatment. A goal like "cure cancer" gives a team no way to tell whether the system is working, so nobody found out it wasn't until the sixty-two million was already spent. ([The Cancer Letter](#))

\$62,000,000
0 patients treated

MD Anderson + IBM Watson, shut down after 4 years

Figure: Sixty-two million dollars aimed at a mission nobody had turned into a decision.

That's one version of the failure, at the top end of the budget. Here's another. One engineer wrote up how their team shipped four LangChain agents that coordinated to help users research market data. Two of the agents got stuck asking each other follow-up questions, and the loop ran for eleven days straight, racking up forty-seven thousand dollars in API costs before anyone noticed. The team had never defined what "done" meant, and nothing capped the conversation, so the agents never finished. (Teja Kusireddy, ["We Spent \\$47,000 Running AI Agents in Production"](#))

The third one I keep coming back to is JP Morgan Chase, the privacy-policy models from the introduction. They hit every accuracy target, shipped a clean interface, and on launch day, in the words of their former head of AI innovation, got "crickets." The gap only surfaced after launch, because nobody had asked the question the whole project depended on.

Stand those three side by side and the same pattern shows up in each. MD Anderson aimed sixty-two million dollars at a mission nobody had turned into a decision. The research team never said what "done" meant, so its agents never stopped. JP Morgan built the answer right and never asked whether anyone wanted it. Three very different disasters, and in every one the framing that should have come before the building never happened.

If you see those as unlucky outliers, they aren't. There are many more, and here are a few from the last couple of years. My favorite is DPD, whose customer service chatbot swore at a customer and wrote a poem about how bad DPD was. It was funny in a screenshot. It was less funny inside DPD. The rest are the same lesson at different companies.

Case	What happened	The framing gap
Air Canada (ABA)	Chatbot invented a bereavement-refund policy; a tribunal held the airline liable for what its bot told a customer	A generative model wired into a customer-facing path with no retrieval anchor and no human review

Case	What happened	The framing gap
DPD (Y94)	Chatbot swore at a customer, wrote a poem trashing the company. Disabled within days.	No release discipline, boundary tests, or escalation path before going live
Meta Galactica	Great benchmark scores, pulled in 72 hours after generating authoritative scientific nonsense	The benchmark and the user experience were measuring different things
Devin, at launch (IEEE Spectrum, Answer.AI, Cognition)	Demoed beautifully in 2024 as an autonomous "AI software engineer." Cognition's own report put it at about 14% on real GitHub issues, and Answer.AI's month of independent testing got 3 successes out of 20 tasks, including work that looked finished and did nothing. Cognition now pitches it for tasks with clear requirements and verifiable outcomes, with human review	The original frame, replace an engineer, couldn't survive that error rate. The reframe, delegate scoped tasks a human reviews, could
Bayezian (AI Hub)	Clinical-trial agent flagged a missing Day 14 lab test without recalling that a Day 13 test had satisfied the requirement	Autonomy without a memory architecture, in a domain where errors compound

The one making the rounds as I write this is the same loop, except the two bots belonged to different companies. A big company pointed a chatbot at its inbound email so threads got handled without a human. So did its client. Neither team planned for the other end of the thread to also be a bot, so the two auto-responders kept politely answering each other for days, nearly twenty thousand emails, before anyone caught it. Each company owned half the exchange and watched only its own outgoing mail, so nobody could see the whole loop. I picked this one up secondhand on social, so hold the number loosely. The mechanism is identical to the agents that opened this section: no cap on how many times it would reply, and no signal for "am I stuck in a loop."

I don't have to take this one secondhand, because a version of it happened at work. At OpenHands, a triage bot reads new GitHub issues and adds a `ready-for-dev` label when it decides an issue is ready. The repo also has a readiness check with its own rule, and on [one issue](#) the bug report had no screenshot, so the check removed the label. Removing the label triggered the triage bot again, which re-added it, which set off the check, and the two went back and forth about 150 times. That drove Issue Triage to roughly 200 runs an hour, and the VM that runs our open-source automation went down. Neither bot was wrong by its own rule. Nobody had decided which one owned the call on "ready," and nothing capped how many times either could touch the same issue. The fix was to make the triage bot read the check's verdict. It won't add the label while the check is failing, it asks the author for what's missing instead, and it no longer re-runs every time the check edits its comment.

I could keep going. I won't, because every example points at the same place. The visible part of every agent project is the agent doing a task. The invisible part is the stack of decisions underneath that decide whether the visible part works. I've put fifteen of them below, and that isn't the whole list, just

enough to show how much sits under the surface. Fifteen is already too many to hold in your head, so hold the four clusters instead.

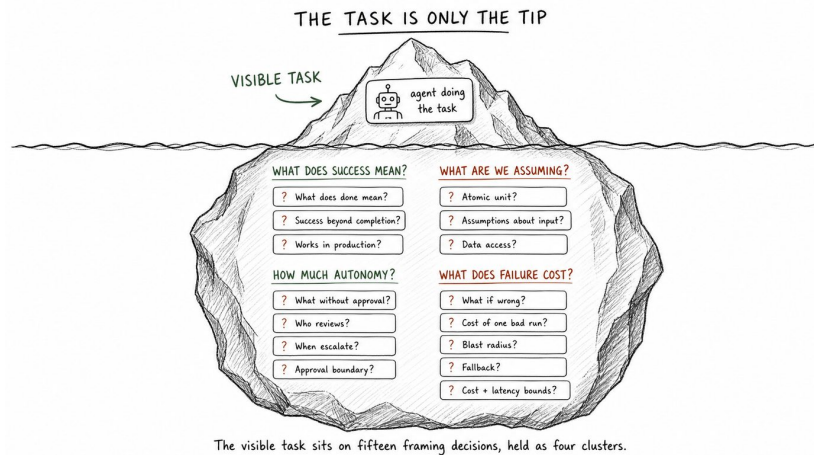


Figure: The visible agent task sits on top of the framing decisions hidden below the surface.

What does success mean?

- What does "done" mean?
- How do we measure success beyond "it completed the task"?
- How will we know it's working in production, not just in demos?

What are we assuming?

- What is the atomic unit of work?
- What assumptions are we making about user input?
- What data does the agent need access to?

How much autonomy does it get?

- What level of autonomy does this need?
- What can the agent do without human approval?
- Who reviews the output?
- When should this system escalate to a human?

What does failure cost?

- What happens when the agent is wrong?
- What is the cost of one bad run?
- What is the blast radius of an autonomous error?
- What is the fallback when the agent fails?
- How do we bound cost and latency?

Those are framing decisions, not engineering ones. You can't prompt-engineer your way out of the wrong answer to "what does done mean," and no amount of fine-tuning will fix a system that needed to be a search index.

The four clusters are also the spine of this book. Chapter 2 turns them into a method: the success questions become Goal and Signals, the assumption questions become Operating Assumptions, the autonomy questions become Alternatives, and the failure-cost questions become Trade-offs. For now, notice that every failure story above traces back to a question in one of these clusters that nobody asked.

The iceberg is only useful if it changes the system you build. A vague request like *"build an agent to handle this task"* should leave behind controls the design can enforce.

A few translations are immediate. If the framing question is *what does done mean?*, the design control might be a maximum number of iterations or an explicit stop condition. If the question is *what is the cost of one bad run?*, the control might be a budget ceiling. If the question is *what happens when the agent is uncertain or wrong?*, the controls are confidence thresholds, fallback responses, and escalation rules. Those controls are the visible trace of better framing.

The diagnostic question here is: *what constraint would we add to the design if we took our framing answer seriously?* If the answer is "none," you probably have a discussion point, not a frame.

There is a coding-agent version of the same idea: **the codebase is part of the prompt**. Your explicit instruction is only a slice of what the agent reads. The rest comes from the environment around the task: consistent naming conventions, existing test files, folder structure, architectural patterns, and examples of similar work already done.

That means a well-structured project gives the agent better implicit guidance. A messy project gives it conflicting instructions. The agent responds to the shape of the system it is dropped into as much as to what you ask in chat.

The practical lesson is to make the environment legible, not to write longer prompts. If you want an agent to behave predictably, give it conventions it can discover, tests it can run, patterns it can copy, and boundaries it can respect. In Gratzer's formulation, agent output improves when the project itself carries the workflow.

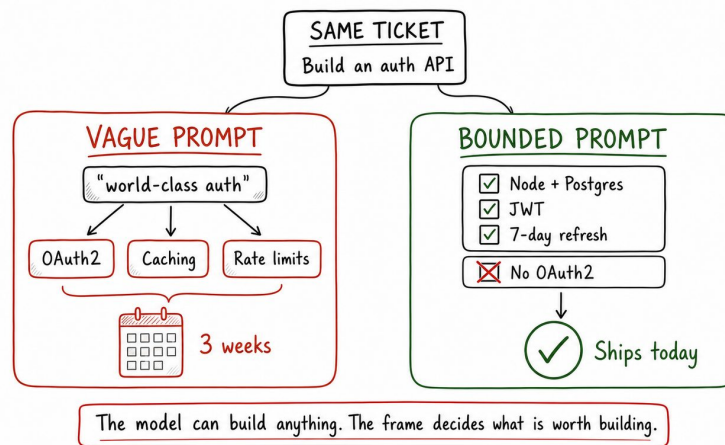


Figure: The model can build either path; the frame determines whether the result is useful.

Soma Kesara has a worked example that makes the contrast concrete. Two engineers get the same ticket: build an API for user authentication. The first writes the impressive-sounding prompt. "Build a world-class auth API with industry best practices. Make it robust and scalable." The coding agent does exactly that, and returns OAuth2, a caching layer, rate limiting, several auth strategies. All of it competent. None of it requested. The team uses simple token auth, has no caching layer, and OAuth2 isn't on the roadmap. Three weeks go into integrating features nobody needed.

The second engineer writes the prompt that reads worse and frames better. "Build a token-based auth API on Node and Postgres, no caching layer. Issue a JWT on login, validate it on protected routes, expire refresh tokens after seven days. Out of scope: OAuth2, multi-factor, social login." The agent returns what was asked. Tests pass. It ships that afternoon.

Same task, same model. The second prompt specified the stack, the acceptance criteria, and what was out of scope, and that is what cut three weeks of integration down to an afternoon. When the model can generate anything in minutes, a vague request is no longer slow. It's a fast, confident pile of the wrong thing.

Teresa Torres makes the same argument from the product management side. Her line: "building is cheap now, but don't build everything." Cheap building amplifies bad framing, because the faster you can ship the wrong thing, the more damage you can do. Same insight, different vocabulary. ([airfocus](#))

None of this is a new idea. Problem framing has been studied for decades under different names. Thomas Wedell-Wedellsborg has spent years telling HBR readers to reframe the problem before they solve it. Clayton Christensen reframed products around the job the customer is hiring them to do. Toyota's five whys pushes past the symptom to the cause. Design thinking spends an entire phase defining the problem before anyone sketches a solution. What changed is the economics. When building was slow and expensive, a bad frame sat in a backlog long enough that someone usually questioned it. When an agent builds it in ten minutes, the bad frame ships, runs, and bills you. AI removed the delay that used to forgive a bad frame.

Why it's hard

If smart people are making these mistakes, why? The part of the brain doing the framing is the wrong part for the job.

Daniel Kahneman gave us the cleanest language for this in *Thinking, Fast and Slow*. Two systems share the wheel. System 1 is fast, intuitive, and effortless. It pattern-matches against everything you've ever done before. System 2 is slow, deliberate, and effortful. It questions assumptions, runs counterfactuals, sits with discomfort. System 1 feels efficient, and most of the time it is. System 2 feels slow, and it's the system that has any chance of catching the errors System 1 missed.

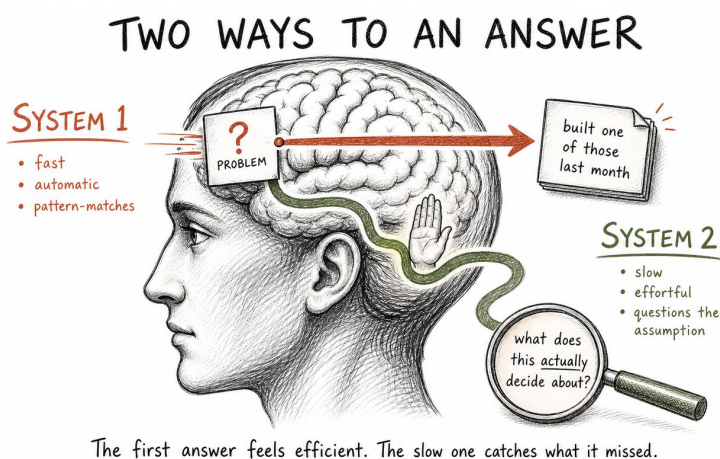


Figure: The fast answer feels efficient; the slow one catches what it missed.

The bat-and-ball problem is the classic demonstration. A bat and a ball cost a dollar ten. The bat costs a dollar more than the ball. How much does the ball cost? Your System 1 says ten cents. The answer is five cents. If you got it right the first time, it's because System 2 stepped in before System 1 finished talking.

Most AI failures look like the bat-and-ball problem in disguise. A team has a backlog of internal documents and System 1 says "RAG." Then System 2 looks at the data and notices that the same dozen documents cover most of the questions, which is not a retrieval problem but a structured lookup. A team has a multi-step internal workflow and System 1 says "agent." Then System 2 walks the steps and notices that nothing between them requires reasoning, just data transformation, which is a script with an LLM somewhere inside it. A team has a hard domain task and System 1 says "fine-tune." Then System 2 asks whether anyone tried prompting first, on the cheap option, before committing to the expensive one. The more you've built, the louder System 1 gets and the harder it is to hear System 2.

The agent era turns up the volume on System 1. The first answer to almost any new problem right now is "we need an agent for this," and that answer is right often enough to feel safe. The cost of being wrong, though, is no longer symmetric. A bad prediction gives you one wrong output and a chance to retrain. A bad agent makes that wrong call repeatedly, autonomously, at cost. The slow-thinking

question, the one System 2 has to ask, is whether the autonomy buys you anything you couldn't get from a more constrained system that you understand.

This pattern has a name at the macro level. Roy Amara observed that we overestimate the short-term impact of new technology and underestimate the long-term impact. The short-term version of "AI agents" right now is "every problem is an agent problem," because that's what's in front of everyone. The long-term version is quieter and more interesting: a durable shift in which problems are even worth solving in the first place. Both of those play out at the same time. System 1 hears the short-term version and gets to work. System 2 keeps an eye on the long-term version. (See [The Hype Cycle and Amara's Law](#) for the visual.)

Jensen Huang has a useful frame for what System 2 looks like in practice. He told an interviewer that the old definition of "smart" was someone who solves technical problems, and that's about to become a commodity. The new definition is someone who is technically capable, empathetic, and able to infer the unspoken. The technical skill still matters. The differentiator is figuring out which problem to solve and why.

Abraham Maslow named a related instinct, the law of the instrument: if the only tool you have is a hammer, every problem starts to look like a nail. The same thing happens in AI, just with a longer list of hammers. If you only speak LLM, every problem becomes a prompt. If you only speak Agent, every problem becomes an autonomous workflow. If your team's expertise is fine-tuning, every problem becomes a fine-tune. The expertise that makes your team good at one approach is what makes the others hard to see. Early GPT models lost at three-digit multiplication for exactly this reason. The team had framed arithmetic as a language task, predicting the next token. Arithmetic is symbolic logic with rules, not patterns, and the reframe was to give the model a calculator and let it call out. Tool use beat generation because the tool finally matched the problem.

A reasonable question at this point is whether the model will eventually learn to frame the problem so we don't have to. The current evidence is that it won't, at least not yet.

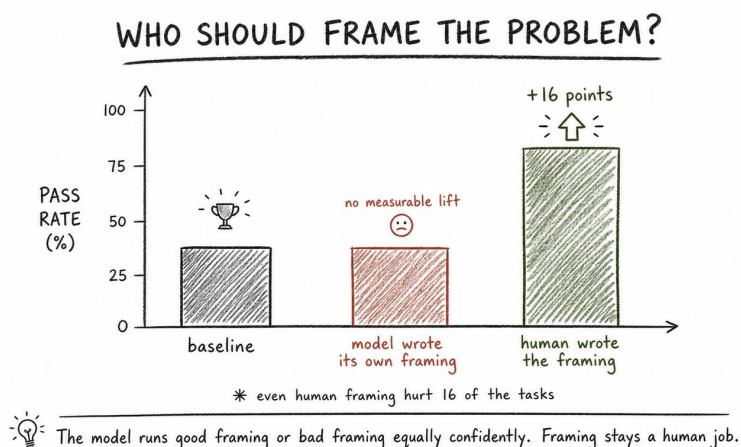


Figure: The model runs good framing or bad framing equally confidently. Framing stays a human job.

A study called SkillsBench tested whether models can generate their own procedural "skills" to guide problem-solving across eighty-four tasks in eleven domains. Self-generated skills provided no measurable benefit on average. Human-curated skills lifted average pass rate by about sixteen percentage points, enough that smaller models on good human skills sometimes matched larger models without them. Even the curated skills made things worse on sixteen of the eighty-four tasks they tested, which tells you that "useful framing" is brittle even when humans write it. The model can't reliably tell good framing from bad. It can execute against either one, confidently. The job of framing remains a human one. ([SkillsBench](#), [arXiv 2602.12670](#))

There's a harder edge to that human job. The reframing literature lists the moves you'd expect: rethink the goal, look for what's missing, study where it already works. The one teams skip, Wedell-Wedellsborg calls looking in the mirror. Ask whether your own assumptions are part of the problem. That's the move this book keeps returning to. The model is not the thing most likely to be wrong about your problem. You are. You picked the frame, you reached for the tool you already knew, you decided what "done" meant, and the agent will execute every one of those choices without questioning a single one. Cheap generation made your judgment the bottleneck, which means the first assumption worth auditing is your own.

Three thinking techniques

Knowing about System 1 doesn't slow it down. You need habits that pull System 2 into the conversation before the decision is made. The next three are small interruptions, each meant for a specific moment when System 1 is about to win.

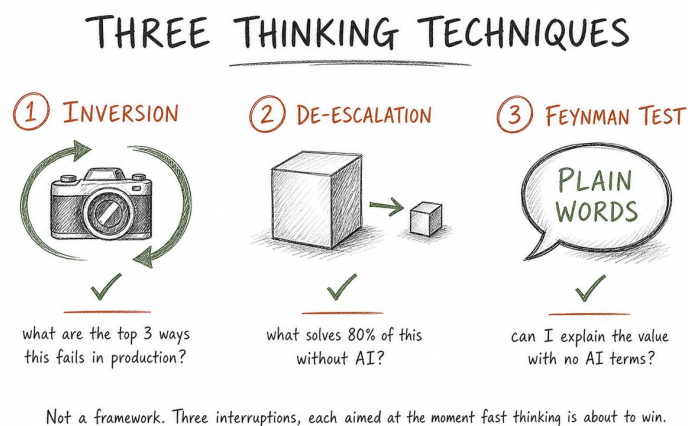


Figure: Three interruptions, each aimed at the moment fast thinking is about to win.

Technique 1: Inversion

Most teams start the design conversation by describing what success looks like. The agent will do this, the user will see this, the metric will go up. Inversion flips the camera around. Instead of "how do I build this," ask "how will this fail?" Then design backward from the failure modes you can imagine.

The diagnostic question is short. *What are the top three ways this agent could go wrong in production?* The trick is to be specific. "It could be inaccurate" doesn't help anybody. "It could confidently cite a policy that was updated last month and is no longer in force" does. The first is a worry. The second is a design constraint.

Once you have your three failure modes, walk through your current design and ask whether it prevents each one. If the answer is no, you've found a framing gap. A team I worked with on a customer onboarding agent went through this exercise and listed three failures: it sends wrong pricing, it loops on ambiguous requests, and it doesn't know when to escalate. The third one stopped them cold. They had no escalation criteria at all. Not "the criteria need work." Zero. They added an escalation policy before writing a single line of agent code, which is the point of Inversion: you find the decisions you forgot you were going to need.

(See [Inversion on Laws of Software Engineering](#) for a visual illustration.)

A sharper cousin of this comes from Elon Musk's design process. His first step is to make the requirements less dumb, and the line that makes it stick is "your requirements are definitely dumb, it doesn't matter who gave them to you." The trap he points at is authority: a requirement from a smart, senior person gets waved through with the least scrutiny, which is exactly the one most worth questioning. Inversion questions your design. Musk's step questions the requirement that produced it. Do both.

Technique 2: De-escalation

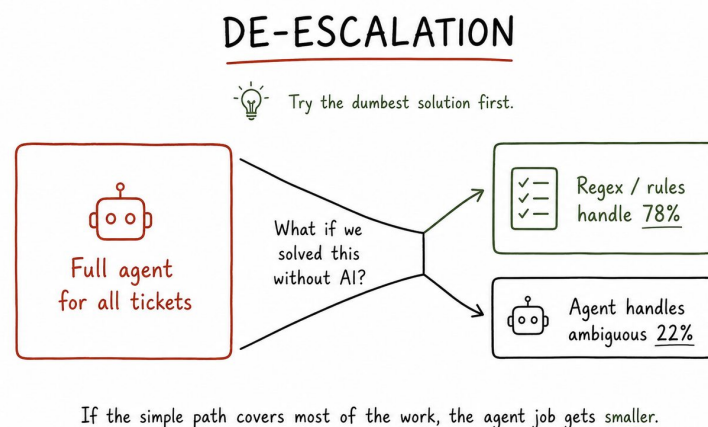


Figure: A simpler solution often absorbs most of the work and shrinks the real agent problem.

Inversion catches gaps inside your chosen approach. De-escalation is more disruptive: it questions whether you needed your chosen approach at all.

The diagnostic question is *if I had to solve this without any AI, how would I do it, and would it be good enough?* Propose the simplest possible solution: a regex, a lookup table, a template, a human with a checklist. Then estimate how much of the problem the simple solution covers. If it gets you above seventy or eighty percent, your agent's actual job is much smaller than you thought. Maybe it just handles the remaining twenty percent. Maybe it shouldn't exist at all.

I worked with a team that wanted an agent to route support tickets. De-escalation revealed that seventy-eight percent of tickets contained keywords that mapped directly to the right team. A regex-based router handled those. The agent only had to deal with the ambiguous twenty-two percent, and once they saw that, the scope, the cost, and the blast radius all came down at the same time. They shipped faster, with less risk, and with a smaller failure surface.

The same instinct applies to the output, not just the model. Mudassar Iqbal, on Genevieve Hayes's Value Driven Data Science podcast, made the point with a scene rather than a deployment: a field operator watching a water geyser erupt doesn't want a fancy dashboard in that moment. They want to know which valve to shut. ([Value Driven Data Science, episode 28, 5:22 to 5:39](#)) De-escalation covers the output you don't over-build as much as the model you skip. A dashboard is the wrong deliverable when the real one is a single instruction, delivered in time to act on it.

Sometimes the answer is yes, you need a full agent, and you arrive there having ruled out everything simpler. That's a much stronger position than starting from "agent" and working backward.

(Related: [Occam's Razor](#). The simplest explanation that fits is usually right, and the corollary is that the simplest solution that works is usually right too.)

Technique 3: the Feynman Test

The third technique comes from Richard Feynman, the physicist sometimes called The Great Explainer. His technique was simple. If you can't explain something in plain language, you don't understand it.

The diagnostic question is *can I explain this agent's purpose to a non-expert without using any AI terms?* Try it out loud. Pretend you're at dinner with a friend in a different industry. Describe what you're building, why it matters, and who it helps, without saying RAG, agentic, multi-step reasoning, tool use, or any of the rest of the vocabulary. Where you stumble or reach for a buzzword to bridge a gap, that's where your framing is weak.

Jargon lets you hide the fact that you don't quite know what you're building. If you can hide behind a word, you don't have to define the reality the word points at. "We're building a multi-agent RAG system with tool use" sounds substantive, but it's a description of architecture, not of value. The first-principle version is what your support team actually does for forty percent of their day, the answer they can never find fast enough, and the time they save when they finally do. Stakeholders need the principle and the consequence, not the math. If the only way you can explain the system is in the math, you don't yet understand it well enough to build the right one.

Jargon as safety blanket	First principle
"We're building a multi-agent RAG system with tool use."	"Our support team wastes forty percent of their time hunting for answers that exist somewhere in our docs. We want to surface the right answer instantly."
"We need an agentic workflow with planning and reflection."	"Customers ask complex questions that require pulling from multiple policy documents and combining the answers. Right now, a human does this in twenty minutes."
"We're deploying an autonomous coding agent."	"Our engineers spend three hours a day on repetitive code changes that follow the same pattern. We want to automate the pattern, not the thinking."

Sol Rashidi, who has been Chief Data and AI Officer at Merck, Sony Music, and Royal Caribbean, puts it cleanly:

"Start with the problem you solve, not what you do. People understand problems. They understand outcomes. The technical details can come later."

The Feynman Test won't tell you how to fix the gap it surfaces. It just tells you where to look. The next chapter is about how to fix it.

What good looks like

Here are two teams that did the framing work before they built.

The HVAC copilot

A team I know wanted to build an "AI copilot" for field technicians servicing industrial HVAC systems. The System 1 take was familiar. Technicians need information, build a chatbot, done. They almost did exactly that. What they did instead was go on fifteen ride-alongs with actual technicians.

On the first ride-along, a technician pulled out a three-ring binder so worn that the cover was held together with duct tape. The product manager asked how often the manual was correct, and the technician laughed: *"We mostly go by what Mike remembers from 2009."* I love this story because the binder told them more than any stakeholder interview ever would. It was sitting right there in the truck, and it was answering the wrong question on purpose.

Across the fifteen sessions, a few things became impossible to ignore. Technicians spent about forty percent of every job hunting for information or calling a senior tech. Tribal knowledge was everywhere, undocumented, stored in people's heads. When something went wrong, the move was to call the one person who knew the system. The phrase "I wish I could just search this" came up over and over. So did "it's always annoying when I can't find the wiring diagram."

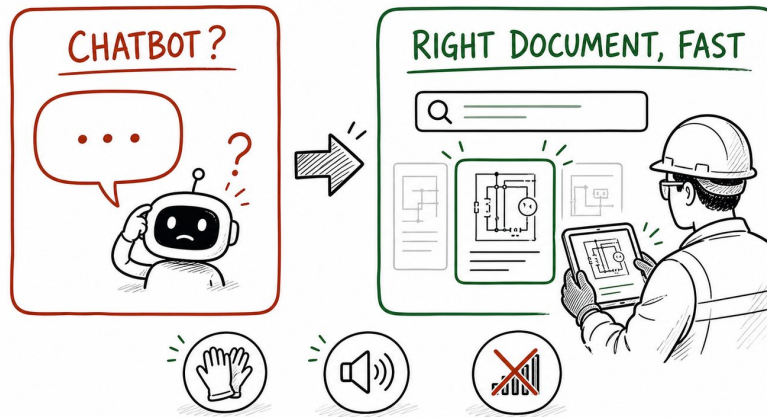


Figure: Going to the work changed the architecture because it changed the team's understanding of the actual problem.

The first-principles insight wasn't that technicians needed a chatbot. It was that technicians couldn't find the right information fast enough. That reframe changed the architecture. The job stopped being generation, where a chatbot writes answers, and started being retrieval, where search surfaces the right document. The team also learned, by being there, that the interface had to work with gloves, in noise, with poor connectivity. None of those constraints were going to come up in a conference room. The team's first attempt at an interactive wizard would have died on contact with a technician wearing gloves.

The three techniques were all in this story, even though nobody put them on a slide. The team de-escalated from agent to search, inverted and found the gloves and the noise and the connectivity, and passed the Feynman Test with a description anyone in the company could understand: technicians can't find information fast enough. All of it happened before anyone wrote code.

JP Morgan Chase

The other story is the second act of one we've already mentioned. JP Morgan Chase built an agent to help customers understand privacy policies. Accuracy was high. User adoption was zero. Crickets. The first-principles question the team should have asked earlier was simple: *if this is accurate but nobody changes their behavior because of it, have we succeeded?* The answer, obviously, is no.

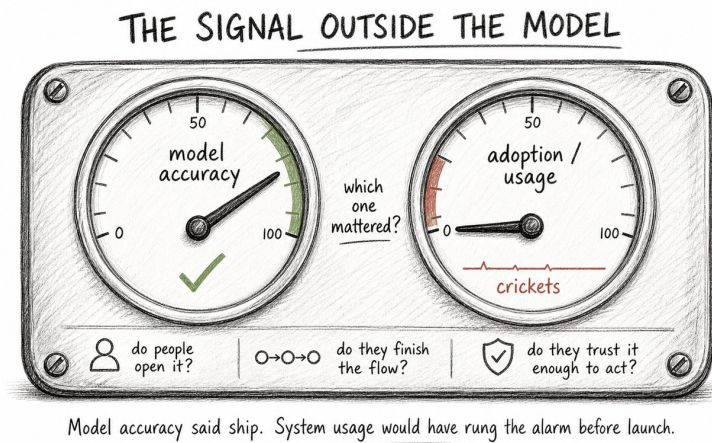


Figure: Model accuracy said ship. System usage would have rung the alarm before launch.

What they did next is the part that makes this a good story. They stepped back and asked what users needed, which turned out to be different from what the agent was producing. Users didn't need answers to questions about privacy policies. They needed help deciding whether a given policy was safe to accept.

Four things changed, none of them in the model. The team connected the agent to existing workflows so it lived where the decision was being made instead of as a standalone tool. They added context that showed users where they were in the document. They added observability so users could see why a given recommendation was being made. And they added a human-in-the-loop approval for high-stakes decisions, which mattered more than they expected.

The output went from "this policy is risky" to "you should be concerned about this privacy policy because we found this segment that says they will sell your data forever to third parties." People used it. People liked it. The model stayed the same, and the team threw out the framing instead.

First principles in the enterprise

The tools work on a whiteboard. You don't work on a whiteboard. You work in an organization with politics, deadlines, team structures, and inherited assumptions, and that environment is what makes first-principles thinking harder than the framework suggests. Four patterns come up often enough to name, because if you don't name them you'll fight them without knowing it. Each one below ends with the phrase you'll hear (the tell), the cost it imposes, and the repair.

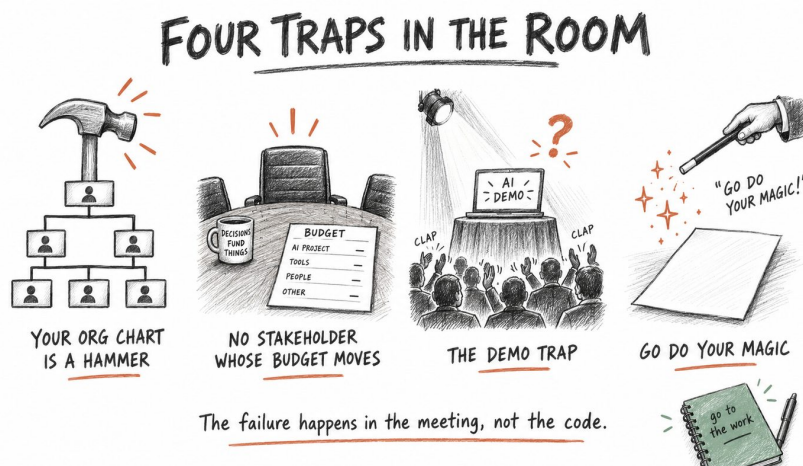


Figure: Four ways a project goes wrong in the meeting rather than in the code. The fix for all four is the same move: go to the work.

Your org chart is also a hammer

Conway's Law says that organizations design systems that mirror their own communication structures. A company with separate ML and Product teams will produce systems with separate ML and Product components, even when nothing about the problem requires that split. The org chart leaks into the architecture. (See [Conway's Law on Laws of Software Engineering](#) for the visual.)

The diagnostic question is uncomfortable on purpose: *is the solution we're imagining shaped by how our team is structured?* Sometimes the cleanest reframe is structurally impossible because the team that should own it doesn't exist. You won't always be able to fix that. At minimum, recognize when your team structure is constraining the solutions you can even conceive of, because that's the moment to push back on the org chart instead of the architecture.

Tell: "that's just how our teams are split." **Cost:** the architecture mirrors the org chart instead of the problem, and the cleanest reframe never gets proposed. **Repair:** name the team structure as a constraint out loud, then push on the org chart, not the design.

The stakeholder problem

Before any of this, check that your project has a stakeholder. If no one's budget moves, no one's team owns the outcome, and no one will be in a different position six months from now if the project ships or doesn't, that's a different kind of failure. A project without a stakeholder produces interesting findings and no decisions. The first move is to find the person whose problem this is. If there isn't one, treat that as a signal that the problem may not be ready for an AI team yet.

Once a stakeholder exists, the next problem is that there's almost never just one. Different people in the room hear different things when you say "AI agent." The CFO hears cost and ROI and wants to know the payback period. The CTO hears technical feasibility and wants to know if anyone can maintain it. The VP of Customer Success hears user experience and wants to know if anyone will

actually use it. Legal hears risk exposure and wants to know what happens when it's wrong. None of those questions is wrong. All of them point at a different facet of the same project.

If you can only articulate value in one of those languages, your framing is incomplete. The Feynman Test applies here too, except now there are four non-experts, each with their own jargon to avoid. If you can describe what you're building only in technical terms, you haven't thought through the business reality, and the room will reject the project for reasons that look like politics but are actually missing answers.

Matt O'Mara, on the Value Driven Data Science podcast, has a name for the gap underneath this: the discipline disconnect. Finance, IT, and HR each run on their own vocabulary, and none of them fully understands what the others do. ([Value Driven Data Science, episode 50, 29:20 to 30:21](#)) He describes it as a problem. I read it as the opening. The person who can sit between those functions and translate one technical capability into each one's language is doing the part none of them can do for themselves. Framing is that translation work, which is why the practitioner who speaks four stakeholder languages is worth more than the one who only speaks model.

Two ideas sharpen how you do that translation. The first is a vocabulary gap another guest named on the same show: stakeholders talk in results, the outcome they want, while data scientists reflexively answer in solutions, the thing they would build. Answer a results question with a solution and you have already stopped listening. The second is Chris Argyris's contrast between unilateral control and mutual learning. Walk in with your answer and ask people to poke holes in it, and you have signaled the decision is made, so the conversation closes. Bring the problem you are actually trying to understand, and it opens, and the constraint that would have killed the project after launch surfaces while it is still cheap to hear.

Tell: you can name the technical win but not whose budget moves. **Cost:** the project produces interesting findings and no decisions, and the room rejects it for reasons that look like politics.

Repair: find the one person whose problem this is, and learn to state the value in their language, not yours.

The demo trap

In enterprises, demos create momentum. A working prototype generates excitement, budget, and headcount. Once a demo has been shown to leadership, questioning the approach starts to feel like questioning the people who championed it, which is a different conversation and a worse one.

First-principles thinking is hardest when the demo already looks good, and that's when you need it most. JP Morgan Chase had a working demo and every metric was green, and the question of whether anyone would change their behavior because of it only got asked after launch. Asking that question after the demo is expensive. Asking it before is the whole job.

Tell: "the demo killed, let's scale it." **Cost:** questioning the approach starts to feel like questioning the people who championed it. **Repair:** ask the Oracle Test question before the demo, not after.

The "go do your magic" trap

There's a specific enterprise failure mode that doesn't show up in demos because it never produces one. Call it the science experiment that never finds its problem.

It happens when a stakeholder hands the AI team an ambiguous mandate. *"We have access to all this data. Go see what you can find."* The team spends months finding interesting patterns. They present findings. The stakeholders nod appreciatively. Nothing changes, ever, and the project winds down. The trap is that the team confused finding something interesting with solving a problem that moves the needle. Those are not the same thing, and from the inside it's hard to tell them apart.

This trap predates AI. Genevieve Hayes, interviewing recruiter Joel Robinstein on her podcast, described the 2015-to-2018 hiring wave: companies bringing on data scientists with no data access, no warehouse, not even the Python packages installed, then blaming the hire when the magic didn't materialize. Robinstein's clients were telling him the same story. Same "go do your magic" mandate a decade earlier, same result. ([Value Driven Data Science, episode 7, 9:57 to 11:09](#))

A few phrases to listen for: "we just want to explore what's possible." "See what the data tells you." "The AI team is doing a science experiment on this." Each one is a signal that nobody has defined the business outcome yet. One question catches it: *what decision gets made differently because of what you find?* If nobody can answer that, the problem isn't ready for an AI team yet.

Fix it before you start. *"I understand you want to explore. Let's agree on what a successful finding looks like. What decision would you make differently if we found it?"* If stakeholders can't answer, the framing isn't done, and your first job is to help them frame, not to start working.

Rob Deutsch, on the same podcast, passed along a rule he'd heard in a talk by an EY consultant. In their framing workshops, nobody is allowed to talk about the data, how clean it is, what it costs, or whether it's even available, until the problem itself is fully articulated. ([Value Driven Data Science, episode 5, 6:55 to 7:40](#)) Whether that's EY policy or one consultant's habit, I can't tell you, and I don't think it matters. The rule sounds bureaucratic. It works because data talk is where teams hide from the harder question of what decision they're trying to change, and banning it early forces the problem to the surface before anyone can retreat into what's easy to measure.

Tell: "let's explore what's possible, see what the data tells us." **Cost:** months of interesting findings that change no decision. **Repair:** agree up front on what a successful finding looks like, and what decision it changes.

Go to the Work

You can't develop product sense from behind a desk. The HVAC team's breakthrough came from shadowing technicians, not from a stakeholder interview or a requirements document. Whatever your domain looks like, the equivalent exists. Sit with the support agents. Watch the analysts build their spreadsheets. Follow the clinician through their workflow. Heidi Araya, who builds AI for small businesses and described her process on the podcast, put the posture plainly: don't walk in as the expert with a solution, start by listening to the pain points. ([Value Driven Data Science, episode 62,](#)

16:01 to 17:20 and 24:17 to 24:59) The constraints that kill projects live in the field, not in the conference room.

Going to the work also catches the problem you've mislabeled. Douglas Squirrel, on the podcast, described the shape it takes: the team is sure the model is broken and keeps tuning it, and it takes a conversation with the customer to hear that the data is in there, they just can't get it out. ([Value Driven Data Science, episode 40, 58:54 to 59:08](#)) From a desk that reads as a model problem. In the room it's an interface problem, and no amount of retraining would touch it.

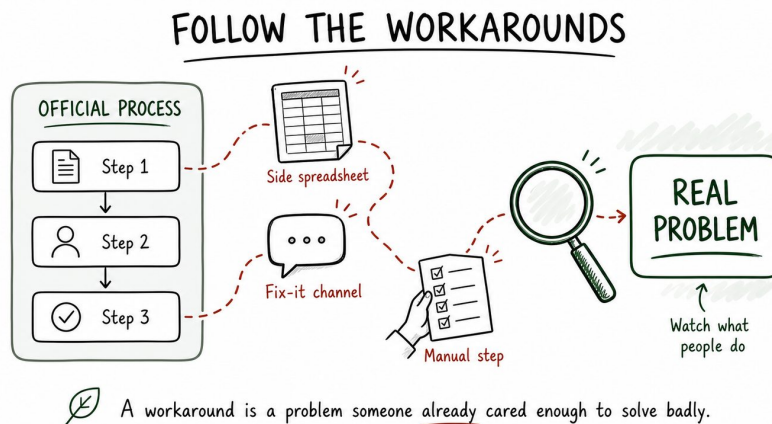


Figure: Workarounds expose the problems people already care enough to solve, even when the official process hides them.

What you're hunting for is the workaround. People rarely name their real problem, they route around it. Look for the side spreadsheet the official system can't produce, the Slack channel that exists only to fix what the tool gets wrong, the step everyone does by hand that the process diagram swears is automated. A workaround is a problem someone already cared about enough to solve badly, which means they've done the framing for you. What people say in the interview and what they do at their desk are different data, and the gap between them is usually where the project is. Your own systems leak the same signal: a report that gets generated every week and opened by no one is activity nobody needed.

The duct-taped binder is the most literal version of a workaround, and one is sitting somewhere in your problem too. Find it.

If this section reads as abstract because you haven't lived a project end to end yet, there's a companion piece for you: [The Life of a First AI Project](#). It walks the typical first project month by month, the data reckoning, the scope drift, the demo that lands flat, and shows where each of the book's tools would have changed the story. Skip it if you have the scars already.

Judgment, the part agents don't have

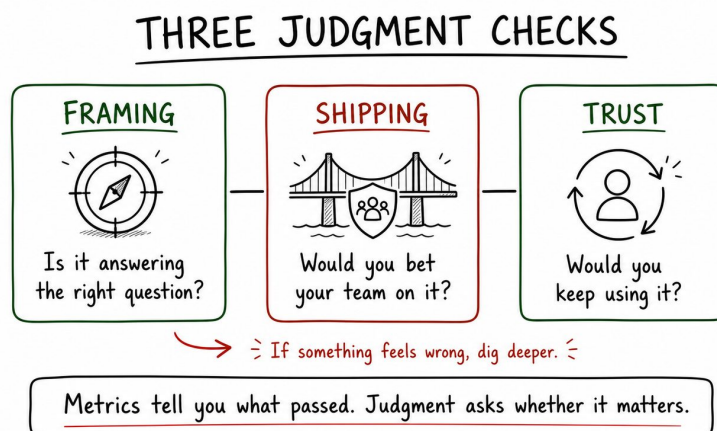


Figure: Metrics tell you what passed; judgment asks whether the result answers the right question, is ready to ship, and earns continued trust.

Agents keep getting better at producing output and no better at knowing whether the output is worth shipping.

That gap is judgment: the sense that the demo works but you would not bet on it in production, that the metric is green and the experience is still wrong, that a coding agent just confidently produced a function nobody asked for and nobody needs. None of that lives in the model.

Andrej Karpathy has been making a version of this argument repeatedly. When generation gets cheap, the bottleneck moves from "can I produce a candidate" to "can I tell which candidate is worth keeping." Judgment is the scarce input in an era where code, mockups, plans, and analyses are all a prompt away.

You make three checks the model can't. The first is on the framing: the output is competent, but is it answering the right question? Coding agents will happily build the thing you asked for when the thing you needed was different, and if something is off and you can't yet say what, the framing is usually the part that's off. The second is on shipping: the eval is green, but would you bet your team on it? Real systems have to survive the average day, not the demo day, and the sense that "this isn't ready" is usually right even when the dashboard isn't backing you up yet. The third is on trust: would you keep using this if you didn't have to? Trust is the leading indicator of adoption, and an experienced practitioner can feel it slipping before any dashboard catches it.

When something feels wrong, dig deeper, and don't override the feeling because the metrics say everything is fine. The metrics measure what you told them to measure. Your judgment measures the gap between what you measured and what you wanted.

Genevieve Hayes, on her Value Driven Data Science podcast, has the analogy I keep reaching for: a restaurant order-taker brings you what you ask for, a doctor diagnoses before treating. ([Value Driven Data Science, episode 54, 21:46 to 21:54](#)) Her version stops there; the rest is mine. An order-taker at

McDonald's builds exactly what you ask. You want a Big Mac, you get a Big Mac. That's fine when you already know what you want. A doctor works the other way. Walk in and ask for a heart bypass and a good one doesn't just scrub in, they diagnose first, because the thing you asked for might not be the thing you need. An agent is the order-taker. It builds the function you asked for, confidently, whether or not it's the function the problem needed. The practitioner is the doctor. Deciding what the real problem is before anyone builds the fix is the part that doesn't live in the model.

This is the meta-skill the rest of this book is teaching. The three thinking techniques, the GOATS Loop coming next, the diagnostic toolkit in Chapter 3, the decision protocol in Chapter 4. Every one of them is a way to convert judgment into evidence and check evidence against judgment. A practitioner who only trusts metrics ships the wrong thing on schedule. A practitioner who only trusts gut ships the thing they wanted to build instead of the thing the users needed.

Preview of the GOATS Loop

The next chapter introduces the GOATS Loop, the repeatable five-step method this book is built around. Goal, operating assumptions, alternatives, trade-offs, signals. Every other chapter runs through it.

The GOATS Loop applies to any AI system, but agent projects raise the stakes per question. A bad prediction gives you one wrong answer. A bad agent makes that decision repeatedly, autonomously, at cost.

The three techniques you just learned feed directly into GOATS. Inversion surfaces hidden failure modes, which is the work of step two, deconstructing assumptions. De-escalation challenges the approach itself, which is step three, alternatives. The Feynman Test exposes weak goal definitions, which is step one, goal. Chapter 2 goes deep on each step. For now, you've practiced the habits GOATS depends on.

The policy bot: the project this book follows. From here on, one project runs through every chapter. A support team wants an internal assistant that answers policy questions for its agents. An agent types "can a customer in Quebec get the annual fee waived?", the bot retrieves from about four hundred policy documents, and drafts an answer with citations. It's a composite of teams I've worked with, with the numbers made specific so we can do real arithmetic on them. Chapter 2 frames it. Chapter 3 diagnoses it when the eval and the support agents disagree about whether it works. Chapter 4 makes the call.

The chapter summarized

This book is giving you a method for framing AI projects. Before the method, though, you need the mindset to use it.

Framing is where projects fail. The model is rarely why an AI project fails. The framing is, and the frame most likely to be wrong is your own. Building used to be slow enough that a bad idea sat in a backlog until someone questioned it. Now an agent builds it in an afternoon, so the bad frame ships, runs, and bills you. The whole job moves upstream, to the questions you ask before anyone writes code.

Slow down the first answer. "We need an agent for this" is the fast, confident answer, and it's right just often enough to be dangerous. The work is catching it before you commit. Three moves force the slower read:

- **Inversion.** What are the top three ways this fails in production?
- **De-escalation.** How would I solve this with no AI, and would that be good enough?
- **Feynman.** Can I explain what it's for with no AI words?

Watch the room, not just the model. Four things go wrong in the meeting, not the code: the org chart designing your architecture, a project with no stakeholder whose budget moves, demo momentum replacing the framing question, and a "go explore the data" mandate that names no decision.

Go to the work. The constraints that kill projects live in the field, not the conference room. Sit with the people doing the job and find the workaround they already built, the duct-taped binder. It points at the problem they actually care about.

Bring this mindset to Chapter 2. GOATS is where it becomes a repeatable method.

Chapter 2: Framing the Agent Task

A repeatable method for framing any AI problem.

Two loops, not one

Most AI teams live inside the inner loop. Build the model. Run evals. Tune prompts. Improve retrieval. Measure precision and recall. There's a lot of important work in there and a lot of excellent courses and tools dedicated to it. We're not going to spend much time on it, because it isn't where most agent projects fail. Even inside that inner loop, the model is a smaller part than teams expect. MongoDB's engineering team put it plainly in 2026: the LLM call is the smallest piece of a production agent, and the harness around it, memory, orchestration, security, observability, and evals, is what decides whether it survives. (MongoDB)

This book is about the other loop, the one that sits above the model.

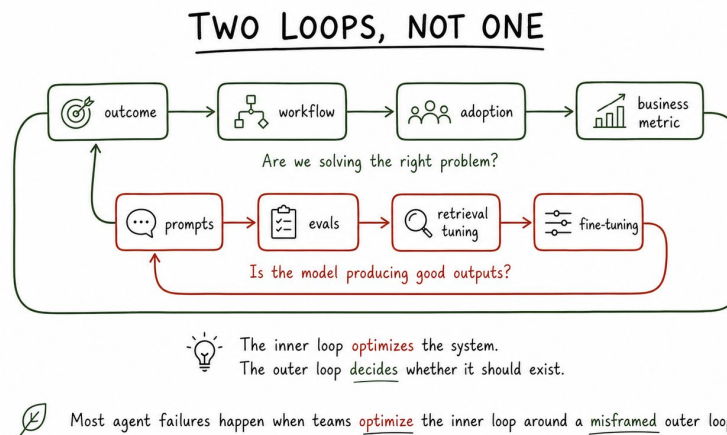


Figure: Teams usually over-invest in the inner loop when the outer loop is where the failure actually began.

	Inner loop	Outer loop
Question	Is the model producing good outputs?	Are we solving the right problem?
Work	Prompt engineering, evals, fine-tuning, retrieval tuning	Outcome engineering: defining what success means before you build
Failure mode	"Retrieval precision is 92% but users aren't being helped"	Doesn't happen if the outer loop is doing its job
Who does it	ML and AI engineers	The whole team. Product, engineering, business

The inner loop optimizes a system. The outer loop decides whether the system should exist in the first place.

The agent projects that fail almost never fail because the model was bad. They fail because the team built the wrong thing, then optimized the inner loop around a mis-framed problem. No amount of eval tuning fixes that. You can hit ninety-five percent precision on the wrong question and still ship a system nobody uses.

Outcome engineering is the practice of getting the question right before you optimize anything. That's what this chapter teaches. The GOATS Loop is the method.

The GOATS Loop

The framework is **GOATS**: **G**oal, **O**perating Assumptions, **A**lternatives, **T**rade-offs, **S**ignals. Five steps, run before you build, run again when something breaks.

For agent projects, you'll run it more often than you expect, because agent projects break in ways traditional ML systems don't.

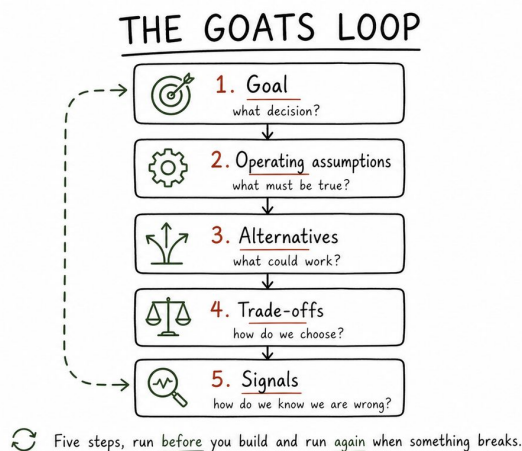


Figure: GOATS is the loop you return to when the current frame stops holding.

There are five steps, each producing a specific output, and each with a specific failure mode if you skip it.

Step	Question	Output	What breaks if you skip it
1. Goal	What decision are we supporting?	Business metric and success definition	You optimize the wrong thing
2. Operating Assumptions	What would have to be true?	Testable hypothesis list	You build on hidden false premises

Step	Question	Output	What breaks if you skip it
3. Alternatives	What approaches could work?	2-3 candidate approaches	You anchor on the first idea
4. Trade-offs	How do we choose?	Justified selection with rationale	You pick based on excitement, not fit
5. Signals	How will we know if we're wrong?	Stop criteria and leading indicators	You can't tell if it's working until it's too late

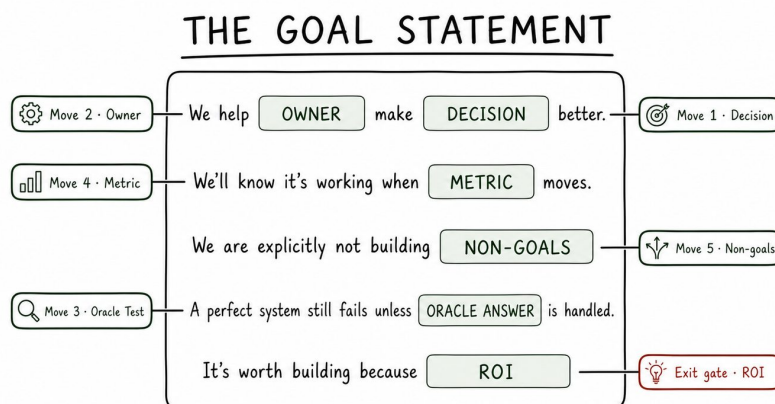
The rest of this chapter walks each step. I'll show what the step is, the diagnostic question that drives it, the failure modes I keep seeing in real teams, and what "done" looks like before you move on.

Step 1: Goal

You haven't defined the problem until you've defined the metric.

Step 1 is done when you can fill in one artifact. I call it the Goal Statement:

We help **[owner]** make **[decision]** better. We'll know it's working when **[metric]** moves. We are explicitly not building **[non-goals]**. If the system were perfect tomorrow, it would still fail unless **[what the Oracle Test surfaced]** is handled. It's worth building because **[the one-line ROI]**.



The Goal Statement is the output of Step 1; each blank is filled by one move, and an empty blank is the finding.

Figure: The Goal Statement is the output of Step 1; each blank is filled by one move, and an empty blank is the finding.

Six blanks, each filled by one move, with the exit gate at the end filling the last one. The sentence doesn't read in the order you fill it, which is why the figure tags every blank with the move that owns it. You name the decision and its owner first, then run the Oracle Test, then pick the metric, then write the non-goals, and the ROI gate closes it out. Fill the blanks in move order and read the sentence back

in sentence order. If a blank stays empty after its move, don't push forward. The empty blank is the finding.

Move 1: Name the decision

Most teams start by asking what the agent produces. Start instead with what decision gets made with this output.

What teams say	What they mean	The real question
"Build a chatbot for customer support"	Deflect tickets	"Which tickets can be resolved without a human, and how do we measure resolution?"
"Create an agent that summarizes documents"	Save reading time	"What decision does the reader make differently because of this summary?"
"Build a code review agent"	Catch bugs faster	"Which bugs matter? What's the cost of a missed bug versus a false alarm?"
"Build a churn model"	Keep customers from leaving	"Which customers are worth an intervention, and which intervention changes their mind?"

A summary that nobody reads is not better than no summary. A code review that flags everything is not better than a human glance. The output exists to support a decision, and until you can name the decision, you can't tell whether the output is working.

Toyota's Five Whys is the cheapest tool for getting from the stated ask to the real one. Ask why enough times and the surface request gives way to the actual constraint. Nicholas Kelly walked through a made-up version on Genevieve Hayes's Value Driven Data Science podcast, and the made-up part is fine, because the shape is what matters. Say people are leaving. Why is that role churning, why can't we keep it filled, and maybe it bottoms out at recruiters who aren't paid on the metric you're asking them to move. ([Value Driven Data Science, episode 70, 17:09 to 18:31](#)) No model would find that, because the answer is in how people get paid, not in the data. Five whys gets you to a decision someone has the power to change.

Naming the decision well changes what gets built. OpenAI's legal team is the clean example.

OpenAI's contract review agent. OpenAI's own legal team faced a scaling problem. Contract volume went from hundreds to over a thousand per month while headcount barely grew. They didn't build an autonomous agent to replace lawyers. They reframed the outcome: take the repetition out of contract review, keep experts firmly in control. The agent parses contracts overnight with structured reasoning and annotations. Humans review the structured output, not raw PDFs. The

human role shifted from data entry to judgment. The outcome was "scale contract processing without linear headcount growth," not "automate legal review." ([OpenAI](#))

Move 2: Name the owner

Naming the decision isn't enough. It has to belong to someone with the authority to act, and it has to move something a leader already cares about. Mark Stouse, on the Value Driven Data Science podcast, put the role plainly: the job is helping business people make better decisions, and meeting them where they are. ([Value Driven Data Science, episode 30, 14:23 to 14:43](#)) That is a different job from shipping models. In a separate episode, Gregory Lewandowski narrowed what executives care about to five things: increasing revenue, reducing costs, reducing risk, improving customer satisfaction, and improving employee satisfaction. ([Value Driven Data Science, episode 60, 7:25 to 8:08](#)) If your metric doesn't trace to one of those five, expect to struggle for a sponsor no matter how clean the model is.

A quick way to force all of this into one sentence comes from a workshop activity David Cohen described on the podcast, which he adapted from design thinking's "how might we" prompt. ([Value Driven Data Science, episode 82, 11:04 to 12:19](#)) Fill in the blanks: *how do we [achieve what outcome] for [which user] so that [what changes as a result]?* Three blanks, and you can't fake any of them. The first names the outcome instead of the feature. The second names the person whose problem this is. The third is the Oracle Test in disguise, because if you can't finish "so that," a perfect system changes nothing. If your team can't fill in all three in a single sentence, you don't have a goal yet. You have a project title. That sentence is the seed of the Goal Statement, and the remaining moves stress-test it.

Move 3: Run the Oracle Test

The most powerful single question in the GOATS Loop is this one: *if this agent was a hundred percent accurate right now, what would still cause it to fail?*

If your answer is short, you have a model problem and GOATS is straightforward from here. If your answer is long, you have a framing problem dressed up as a model problem.

What the Oracle Test surfaces	What it actually reveals
"Users wouldn't trust the output"	A trust problem, not a model problem
"The data changes faster than we can update it"	A freshness problem, not an accuracy problem
"Nobody would change their workflow"	An adoption problem, not a technology problem
"The wrong person receives the output"	A distribution problem, not a quality problem
The policy bot: "agents wouldn't send an answer they can't check"	A trust problem, which is why every answer carries its citation

A perfect model doesn't fix a trust problem. A bigger model doesn't fix a workflow problem. If a perfect version of your system wouldn't solve the problem, a better version won't either, and you're about to pour money into the wrong inner loop.

A worked example makes the trap concrete. Douglas Squirrel, on the podcast, described proposing to a client that a financial report should run faster, and learning from the people paying for the work that overnight was fine. Nobody was waiting on it. What they wanted was better charts. ([Value Driven Data Science, episode 40, 30:45 to 31:57](#)) Run the Oracle Test on the frame he started with and it fails on contact. A report that renders instantly, a hundred percent on the speed metric, still misses what the client valued, because speed was never it. He only found that out by asking. Optimize the metric you assumed and you ship a faster version of the wrong thing.

Churn prediction is the version of this I run into most, and it fails the test in a different way. A team builds a model to flag which customers are about to leave, gets the accuracy up, ships it, and the churn rate doesn't move. Run the Oracle Test before any of that and you can see it coming. A hundred percent accurate tomorrow gives you a perfect list of customers who are leaving and no idea what to do about a single one of them. What the team needed was the intervention: which offer, at what cost, keeps a given customer, and whether that customer is worth keeping at that price. Nobody had priced a retention offer or tested whether one works. Michael Schrage of MIT Sloan makes the same point in his Sloan Management Review writing on KPIs, that a number like churn is worthless until it's tied to a decision someone can act on, which means the hard part was never predicting churn but knowing what intervention, at what cost, would prevent it. The model was never the bottleneck, and a better model buys you a more precise list of people you still can't keep.

Two more angles on the same test live in The GOATS Loop in Practice, part of the course materials. Jeff Camm's lever test asks whether the person receiving the output can pull the lever it points at, because a perfect recommendation about something the stakeholder can't change is a pointless recommendation. Mark Stouse's decision-speed story is about an HP executive who never needed a high-confidence answer, just a faster one, which means a more accurate model was never going to help him.

Move 4: Pick the metric

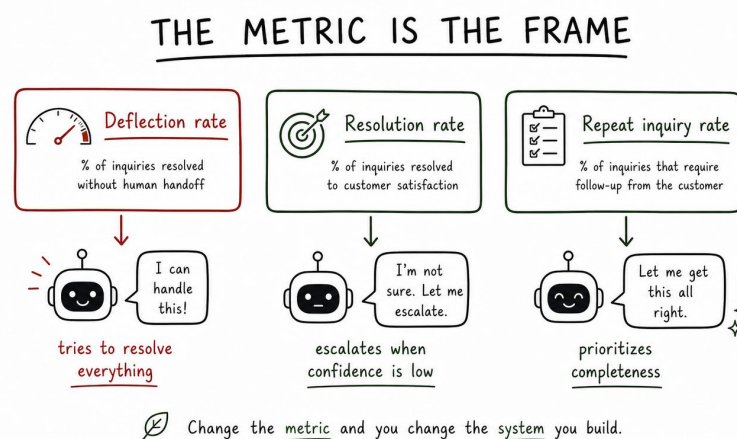


Figure: Choosing a metric is not neutral; it shapes the behavior the system learns to optimize.

The metric you choose determines the system you build. Two teams with the same technology and the same data can build entirely different systems depending on how they measure success. The metric is an architecture choice as much as a measurement choice.

Chris Dolman, on the podcast, made the point with a blunt example: "maximize sales" is never the real goal, because a human salesperson knows not to give the product away. ([Value Driven Data Science, episode 17, 17:27 to 17:57](#)) Add the rest of what a salesperson knows without being told: don't burn the brand, don't promise what we can't ship. The model knows none of that unless you put it in the metric. Hand an agent "maximize sales" and it finds the discount that books the sale and wrecks the margin, exactly as instructed. The constraints you never said out loud are still part of the frame, and the metric is where they have to live.

Klarna's AI assistant handled 2.3 million conversations in its first month, by the company's own announcement. The results they reported are the record. The deliberation below is my illustration: three metrics a team in their position could have picked, and the different agent each one would have produced. Klarna's announcement doesn't say they weighed these three, and I'm not claiming it does.

Deflection rate measures how many conversations the agent kept away from a human. Optimizing for deflection produces an agent that tries to resolve everything, including the cases it should escalate. It "handles" the conversation by not escalating, regardless of whether the user got an answer. The number looks great on the dashboard. The user finds out the agent didn't help, and now they're stuck.

Resolution rate measures whether the conversation solved the user's problem. Optimizing for resolution produces an agent that escalates when its confidence is low, because escalation is sometimes the right resolution. The number captures real outcome, not just routing.

Repeat inquiry rate measures how often a user comes back about the same issue. Optimizing for it produces an agent that prioritizes completeness over speed, because a partially answered question is just a future ticket. Answer fast but incomplete and the user is back next week.

Metric	What it optimizes	System behavior
Deflection rate	Keep humans out of the loop	Agent tries to resolve everything, even when it shouldn't
Resolution rate	Actually solve the problem	Agent escalates when confidence is low
Repeat inquiry rate	Solve it the first time	Agent focuses on completeness over speed

What Klarna reported lines up with the resolution column: human-level CSAT, repeat inquiries down twenty-five percent, resolution time from eleven minutes to under two. Those are company-reported numbers and I can't verify them independently. What I can say is what a deflection target would have bought: an agent that "handles" more conversations and resolves fewer of them, with a headline number that looks great while the outcome gets worse. That's the danger of picking a metric badly. The system optimizes against your business while looking like it's winning.

Coding agents are the clearest current version of the same trap. Most teams count tokens, acceptance rate, and PR throughput, all activity metrics, and all of them go up the moment you turn the tool on. An NBER study of more than five hundred thousand GitHub developers found autonomous coding agents raised commit volume by two hundred and forty percent, but only thirty percent for actual releases ([Demirer, Musolff, and Yang, NBER w35275](#)). A 2025 METR trial put experienced developers nineteen percent slower with AI while they believed the tool had sped them up by twenty percent ([METR](#)). Pick commits and the tool looks transformative. Pick releases and it looks useful but ordinary. The fix is the same move Klarna made: pair every speed metric with a quality metric and measure outcome, not activity. The full numbers, and the net-time-gain calculation for making the ROI case honest, are in The GOATS Loop in Practice, in the course materials.

One more test for the metric you picked: the Principle of Least Astonishment. The outcome a user expects and the outcome the system produces have to match. When they don't, users stop trusting the system, no matter how technically correct the output is. (See [Principle of Least Astonishment](#) for the visual.)

Move 5: Name the non-goals

Defining the goal includes saying what the goal is not. An unstated non-goal is an invitation. Ask an agent to "build a robust auth API" and it adds OAuth2, caching, and rate limiting you never wanted, because nothing told it to stop. The Chapter 1 contrast turned on exactly this. The engineer who shipped in an afternoon was the one who wrote "out of scope: OAuth2, multi-factor, social login" into the task. The boundary did as much work as the goal.

Agents raise the cost of skipping this. When building was slow, an unscoped request sat in a backlog. When an agent generates anything in minutes, an unscoped request gets built, and now you maintain it. Writing the non-goal costs a sentence. Skipping it costs the sprint you spend un-building what you didn't want.

Write down three things you are explicitly not doing, and why. Not multi-currency yet, we have one market. Not real-time, the daily batch is fine. Not the long tail, the top ten intents cover eighty percent. Each line is a decision you made on purpose instead of one the model makes for you by default. Soma Kesara's framing is the sharp version: good framing names what you don't want, not just what you do.

The exit gate: is it worth building?

Before you build, translate the outcome into business terms, because the translation tells you whether the problem is worth solving in the first place. A surprising number of agent projects don't survive this step, which is why you do it before writing code instead of after.

$$\text{Value of agent} = (\text{decisions improved} \times \text{value per decision}) - (\text{cost to build} + \text{cost to run} + \text{cost of errors})$$

The math has five inputs, and each one is where teams get the calculation wrong.

Decisions improved is the volume the agent gets to act on. Teams overestimate this constantly. The slide deck says "we process fifty thousand tickets a month, the agent will handle all of them." Production reality says fifteen percent of tickets fit the criteria the agent can decide on. The denominator collapses, and the ROI math collapses with it. Count from a real production sample, not from the volume the team imagines.

Value per decision is what each decision is worth in dollars. Not every decision is worth the same amount. A correctly classified spam ticket is worth pennies. A correctly approved high-value loan is worth thousands. Treating decisions as fungible and multiplying volume by an average is how teams arrive at imaginary ROI. Segment by value, estimate per segment, add them up.

Cost to build is engineering time plus infrastructure. The line item teams forget is integration. Wiring the agent into existing CRM, ticketing, billing, and identity systems is usually two to three times the cost of the model work itself. Counting only model and prompt time understates build cost by an order of magnitude.

Cost to run is API calls, compute, observability, monitoring. The trap is treating these as fixed when they scale with usage. A pilot at five hundred queries a day costs almost nothing. The same architecture at five hundred thousand queries a day is the angry email from finance. Run the numbers at production scale, not pilot scale.

Cost of errors is what happens when the agent is wrong, multiplied by how often the wrong thing ships. This is where agents diverge hardest from traditional systems, and it is the row that kills the most ROI calculations.

Component	How to estimate	What people miss
Decisions improved	How many per week or month?	Overestimating volume
Value per decision	Time saved? Revenue gained? Risk reduced?	Assuming every decision has equal value
Cost to build	Engineering time, infrastructure	Underestimating the integration work
Cost to run	API calls, compute, monitoring	Forgetting that costs scale with usage
Cost of errors	What happens when the agent is wrong?	The asymmetry. One bad agent action can cost more than a thousand good ones

The agent-specific trap is in that last row. A human making a mistake processes one case wrong. An agent making a systematic mistake processes thousands of cases wrong before anyone notices. The error cost scales with throughput, and that scaling is the thing that turns a profitable agent into a six-figure incident report.

Put real numbers on it and watch the gate bite. Every deck proposes some version of the support-ticket agent: fifty thousand tickets a month, ten minutes of human time each, so the agent "saves" eight

thousand hours. Production shrinks that on contact. Only about fifteen percent of those tickets are ones the agent can close on its own, so the real volume is seventy-five hundred a month. And the ones it can close are the simple, high-volume tickets worth a couple dollars of saved handling each, not the twenty-dollar complex cases. The upside lands nearer fifteen thousand dollars a month than the number on the slide. Then subtract the integration into ticketing and identity, the run cost once traffic is real, and the single bad auto-reply to a regulated request that costs more than a thousand good ones saved. The project that looked like an obvious win is a coin flip, and a calculator found that out in an afternoon instead of a quarter into the build.

The agent-specific questions in the gate

Traditional ML projects didn't have to ask these, and agent projects do. Two of them are versions of work you've already done: what decision is the agent making, and if you can't name it cleanly the scope is still vague, so you're back at Move 1. What's the cost of getting it wrong once versus a hundred times, which is the error-cost row you just filled in, read at two different scales. One bad call gives you the per-instance loss. A hundred bad calls reveal systematic risk that never shows up per instance.

The two new ones belong here. Is the agent the right decision-maker at all? Some decisions should stay with humans even when the ROI math clears, and a perfect agent making a decision that should have been a human's is still the wrong system. And can the outcome be objectively verified? Verifiability determines how much autonomy you can safely grant, which makes it an architecture question, so the work on it lives where the architecture gets chosen, in Step 3. At this gate, just notice whether the answer is yes or no.

The policy bot: the Goal Statement. We help support agents decide what to tell a customer about a policy, and the support director owns the number, because the agents' handle time and the policy team's interrupt load both move if it works. We'll know it's working when answer accuracy on the weekly production sample is above eighty percent and routine policy questions stop landing on the policy team. We are explicitly not building a customer-facing bot, not touching account data, and not deciding exceptions. If the system were perfect tomorrow, it would still fail unless agents trust the answer enough to send it, so every answer carries its citation. It's worth building because the policy team spends about a third of its week answering the same questions by chat, and the error cost is bounded: an agent reads the citation before anything reaches a customer.

Before you move on

Read your Goal Statement out loud with every blank filled in.

We help **[owner]** make **[decision]** better. We'll know it's working when **[metric]** moves. We are explicitly not building **[non-goals]**. If the system were perfect tomorrow, it would still fail unless **[oracle answer]** is handled. It's worth building because **[ROI]**.

Blank	The check
Decision	Is it a decision someone makes, not a thing the system produces?
Owner	Does this person's budget or headcount move if it works?
Metric	What behavior does this number buy? Is speed paired with quality?
Non-goals	Did you write down at least three, with reasons?
Oracle answer	If it were a hundred percent accurate, what would still fail, and who owns that?
ROI	Does the value survive the error-cost row at production scale?

If any blank is hand-wavy, Step 1 isn't done. Tighten the answer or back up to the Oracle Test. Step 2 is only worth running once Step 1 is solid.

Step 2: Operating Assumptions

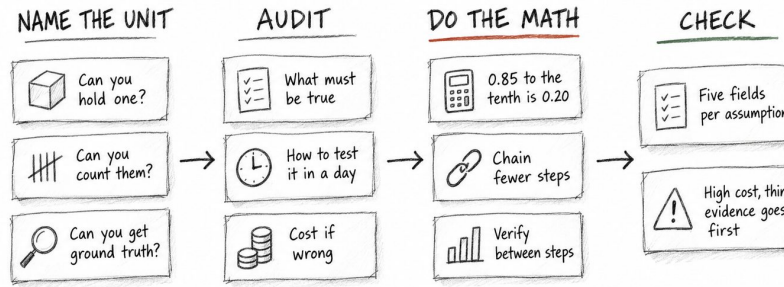
Most projects fail here, and usually because the assumptions were never stated, not because they were wrong.

Step 1 gave you the question. Step 2 answers it in two moves, in order. First you pin the unit the system operates on, one decision at a time. Then you list what would have to be true for your approach to that unit to work, and write each assumption so you can test it in a day.

This is the step teams skip the most often, because it feels obvious in retrospect and uncomfortable up front. You have to write down the things you've been assuming, then test them. The discomfort is the point. An assumption that feels too obvious to state is exactly the assumption that will destroy the project later.

This is also where AI work departs from software. Bill Schmarzo put it sharply on the podcast: software defines its requirements up front, data science discovers them. ([Value Driven Data Science, episode 53, 36:05 to 36:23](#)) You can't write the full spec before you build, because half of what you need to know only shows up once the system meets real data and real users. John Thompson, on the same panel, had the jab at forcing it into a rigid process: "agile is a babysitting methodology." ([episode 53, 34:30 to 34:50](#)) Plenty of AI teams run fine on agile, and Schmarzo's distinction holds either way. An AI project is a discovery loop, so the honest move is to write your assumptions as things to test, not requirements to sign off, and to expect the list to change as you learn.

STEP 2: OPERATING ASSUMPTIONS

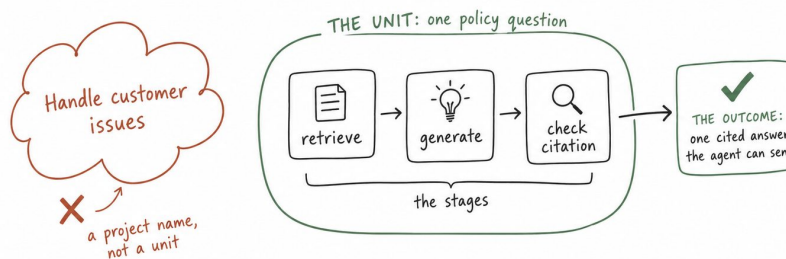


Hypotheses to test, not facts to assume.

Figure: Step 2 runs left to right. Pin the unit with three tests, audit what would have to be true, run the compounding arithmetic, and leave with five fields filled in per assumption.

The atomic unit

THE UNIT, THE STAGES, THE OUTCOME



Name the unit you can count, the stages inside it, and the outcome you grade.

Figure: Name the unit you can count, the stages inside it, and the outcome you grade. The slide usually names only the stages.

Before any assumption work, answer one question. *What does the system decide about, one at a time?*

This is the single most important question in Step 2. It forces you to identify the fundamental entity your system operates on, and most teams are surprised how hard it is to answer cleanly.

Three things get tangled here, so name them separately. The unit is the thing the system handles one at a time: one support question. The stages are what the system does to that unit: retrieve, synthesize, verify. The outcome is what you grade one unit on: a correct, grounded answer the support agent can send. Teams write the stages on the slide and call it the unit, and then nobody can say what one of them looks like or how to tell if it came out right.

The parallel to SOLID, specifically the Single Responsibility Principle, holds at the stage level: each stage should do one thing, just like each class should have one reason to change. When one stage

bundles multiple responsibilities, the system becomes as hard to debug as a god-class. (See [SOLID Principles](#) for the visual.)

System	What the slide says	The unit	The stages inside it	The outcome you grade
Customer support agent	"Handle customer issues"	One support ticket at triage	Classify, retrieve the policy, draft	A draft a human can approve without edits
Code review agent	"Review pull requests"	One potential issue in one file	Locate, explain, cite the evidence	A flag the reviewer acts on
Research agent	"Research this topic"	One candidate source	Find, extract the claim, assess credibility	A claim with a credibility call a human agrees with

"Handle customer issues" is what you write on the slide. It gives the agent no structure. Naming one ticket as the unit, three stages inside it, and one outcome to grade does. When something breaks, you know which stage failed. When something is slow, you know which stage to optimize. The atomic unit determines whether your system is debuggable.

Three tests for the atomic unit

A good atomic unit passes three tests.

First, can you hold one in your hand? If your unit is "customer satisfaction," you can't. If it's "one support ticket at the point of triage," you can. Concreteness is the bar.

Second, can you count them? "User engagement" is uncountable, which is a useful tell that it's not a unit at all. "Sessions where the user completed their stated task" is countable and gets you somewhere.

Third, can you get ground truth for one? If you can't verify whether the system handled one unit correctly, you can't evaluate the system at all. Ground truth is the bar that keeps you honest.

If your unit fails any of those, it's still a project name, not an atomic unit.

Anthropic's three structural failure modes. At ODSC East 2026, Anthropic walked through three structural failure modes they kept seeing in agent projects. The first was the wrong unit of work. Their example: "Transform onboarding" has no owner, no input contract, no review gate. "Draft the KYC refresh letter" has all three. Same domain, two framings. The first is a project name. The second is an atomic unit, because someone owns it, the inputs are defined, and there's a checkpoint at the end.

Owner, input contract, review gate is the operational version of what the three tests give you abstractly. Use whichever lens sharpens your unit faster. If you can't name all three for your unit, it isn't ready yet.

The assumption audit

Pinning the unit was the first move. The second is the assumptions: list what would have to be true for your approach to work. These are testable hypotheses, and the agent era surfaces a few that traditional ML didn't have.

Hidden assumption	Why it's dangerous	How to test
"The LLM can reliably do this step"	You assumed capability without testing	Run fifty examples through the step in isolation
"Context from step N is sufficient for step N+1"	Information gets lost or garbled between steps	Check what actually arrives at step N+1
"The user's input is unambiguous enough"	Real users are vague, contradictory, incomplete	Test with real user inputs, not curated examples
"The task is bounded"	Agent might loop, branch, or escalate indefinitely	Define what "done" means for every path
"Errors in one step don't cascade"	They almost always do	Trace an error in step 1 through to the final output

The trick is to write assumptions in a form that can be tested in a day, not a quarter. "The LLM can do this step" is testable. "The system will be reliable" isn't, because reliability is a property of the whole, not a hypothesis you can run fifty examples through.

Teresa Torres's *Continuous Discovery Habits* decomposes assumptions one level deeper. Run your list against her five categories: value (do users actually want this), discoverability (will they find it), behavior change (will they use it), technical feasibility (can we build it), and trust (will they rely on it). The atomic unit work above is one slice of the technical feasibility category. The other four are easy to forget in agent projects, where the assumptions teams default to listing are mostly about the model and rarely about the user. ([Cleverx](#))

The math that kills agent projects

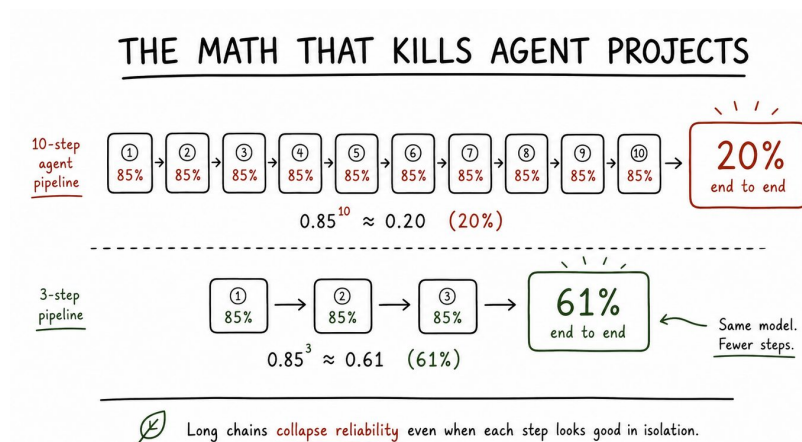


Figure: Same model, same per-step accuracy. Chain length alone moves end-to-end reliability from 20 percent to 61 percent.

The last assumption on that list is the one teams wave off, so here is the arithmetic on it. Every step you chain is a step the whole thing can fail at, and the numbers are worse than intuition suggests. Say each step in your pipeline gets its part right eighty-five percent of the time when it's handed correct input, and that one step's failure doesn't change the odds for the next. Chain ten of them:

$$0.85^{10} = 0.20$$

Eighty-five percent per step. Twenty percent end to end. Two conditions sit inside that multiplication, and both are worth stating out loud. The per-step number has to be conditional on good input, because a step that's fed garbage fails at a different rate than the one you measured in isolation. And the steps have to fail independently. If one bad input tends to break every step after it, the math gets worse than the table, not better.

Steps	Per-step accuracy	End to end
3	85%	61%
5	85%	44%
10	85%	20%
3	95%	86%
5	95%	77%
10	95%	60%

A pipeline that does ten things, each well, fails at the system level four times out of five. The instinct is to go improve each step, and the table shows why that's the expensive road: lifting every step from eighty-five to ninety-five percent buys you sixty percent end to end, which is roughly what dropping from ten steps to three gave you for free. Length is the cheaper lever.

You can pull it three ways: chain fewer steps, which usually means one call is doing work you split across three for no reason; make some steps deterministic, because a lookup or a validation rule fails only when its rule or its input is wrong, which is a rate you can measure once and mostly take out of the multiplication; or verify between steps, so an error is caught where it happens instead of travelling downstream disguised as an input. Deterministic doesn't mean perfect. A rule can encode the wrong policy and a lookup can hit a stale table. It means the step fails for reasons you can audit instead of reasons you can only sample.

Which shape you choose, one call or a pipeline or an agent, is a Step 3 question. What Step 2 owes you is the number, because "errors in one step don't cascade" is an assumption that fails without warning, and this is what it costs when it does.

The hidden-assumption pattern

Teams discover hidden assumptions the hard way, every time, and the story usually goes like this:

"We assumed [X] was true." → "It wasn't." → "We didn't find out until [expensive moment]."

The expensive moment varies. It might be the demo that didn't survive contact with real users. It might be the bill at the end of the month. It might be the post-launch retrospective where everyone agrees, very gently, that nobody asked the question.

What you assumed	What was actually true	When you found out
Users would provide clear instructions	Users give vague, contradictory inputs	After building the full pipeline
The LLM would follow the system prompt reliably	It drifts, especially on edge cases	After deploying to real users
Tool calls would succeed	APIs time out, return errors, change formats	After the demo went well
The agent would know when to stop	It loops, retries, or escalates indefinitely	After a \$47K API bill

The fix is to write them down. State the assumption, test it in isolation, and verify before you build. It's boring, deliberate, cheap, and the difference between a project that ships and a project that lives in retrospectives.

The assumptions checklist

For each assumption you list, fill out the same five fields.

Assumption	Evidence for	Evidence against	How to test	Cost if wrong
"LLM can classify tickets into 5 categories"	Works on examples	Haven't tested edge cases	Run 100 real tickets	Medium: misrouted tickets
"Users will provide enough context"	Internal testers did	Real users might not	Shadow mode with real traffic	High: bad responses
"Three steps is enough"	Covers 80% of cases	20% need more steps	Trace failures in the 20%	Medium: escalation rate

The "cost if wrong" column is the one teams skip. It's the column that tells you which assumptions deserve testing now and which can wait for a later iteration. If the cost is high and the evidence is thin, that's the test you run first.

The policy bot: the unit and the assumptions. The system decides about one policy question from one agent, one at a time. Three assumptions would hurt most if wrong. The answer to a question lives in one document, which the team tested by pulling thirty recent questions from the chat logs and finding a single source for twenty-six of them. The corpus is current, which nobody tested, because the policy team said it was. And agents will read the citation before sending, which a week of shadowing would settle. Two steps, retrieve and generate, so the compounding arithmetic is kind here. The risk is in the first assumption, not the chain length.

Before you move on

Check	Question
Atomic unit defined	What does the system decide about, one at a time?
Compounding risk assessed	How many steps? What's the end-to-end reliability?
Assumptions listed	What would have to be true for this to work?
Hidden assumptions surfaced	What are you assuming about users, data, tools?
Each assumption testable	Can you verify it before building?
Perfect-unit check	If every unit came back perfect, would the goal move? If not, the unit is wrong, not the model.

If your atomic unit is still vague or your assumptions still feel "obvious," go back. Step 3 will multiply whatever's wrong here.

Step 3: Alternatives

The point of this step is to avoid anchoring on the first approach.

Most teams generate one alternative, fall in love with it, and spend the rest of the project defending it. Step 3 is the deliberate interruption of that pattern. It costs maybe an hour. It saves months.

Three decisions, in order

Step 3 is three decisions, made in order. Do you need AI at all? If AI, how much autonomy does it get? And for each step the system performs, does the model already know how to do it? Each decision shrinks the space the next one has to search, which is why the order matters.

Decision 1: Do you need AI at all?

Rules, heuristics, and lookup tables solve more problems than most teams admit. If the logic can be expressed as "if X then Y," you don't need a model. You need an if statement. (This is YAGNI applied to AI: don't add capability until the simpler approach has been proven insufficient. See [YAGNI](#) for the visual.) And if you do need a model, it doesn't have to be a new one. Chris Dolman, by his own account on the podcast, got two years of value out of plain word-counting, and made the case for "boring AI": if a fifty-year-old technique still solves the problem, use it. ([Value Driven Data Science, episode 17, 51:29 to 52:54](#)) The newest model is a default, not a requirement.

Three problems wearing the same clothes

Before you answer the build question, sort the problem. Take the Oracle Test answer back out and read it. The things that would still fail if the system were perfect today land in three piles, and the pile you land in tells you what you are building.

If a perfect system would fix it, you have an AI problem. The judgment is real, the inputs carry enough signal to make the call, and no reasonable set of rules covers the long tail. That is the case where a model is worth building.

If a perfect system would land in an inbox nobody reads, you have a workflow problem. The prediction was never the bottleneck. Somebody downstream has to act on it, and either the handoff does not exist, the person does not trust the output, or their incentives point somewhere else. A churn model that scores every account nightly does nothing if the CSM team is already at capacity and cannot call anyone new. Fix the handoff first, then find out whether you still need a model.

If a person holding your exact inputs could not do the task, you have a data problem. I check this one first because it is cheap. Hand a domain expert fifty examples with only the fields the model will see and ask them to make the call. If they cannot, more model will not help you. You need different features, a different source, or a different target. Write down how well the expert does while you are there, because that number is your baseline for the rest of the project.

Most projects are two of these at once. An invoice-matching project that looks like an AI problem is often a data problem in the vendor master plus a workflow problem in who is allowed to approve an

exception, with a thin slice of genuine judgment in the middle. Naming the split is what keeps a team from spending two quarters improving a model that was never the constraint.

Sort your own Oracle Test failures into three columns, AI, workflow, and data. If most of them land outside the AI column, the model is not your next piece of work.

Three ways to attack any problem

Andrew Gelman's framing is useful here. There are three fundamentally different ways to answer any question.

Approach	What you do	What it reveals
First-principles reasoning	Reason from assumptions, build a simple model	What you would expect to see. Catches bugs in your thinking before you touch data.
Purely empirical	Get the data, count what happened	What actually occurred. The simplest possible answer.
Statistical modeling	Fit a model, estimate parameters	Richer understanding: effect sizes, uncertainty, counterfactuals.

Most teams jump straight to approach three: build the complex system. The simpler approaches come first for a reason. First-principles reasoning catches logical errors. The empirical baseline sets the feasibility bar. The complex approach builds on both.

Apply the same pattern to agent projects. Before building the complex pipeline (approach three), ask what back-of-envelope reasoning tells you to expect (approach one, the Oracle Test) and what the simplest version shows (approach two, the baseline). Only then does a more sophisticated approach earn its complexity.

Teresa Torres's Opportunity Solution Tree is a visual companion to this step. The OST maps a desired outcome down through opportunities (customer needs and pain points) and into solutions (the things you might build). It's stronger than the Automation Spectrum at making the problem space visible before you commit to a solution. The Automation Spectrum is opinionated about the answer (start left, move right when forced). The OST is opinionated about the question (what outcome are we chasing). ([Product Talk](#))

Decision 2: How much autonomy?

This is the question that separates the agent era from what came before, and it's the one teams answer reflexively rather than deliberately. The right level of autonomy depends on how structured the task is, and the structure determines architecture.

The automation spectrum

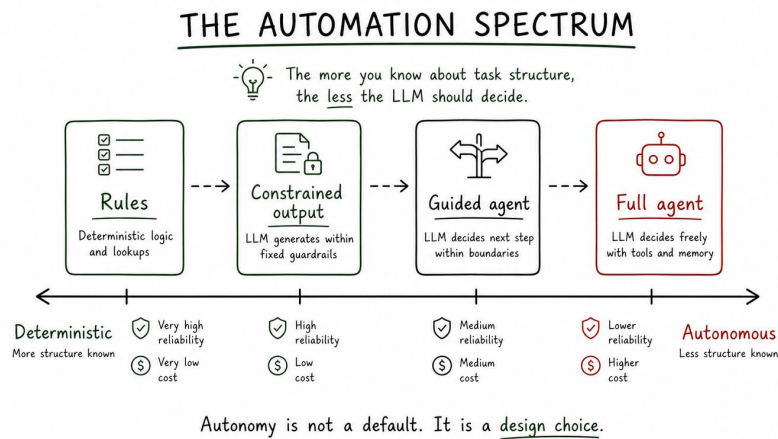


Figure: Autonomy is a deliberate architectural choice, not the default setting for every AI problem.

Most agent decisions live somewhere on this line.

Level	What you decide	What the LLM decides	Reliability	Cost
Rules	Everything	Nothing	Very high	Very low
Constrained output	The structure and constraints	Content within those constraints	High	Low
Guided agent	The goal and the guardrails	How to achieve it, within limits	Medium	Medium
Full agent	Only the goal	Everything else	Lower	Higher

The principle is simple. The more you know about task structure, the less the LLM should decide. (Related: KISS. The spectrum is a KISS heuristic for agent architecture: add autonomy only when the simpler level can't do the job. See [KISS](#) for the visual.)

One more lever rides on top of the spectrum. Verifiability tells you how much of what the model decides you can leave unchecked: an output a compiler, a schema, or crisp acceptance criteria can catch is safe to push right, while an output where "better" is a judgment call needs a human gate no matter how capable the model is.

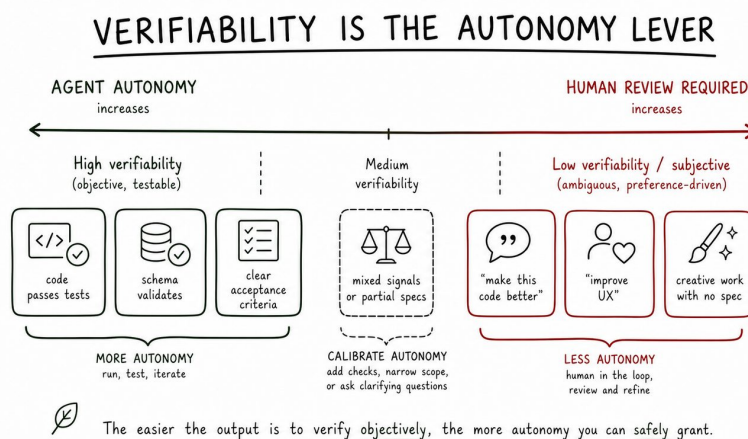


Figure: Autonomy is earned by checkability. Engineering the output to be verifiable is usually cheaper than moving to a bigger model.

That is the shape of the autonomy decision. The deep version lives in The Alternatives Guide, part of the course materials, under "The autonomy spectrum in practice": verifiability in detail, how to design the escalation path so humans handle exceptions instead of approving everything, four real teams that moved along the spectrum in both directions, and the signals for which way to move.

Decision 3: Established, retrievable, or teachable?

The spectrum told you how much autonomy the agent gets. The third decision is one traditional ML rarely had to ask: *does the model already know how to do this?*

In traditional ML, you always trained on your data. The model arrived empty. With LLMs, the model arrives with vast general knowledge but with gaps in specific places, and recognizing where those gaps live changes your architecture.

Type	What it means	Architecture implication	Example
Established knowledge	The model already knows how to do this	Prompt it. Minimal scaffolding.	Writing code in a popular language, summarizing text, general reasoning
Retrievable knowledge	The information exists but the model doesn't have it	Retrieval: RAG, tool use, context injection, API calls	Company policies, product catalogs, customer history, legal documents
Teachable skills	The model doesn't know the <i>process</i>	Teach it: few-shot examples, fine-tuning, structured workflow prompts, explicit step-by-step instructions	Your company's specific code review checklist, a domain-specific diagnostic workflow, a regulated approval process

A team building a support agent might assume the LLM "knows" their product. It doesn't. Product knowledge is retrievable, and without retrieval the agent will hallucinate confidently. A different team might assume the LLM can follow their twelve-step compliance review. It can't, not reliably, without

being taught the process. Compliance review is a teachable skill, and treating it as established knowledge is what produces the agent that confidently approves things it shouldn't.

The diagnostic question is short. *For each step your agent performs, is the model drawing on established knowledge, retrievable knowledge, or a skill it needs to be taught?* That single answer determines your architecture more than the autonomy level does.

The hard case is knowledge that isn't written down anywhere. Mudassar Iqbal, on the podcast, pointed at it: tacit knowledge, the judgment that lives in an expert's head and never made it into a system. (Value Driven Data Science, episode 28, 4:31 to 5:13) It's what makes the retrievable-versus-teachable call harder than it looks. You reach for RAG assuming the knowledge exists somewhere to retrieve, then discover the senior technician never wrote down why he checks the compressor first, the way Mike from 2009 never wrote down the HVAC fix in Chapter 1. Before you pick RAG, prompt, or fine-tune, ask whether the knowledge has been captured or is just assumed to exist. If it lives only in people's heads, your first job is getting it out of their heads and into something a system can read.

If the model needs...	Don't build...	Build...
Established knowledge	Complex retrieval pipelines for things the model already knows	Simple prompts with verification
Retrievable knowledge	A bigger model hoping it memorized your docs	RAG, tool use, or context injection
Teachable skills	A vanilla agent and hope it figures out your process	Structured workflows with explicit instructions, or a fine-tune

This is one of the cleaner differences between agent-era framing and traditional ML framing. In traditional ML, "the model needs to learn this" always meant "collect more training data." With agents, you have three distinct strategies, and picking the wrong one wastes months.

Ramp is a clean example of getting the retrievable call right, once they fixed what they were retrieving against.

Ramp: stop patching the taxonomy, retrieve against a standard one. Ramp classified every business through a home-grown web of industry categories, stitched from third-party data, sales entries, and what customers typed about themselves. The same kind of business landed in different buckets depending on which source the data came from, and nobody could explain the result or pass a compliance check on it. The instinct was to keep patching the taxonomy. The fix was to throw it out. They mapped businesses to the standard six-digit NAICS codes instead, embedded the code descriptions, retrieved the closest matches, and let two guarded LLM calls pick the final code, logging every intermediate step so a person could audit why a business got the code it got. A

maintenance problem became a retrieval problem. The assumption that a proprietary taxonomy was a feature turned out to be the thing that broke. ([Ramp](#))

Your slate: three candidates for one unit

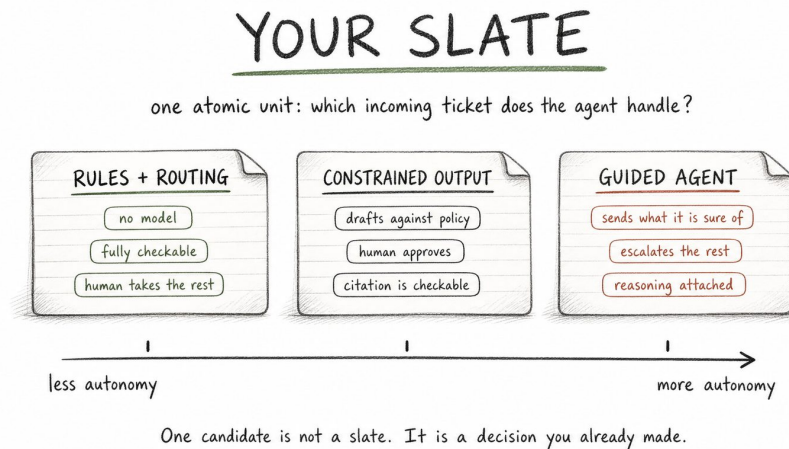


Figure: Three candidates for the support-ticket unit, sitting at three different points on the automation spectrum. Step 4 scores exactly these.

Step 3 is done when the slate holds two to four candidates that are different systems rather than variations on the one you already wanted to build. For the support-ticket agent, that means rules plus routing (no model, fully checkable, humans take the rest), constrained output with human approval (drafts against a retrieved policy, citation is checkable), and a guided agent with escalation (sends what it's confident about, escalates the rest with its reasoning attached). Different spectrum positions, different knowledge needs, different human involvement. If your three candidates all sit at the same point on the spectrum, they're the same system with different prompts.

Document your rejected alternatives

Document the alternatives you didn't pick, and write down why, because constraints change.

A dollar-per-query solution that was too expensive last year might cost five cents today. Long-context models didn't exist when you chose RAG. What was "too slow to ship" for v1 might be the right investment for v2. Your rejected alternatives are future pivot options, and a written rationale is what makes them rediscoverable later.

One slide: the Value Opportunity Map

The Value Opportunity Map distills the framing work onto a single slide, capturing the use case, business impact, workflow context, and the alternatives being weighed. The forcing function is compression. If you can't fit your framing on one page for a stakeholder, it isn't sharp yet. Variants exist from SAP's Business AI Explore, INNOQ's AI Opportunity Mapping, and the Design Sprint

Academy, and they share the same structure: a one-page comparison that makes the alternatives legible to people who weren't in the framing conversation.

Warning signs

Warning sign	What it means	What to do
Only one alternative	You anchored on a solution	Force yourself to generate one more. Ask "what would we do if this approach didn't exist?"
All alternatives are agents	You may be solving an engineering problem with autonomy	Add a rule-based or pipeline alternative
No non-AI option	You skipped the "do we need AI?" question	Include "human-only" or "simple rules" as a baseline

The policy bot: the slate. Three candidates, at three points on the spectrum. Keyword search over the policy library plus the FAQ template the team already uses: no model, fully checkable, and the baseline the others have to beat. One-shot retrieval with a drafted, cited answer: constrained output, one retrieval call per question. And iterative retrieval, where the model decides whether it has enough, searches again, and synthesizes across documents: a guided agent, with more cost and more to debug. The Oracle Test failures sorted mostly into the workflow column (will agents trust it), one into data (is the corpus current), and the question itself into the AI column, which is why a model is on the slate at all.

Before you move on

Check	Question
Multiple alternatives	Do you have 2-4 candidate approaches?
Non-AI option included	Did you consider rules, heuristics, humans?
Spectrum position justified	Where on Rules-to-Agent does each alternative sit, and why?
Verifiability weighed	How checkable is the output, and how much autonomy does that earn?
Knowledge/skills identified	For each step, is the model using established knowledge, retrievable knowledge, or a teachable skill?
Rejections documented	Did you write down why you rejected options?

This section is the overview. For the full menu of alternatives, see The Alternatives Guide in the course materials, which runs three parts you can read by modality: the ML lenses (classification, ranking, forecasting, and so on), the GenAI lenses (generation, retrieval, extraction, and so on), and the agent architectures, where the deep version of the autonomy decision now lives. For worked ML and GenAI examples of the GOATS Loop, see The GOATS Loop in Practice.

Step 4: Trade-offs

We don't optimize accuracy. We optimize value.

By Step 4, you have alternatives. None of them is going to be best on every dimension. The job here is to be honest about what you're giving up to get what you want, and to make that trade-off deliberately rather than by accident.

THE VALUE EQUATION

	SAID YES	SAID NO
ACTUAL YES	+\$50	-\$500
ACTUAL NO	-\$100	\$0

$$EV = TP \times \$50 - FP \times \$100 - FN \times \$500$$

If false negatives cost 5x false positives, optimize recall, not accuracy.

Figure: When false negatives cost far more than false positives, you optimize the expensive error, not raw accuracy.

The distributed-systems analog is the CAP Theorem: you can have consistency, availability, and partition tolerance, but only two at once. Agent trade-offs work the same way. Cost, latency, quality, and autonomy cannot all be maximized at once. Trying to maximize all of them is the most reliable way to land in the middle of all of them. (See [CAP Theorem](#) for the visual.)

The six dimensions

Every AI solution involves trade-offs across six dimensions. Skip any of them and you'll discover that dimension in production.

Dimension	The question	Why it matters
Cost	What does it cost to run?	A dollar-per-query solution dies at scale
Latency	How fast does it need to respond?	Real-time vs. batch changes everything
Quality	How good does it need to be?	95% accuracy might be worse than 80% with human review
Risk	What happens if it's wrong?	Medical diagnosis vs. movie recommendation
Maintainability	Can we debug it? Update it?	Black-box agents vs. staged pipelines

Dimension	The question	Why it matters
Time to ship	How fast can we get v1 out?	Sometimes "good enough now" beats "perfect later"

Agent-specific trade-offs

Agents add trade-offs traditional ML didn't have, and the one to put on the wall is blast radius. A wrong classification is one bad label. A wrong agent action can delete a database, send an email, or spend \$47K. Errors scale with autonomy, and that asymmetry between a wrong prediction and a wrong action is the single biggest reason the agent era is harder to ship in than what came before.

The full set, cost-explosion risk, trust boundaries, blast radius, and debuggability, plus two teams that paid these trade-offs deliberately (Shopify Sidekick and Harvey), is in the agent-architectures section of The Alternatives Guide, in the course materials.

The trade-off canvas

For each alternative from Step 3, score the six dimensions. The numbers don't have to be precise. They have to be comparable, and the honest version carries a row for what you don't know yet. Here is the policy bot's slate, scored the way the team scored it at kickoff.

	Keyword search + FAQ template	One-shot retrieval, cited draft	Iterative retrieval
Cost per query	Near zero	Low (an embedding call and one generation)	Roughly three times one-shot (second retrieval plus synthesis)
Latency	Instant	About four seconds	Ten seconds or more
Quality	Covers the questions the FAQ anticipated, roughly half	Expected to beat the template on phrasings the FAQ missed; thin on multi-document questions	Expected to be the only one that reaches multi-document questions
Risk	Low: every answer is a template, grounded by construction	Medium: an answer can read well and cite nothing that supports it; the citation gate is the control	Medium: same, plus more steps to go wrong
Maintainability	High: the policy team edits templates by hand	Medium: one index and one prompt	Lower: a loop to debug, and a stop condition to get right
Ship time	Already exists	Six weeks	A quarter
What we don't know	How much agents trust it today	How often questions span documents (tested on thirty, judged rare)	Whether the loop terminates reliably on real questions

The decision question is which trade-offs are acceptable given the business context from Step 1. An option's profile only makes sense once you know what the business values, and the last row is the one to read twice, because it's where the pivot in Chapter 4 was already written.

The cool-demo trap

AI systems are impressive in demos, agents most of all. Demos hide production realities, every time, in roughly the same ways, so score latency, cost per query, and failure recovery before the pilot grows.

Demo	Production
"Watch it handle this query!"	Does the user have thirty seconds for the system to think?
"It figured out the right approach!"	What does it cost per query at scale?
"It handled the edge case!"	If it fails on a different edge case, can you debug it?

A staged pipeline (search, then summarize) often beats a black-box agent on every dimension except cool factor. Cool factor is not a dimension you can optimize for, because cool factor doesn't survive contact with the second user.

Scale and latency as framing constraints

Sometimes a scale or latency requirement eliminates an approach entirely, and then you have a framing problem rather than a trade-off.

Constraint	What it kills	The reframe
p99 < 50ms	Multi-step reasoning, most LLM calls	Precomputation, caching, smaller models
10K QPS	Per-request model inference	Batch prediction, feature-store lookup
\$0.001/query budget	LLM-based solutions	Classical ML, rules, embeddings

If your constraint eliminates your approach, go back to Step 1.

When the trade-offs force a decision

Once you've scored your alternatives, one of two things happens. A clear winner emerges, or you're stuck between options with different strengths. If you're stuck, that's a signal you need more information, not more debate. Build the cheapest version you can test and let real usage break the tie.

Chapter 4 introduces the Shipping Matrix, a framework for deciding how much rigor a decision deserves based on reversibility and consequence. That's a decision-point tool, not a trade-off tool. For now, focus on understanding what you're trading.

The policy bot: the choice. One-shot retrieval won. It beats keyword search on phrasings the FAQ never anticipated, ships in six weeks instead of the quarter iterative retrieval would take, and costs a fraction per query. The team wrote down what it was accepting: questions whose answers

span two documents would get a thinner answer, and they judged those rare. Hold on to that sentence for Chapter 3.

Before you move on

Check	Question
Six dimensions considered	Cost, latency, quality, risk, maintainability, time to ship?
Agent-era trade-offs addressed	Cost explosion, trust boundaries, blast radius?
Scale and latency checked	Do production constraints force a different frame?
Decision justified	Can you explain why you chose this alternative?

Step 5: Signals

Before you build, define what would prove you're right, and what would prove you're wrong.

Signals are the part of GOATS that gets cut for time. Don't cut them. The whole reason you ran the previous four steps was to make a decision you can later check. Step 5 is where you write down how you'll check it, and it's the step that determines whether you can pivot quickly or keep grinding on a thing that isn't working.

(Gilb's Law: anything you need to quantify can be measured in some way that is better than not measuring it at all. A rough signal you defined upfront beats a "perfect" signal you never shipped. See [Gilb's Law](#) for the visual.)

Inner loop and outer loop, revisited

The two-loop distinction from the top of the chapter becomes concrete here. Your signals need to cover both loops or they aren't doing the job.

Inner-loop signals tell you whether the model is performing well: precision, recall, task completion, latency. Outer-loop signals tell you whether the system is solving the right problem: business outcomes, user behavior, real-world impact. Most teams define the inner-loop signals only, then are surprised six months in when the model is great and the project is dying. Step 5 is where you force yourself to define both.

Three types of signals

There are three types you need, and they're doing different jobs.

Success criteria. What does "working" look like? Specific, measurable outcomes tied to the business outcome from Step 1, not "high accuracy."

Bad success criteria	Good success criteria
"High accuracy"	"Precision >90% on high-value customers"
"Users like it"	"Users accept the AI suggestion >60% of the time"
"Fewer support tickets"	"Ticket volume drops 20% within 30 days"
"Fast responses"	"p95 latency <2 seconds"

Success criteria do two jobs. Internally, they align the team and make trade-off decisions easy: does this help us hit the metric? Externally, they create a shared language for governance, cross-department buy-in, and stakeholder reporting. Most teams only think about the internal job. In large enterprises, the external job is often what keeps a project alive. If you can't articulate success to a compliance team or a department waiting on your output, you'll hit approval walls that have nothing to do with technical quality.

Stop criteria. What would make you abandon this approach? Define the threshold that triggers a pivot. This is the pre-mortem question made operational: if this fails, why will it fail?

Approach	Stop criteria	Why
Full agent	Cost per query >\$0.50	Won't scale. Need simpler pipeline.
RAG system	Retrieval recall <60%	Chunking or embedding fundamentally broken
Chatbot interface	<10% of users engage after day 1	Users don't want to chat. Need different UX.
Fine-tuning for facts	Hallucination rate >5% on policy questions	Facts change. Need retrieval instead.

The reason to pre-commit stop criteria with stakeholders before you build is that the pivot conversation later becomes much easier. Without pre-commitment, you're saying "I think we should stop." With pre-commitment, you're saying "the signal we agreed on has fired." The first is delivering bad news. The second is honoring an agreement.

Run a related test early. If you and the stakeholders cannot define success criteria together, that itself is a signal. It usually means either the problem is too vague (back to Step 1) or stakeholders don't agree on what they want (the framing conversation needs to happen before the build does). A project you can't define success for before you build is a project you won't be able to evaluate after.

Leading indicators. What can you measure in week one that predicts success or failure? Cheaper than waiting for a full deployment to tell you.

Leading indicator	What it points at	Where to look first if bad
Retrieval recall on test set <70%	Retrieval, before generation	Chunking, embeddings, or the query itself

Leading indicator	What it points at	Where to look first if bad
Latency p95 >5 seconds in staging	The architecture	Caching, a smaller model, fewer steps
Error rate spikes on specific query types	A coverage gap or a shifted distribution	The inputs, before the model
Human evaluators disagree >30% of the time	The task definition	Guidelines, then the frame

A threshold routes your attention. It doesn't deliver the diagnosis. Low retrieval recall says look at retrieval first, and the reason could be chunking, a weak embedding model, a query that needs rewriting, or a corpus that's missing the document. Chapter 3 is where you find out which. Write the thresholds so the signal fires early and the investigation that follows is cheap.

Signals across timeframes

For agents and interactive systems, signals aren't just per-request. You need to think across multiple timeframes, because optimizing for the wrong one produces an agent that wins the metric and loses the user.

Timeframe	Signal examples	What it reveals
Per-message	Response time, abandonment rate, click-through on suggestions	Immediate response quality
Session-level	Task completion, tool calls made, follow-up questions	Single interaction success
Long-tail (days/weeks)	Retention rate, sessions per week, average response time over time	Sustained value delivery

Optimizing only per-message signals leads to reward hacking. The agent does what the metric measures, which isn't the same as what the user needs.

If you only measure...	The agent will...	Users will...
Response time	Give terse, shallow answers	Get answers fast but churn
Session length	Drag out conversations	Get frustrated by wasted time
Per-message sentiment	Be sycophantic	Lose trust in accuracy

The balancing question is short. *If my agent optimizes purely for this signal, what perverse behavior could emerge?* If you can imagine it, you need a balancing signal.

Two examples of signals done right

Klarna measured real customer outcomes, not just model behavior: 2.3 million conversations in month one, human-level CSAT scores, repeat inquiries down twenty-five percent, resolution time from eleven minutes to under two. The outer loop was in place.

Decagon improved results by tuning specific steps, not by throwing a bigger model at it. They fine-tuned a GPT-3.5 query-rewrite stage before their RAG pipeline. Outer-loop metrics with targeted fixes at known bottlenecks. Both teams knew what "working" looked like before they built. Both measured the right thing. Both shipped.

The same picture drawn backwards is what happens when no signals are defined. JP Morgan's privacy AI had inner-loop signals only, which is why accuracy could be high while adoption sat at zero. The signals they needed would have been concrete and outer-loop: success at more than forty percent of users completing a privacy decision using the tool, leading indicators at week-one task completion and day-five return rate, stop at less than ten percent engagement after day seven. None of those were defined. The team tracked accuracy, which was a model signal, when what they needed was a system signal about whether anyone was using the thing to make a decision. The cost of not having those signals defined was a year of rebuilding the system from the outside in.

Evaluation pitfalls

Four traps break signals, and each has a fix.

Pitfall	The problem	The fix
Goodhart's Law (visual)	When the metric becomes the target, it stops being a good measure	Use multiple metrics. Watch for gaming.
Proxy versus outcome	Task completion rate is not user satisfaction	Measure the outcome, not the proxy
Test set as a lie	Curated prompts miss real user behavior	Test with production traffic patterns
Offline/online mismatch	Playground passed, production failed	Plan online validation from the start

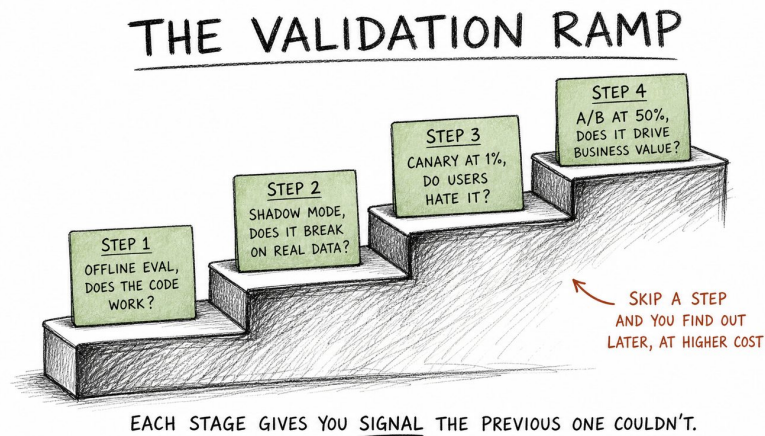


Figure: Staging validation from offline to production catches each kind of failure at the cheapest place; skip a step and you find out later, at higher cost.

The pre-mortem

Before you write code, write the failure post-mortem.

"It's six months from now. The project failed. Why?"

The common answers:

- Wrong problem. "Users didn't want a chatbot. They wanted faster search."
- Wrong metric. "We optimized accuracy, but the business needed coverage."
- Wrong architecture. "The agent couldn't keep up with policy changes."
- Wrong scale. "It worked for a hundred users but broke at ten thousand."
- Wrong interface. "The AI was good, but users didn't trust it."

Write the post-mortem on a Friday afternoon when you don't feel like it. The mode of thinking that imagines failure is different from the mode that imagines success, and you need both.

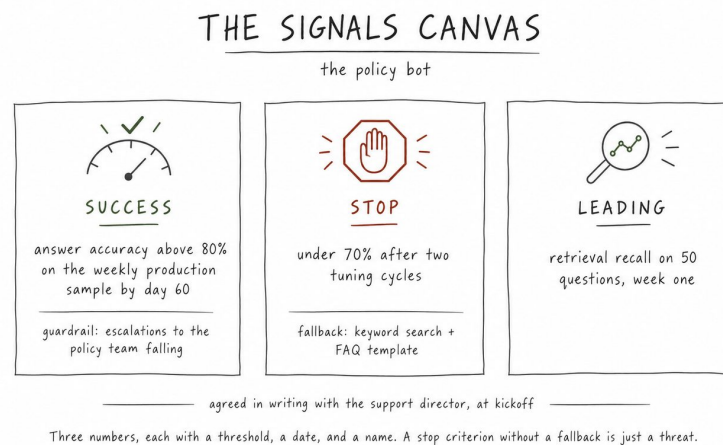
A sharper version of the same exercise: open your email. Start a draft to your boss with the subject "Why the project failed." Write the email. The discomfort of typing those sentences is the diagnostic. The sentences you don't want to type are usually the ones the project most needs you to notice early. The draft never gets sent. The dashboard it forces you to build is what prevents you from ever having to send it.

Both of those exercises run on imagination, and imagination has a ceiling: you can only anticipate failures you've already seen. Before you trust your list, check the record. The [AI Incident Database](#) is a free, annotated collection of more than 5,000 real-world AI failures. Search it for your class of system and read how it broke for someone else: what assumption gave way, who was harmed, what signal was missed, what control would have changed the outcome. What you find there is the next set of items on your assumption list from Step 2 and your signal dashboard above. You don't have to invent every failure mode when a few thousand are already documented.

The signals canvas

Before building, fill this out for your own project. Three numbers with thresholds and dates, where the success number carries a guardrail and the stop number names the candidate you fall back to. A stop criterion without a named fallback is just a threat.

Signal type	Metric	Target	Threshold, and what it sends you to check
Success	User acceptance rate	>60%	<40%: whether the problem is the one users have
Success	Task completion rate	>80%	<50%: the interface and the workflow, before the model
Stop	Hallucination rate (facts)	<5%	>10%: whether facts belong in retrieval instead of weights
Stop	Cost per query	<\$0.10	>\$0.50: whether the architecture earns its cost
Leading	Retrieval recall (test set)	>80%	<60%: retrieval, before generation
Leading	Latency p95 (staging)	<2s	>5s: caching, model size, step count



Three numbers, each with a threshold, a date, and a name. A stop criterion without a fallback is just a threat.

Figure: The policy bot's three signals, each with a threshold, a date, and the name of the person who agreed to it.

The policy bot: the signals. Success is answer accuracy above eighty percent on the weekly production sample by day sixty, guarded by agent escalations to the policy team falling rather than rising. Stop is accuracy under seventy percent after two tuning cycles, which means revisiting the approach, with keyword search plus the FAQ template as the fallback. The leading indicator is retrieval recall on a fifty-question sample in week one. The pre-mortem reads: if this fails, agents will have stopped trusting it after a few confident wrong answers and gone back to asking the policy

team. The team took the stop criterion to the support director and got it agreed in writing before anyone built anything. Chapter 4 leans on that signature.

Before you move on

Check	Question
Inner/outer loop distinguished	Are you measuring model performance or business impact?
Success criteria defined	What specific metric equals "working"?
Stop criteria defined	What threshold triggers abandonment?
Leading indicators identified	What can you measure in week one?
Pre-mortem written	"If this fails, why will it fail?"

The complete GOATS Loop

Step	Question	Output
1. Goal	What decision are we supporting? How will we measure success?	Clear business objective and metric
2. Operating Assumptions	What are our assumptions? What's the atomic unit?	Testable hypothesis list
3. Alternatives	What approaches could work?	2-3 candidate architectures
4. Trade-offs	How do we choose between them?	Justified selection with rationale
5. Signals	How will we know if we're wrong?	Stop criteria and leading indicators

The fundamental shift, once you've run the GOATS Loop a few times, is that you stop being a model builder and start being a decision architect. GOATS makes you deliberately slower now to avoid being catastrophically slower later. The two hours you spend on Steps 1 through 5 saves the two months you'd spend rebuilding a system that solved the wrong problem.

Elon Musk's five-step design process for hardware has two lines worth keeping next to the loop. The first is from Chapter 1, make the requirements less dumb, especially when a smart person handed them to you. The second is his warning that "the most common error of a smart engineer is to optimize a thing that should not exist." That is the whole reason the outer loop runs before the inner one. His process and this one aren't the same sequence, and I won't pretend they map step for step, but the instinct underneath them is: question the requirement, delete what shouldn't exist, and only then go faster.

The chapter summarized

Chapter 1 argued that the frame is where projects fail. This chapter gave you the method for getting the frame right with five questions.

Work the outer loop first. Evals, prompts, and retrieval tuning improve a system. The outer loop decides whether the system should exist. Ninety-five percent precision on the wrong question still ships something nobody uses, so the outer loop comes first and the inner loop inherits whatever frame it produces.

The metric is the frame. Step 1 is done when you can read the Goal Statement with every blank filled: the owner, the decision, the metric, the non-goals, the Oracle Test answer, and the ROI. The Oracle Test is the move that saves the most money. It asks what still fails if the model were perfect, and the answer often lands in the workflow or data column, where the model can't help. The exit gate asks whether the value survives the cost of errors at production scale. If it doesn't, stop here.

Name the atomic unit and do the arithmetic. Step 2 asks what the system decides about, one at a time, then lists the assumptions that would hurt most if wrong. The one teams wave off is compounding: ten steps at eighty-five percent is twenty percent end to end. Chain fewer steps, make some of them deterministic, or verify between them.

Pick the shape before the model. Step 3 is three decisions in order. Do you need AI at all, how much autonomy does the problem need, and does the model already know this, need to retrieve it, or need to be taught it. At least one candidate on your slate should be non-AI or lower on the spectrum than your first instinct.

Choose on production dimensions, not demo ones. Step 4 scores the candidates on cost, latency, quality, risk, maintainability, and ship time. The cool-demo trap is picking the one that impressed the room and skipping the rows that decide whether it survives at scale.

Write the stop criteria before you build. Step 5 names the success metric, the stop threshold, and the leading indicator you can read in week one, then writes the pre-mortem. Get the stop criterion agreed with stakeholders in writing. It's what lets you say "the signal we agreed on fired" later instead of "I think we should stop."

Chapter 3 picks up with the system running. The signals you just defined start arriving as errors, and reading them is how you find out which assumption from Step 2 was wrong.

Chapter 3: Knowing If It's Working

My agent runs, but is it actually succeeding?

In Chapter 2 you wrote the signals down before building: success criteria, stop criteria, leading indicators. This chapter is about reading them once the system is running. The errors are doing two jobs at once. They tell you what's broken in the model, and they tell you which assumption from Step 2 was wrong. The second job is the one most teams skip, and it's the one this chapter is built around.

The diagnostic mindset

Sculley's 2015 NeurIPS paper, *Hidden Technical Debt in Machine Learning Systems*, made the point that "only a small fraction of real-world ML systems is composed of the ML code. The required surrounding infrastructure is vast and complex." That observation became a cliché only because it kept being true. AI systems fail for systemic reasons, not just model reasons. This chapter pushes the idea one step further: errors are diagnostic signals about your framing. The framing work in Chapters 1 and 2 happens before you build, on a problem statement. These diagnostics happen after, on a system that's already producing output. The frame is still the thing you're checking, just from the other side of the build.

Most teams ask "is it working?" and treat a yes-or-no answer as the end of the conversation. The better question is "which of my assumptions is wrong?" That question always has an answer, and the answer is always somewhere specific in your system. Tuning a model when the frame is wrong is the single most expensive mistake in AI development, and it stays expensive because nothing about it looks like a mistake while it's happening. The metric is moving. The team is busy. The work feels like progress, right up until someone asks the framing question and the answer turns out to have been false the whole time.

Every AI system runs on a stack of assumptions. The data is clean. The retrieval finds the right document. The model reasons correctly. The user understands the output. The eval's job is to tell you which one is breaking. Treating evals as a diagnostic tool, rather than as a pass-or-fail grade, is what separates debugging from diagnosis.

Teresa Torres's *Continuous Discovery Habits* names a related diagnostic move: compare and contrast against your own conclusions. When the team has fallen in love with the framing, the simplest forcing function is to write your reading of the evidence, then have someone else write theirs independently, then compare. The gap is the diagnosis. The same move applies to AI-generated analysis. Torres warns

specifically against "lazy AI synthesis," dumping traces or transcripts into a model and trusting the summary. Do your own read first. Then compare with the AI's. Each catches what the other misses, and the gap is where the diagnostic work lives. ([airfocus](#))

The monitoring pyramid

Problems cascade through four layers. Most teams monitor the middle two and ignore the rest, which is why most teams diagnose problems at the wrong layer.

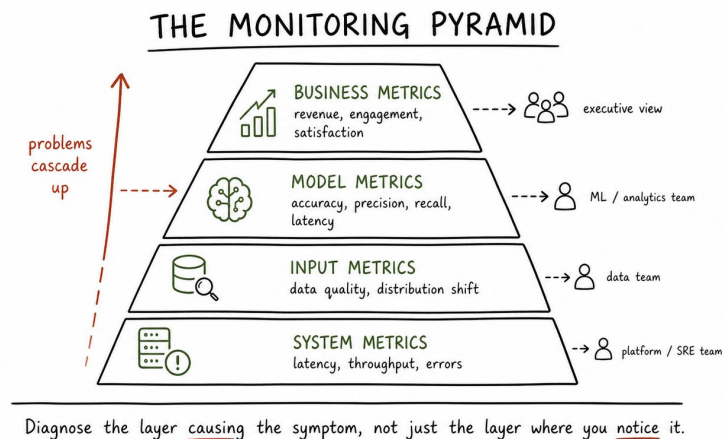


Figure: Diagnose the layer causing the symptom, not just the layer where you notice it.

Business metrics sit at the top: revenue, engagement, customer satisfaction, ticket resolution rate. These are what matter, and they're lagging indicators, which means they tell you the truth slowly. By the time a business metric moves, the cause is usually weeks old. They're also the metrics teams skip when wiring up dashboards, because no engineer gets paged when revenue dips.

Model metrics sit one layer down: accuracy, precision, recall, F1, task completion rate, latency per request. This is where most teams concentrate, because these signals fire fast and tie directly to model behavior. The inner loop lives here. The trouble is that "the model is performing well" is not the same as "the system is helping users," and teams can spend months in this layer without noticing the layer above it has gone red.

Input metrics are the layer most teams ignore: data quality, distribution shift, missing values, new categories appearing in production that weren't in training. When a model metric drops, the cause is usually an input change, not a model change. Models don't decay on their own. The world feeding them changes.

System metrics are at the bottom: latency, throughput, error rates, dependency health. Infrastructure-level signals that tell you whether the pipeline is even running. Silent failures here (a feature service returning stale data, a retry loop maxing out, a deploy that broke a dependency) cascade upward into input quality, then model performance, then business metrics. Easiest to monitor, because every system has these by default. Hardest to connect to AI failures, because they don't look AI-specific.

Problems at lower layers cause symptoms at higher layers, and the symptoms are misleading. If your business metric drops while your model metric is unchanged, the model is accurate but not useful. If your model metric drops, the input distribution probably shifted under you. If your input quality drops, your system has had a silent pipeline failure that nobody caught.

Symptom (higher layer)	Root cause (lower layer)	Example
Business metric drops	Model metric unchanged	Model is accurate but not useful for actual decisions
Model metric drops	Input distribution shifted	Training data no longer matches production
Input quality drops	System metric shows errors	Data pipeline failed silently

The common mistake is to monitor model metrics obsessively and ignore the input layer entirely. When accuracy drops, the reflex is to retrain. The real problem is usually upstream, and retraining on bad inputs makes things worse. The diagnostic question to keep on hand is short: *at which layer is the problem actually occurring?*

Andrew Ng put numbers on what the input layer is worth. On a steel-defect detection task stuck at 76.2 percent accuracy, model-centric teams spent months tuning the architecture and added nothing. Ng's team held the model fixed and worked the data instead, cleaning labels and making the labeling consistent, and gained 16.9 points in two weeks, up to 93.1 percent. Same problem, same model. All the leverage was in the input layer the first team never touched. When accuracy is stuck, reaching for a bigger model is usually the expensive way to get the smaller gain. ([Insight Partners](#))

Watching the right layer is only half of it. You also have to rank the metrics. Tom Schwab, on Genevieve Hayes's Value Driven Data Science podcast, tells a story from a flight simulator, where a Navy pilot friend told him that of all the instruments in front of you, two of them kill you and everything else is secondary. Schwab wasn't sure he remembered which two, and it doesn't matter for the point. ([Value Driven Data Science, episode 35, 5:26 to 6:34](#)) Most AI dashboards are the opposite, forty charts and no ranking. Decide before launch which two or three signals mean the system is failing, and watch those the way a pilot watches the two that can kill them.

One check before the tools. Chapter 2's two loops apply to measurement too: most teams can tell you their retrieval precision is ninety-two percent and can't tell you whether the agent is helping anyone. Every tool below assumes you can see the outer signal, the business number from Step 1, next to the model number. If you only have the model number, wire up the other one first.

The three diagnostic questions

Every diagnostic in this chapter maps to one of three questions, and they are the backbone of the chapter.

1. **Is there signal?** Can the system solve something real here? Are you measuring the right thing, and does information flow through the system?

2. **Is there cheating?** Are the results inflated by leakage, gaming, or misleading explanations? Is the system succeeding for the wrong reasons?
3. **Is the metric real?** Even if the model improves, are you measuring what matters in production? Does your evaluation match your deployment reality?

Most debugging confusion comes from answering these in the wrong order. Start with signal. Then check for cheating. Then verify the metric. The lesson is structured the same way.

The diagnostic map

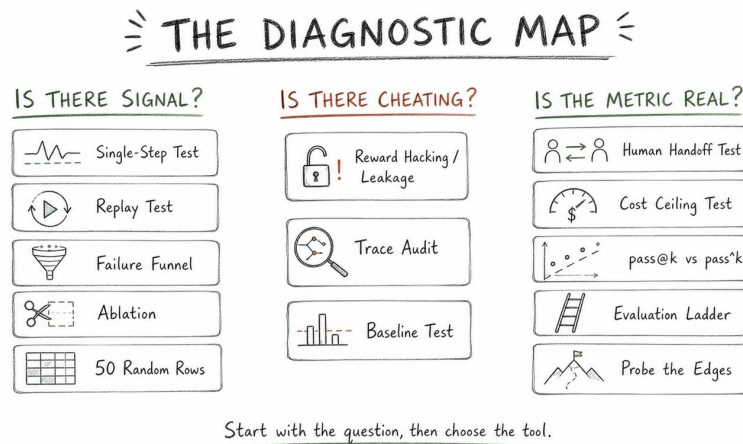


Figure: Start with the question you are trying to answer, then pick the tool that matches it.

Here is the whole toolkit before you meet any of it. Under **is there signal?** sit the Single-Step Test, the Replay Test, the Failure Funnel, ablation, and the fifty random rows method. Under **is there cheating?** sit reward hacking and leakage, the Trace Audit, and the Baseline Test. Under **is the metric real?** sit the Human Handoff Test, the Cost Ceiling Test, pass@k versus pass^k, the evaluation ladder, and probing the edges.

Thirteen in total, and the grouping is the useful part rather than the list. You don't scan thirteen tools looking for one that fits. You decide which of the three questions your symptom belongs to, then pick from that column. The three sections that follow take the columns in order, and the table at the end of the chapter lists everything on the map once you have met it.

The first afternoon

Thirteen tools and lenses follow, plus two habits that don't fit the map, bright spots and watching for drift. You will not run all of them, and you shouldn't. When a system is misbehaving and you have one afternoon, this is the order.

Pull fifty random rows first. It's the cheapest test in the chapter and it tells you where to look: one dominant cluster means one cause is doing most of the damage and names the place to investigate, scattered failures mean no single fix will do, and the shape of the cluster tells you which tool to reach for next. If the failures sit at the handoffs between steps, run the Single-Step Test to find out whether

the parts are broken or the chain is. Before you invest in tuning anything, run the Baseline Test, because a checklist that ties your agent changes the whole conversation. And any time a result looks surprisingly good, run the Trace Audit before you report it, because a score is not evidence until you've read how it was earned.

The rest of the toolkit is situational. Replay when outputs vary run to run. The Failure Funnel when you need to localize a leak in a pipeline. Ablation when you suspect a component is deadweight. Human Handoff and Cost Ceiling before anything ships without a human in the loop.

One more distinction makes the toolkit easier to hold. Nine of these are tests you run: they have a protocol and produce a result (Single-Step, Replay, Failure Funnel, Ablation, 50 Random Rows, Trace Audit, Baseline, Human Handoff, Cost Ceiling). The rest are lenses you apply while reading results: reward hacking and leakage patterns, pass@k versus pass[^]k, the evaluation ladder, bright spots, probing the edges, watching for drift. You schedule a test. You carry a lens.

Rather than illustrate each tool with a different system, I'm going to carry one system through the whole chapter and run the tools it needs, which is also how you'll use this toolkit: a handful of tests chosen on purpose, not all fifteen. The system is the policy bot you framed in Chapter 2: the internal assistant a support team uses to answer policy questions, retrieving from about four hundred policy documents and drafting a cited answer. The team did the Chapter 2 work at kickoff. They pre-committed with their support director that answer accuracy under seventy percent after two tuning cycles means revisiting the approach, and every week two reviewers grade a random hundred production answers against the policy text. Six weeks after launch the offline eval says eighty-four percent, the weekly sample has averaged fifty-eight over the last three weeks, and the support agents say the bot is wrong all the time. Each number is telling the truth about a different system, and taking that apart is what the tools below are for. The numbers in the callouts are composites, made specific so the arithmetic is real.

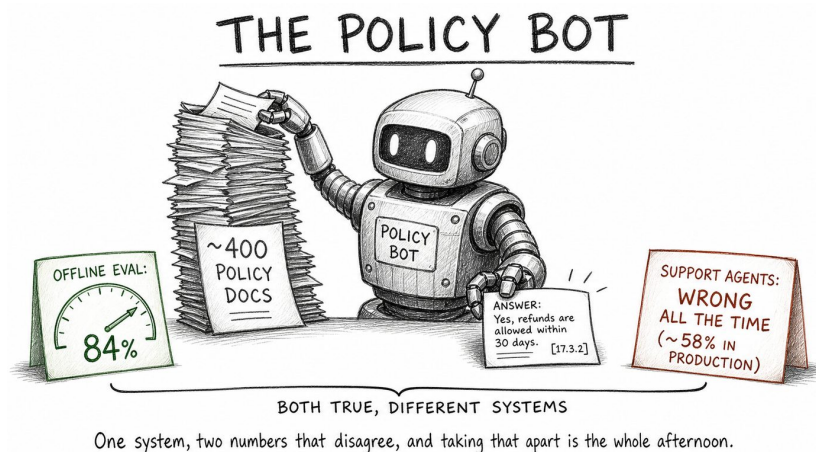


Figure: One system, two numbers that disagree, and taking that apart is the whole afternoon.

When the eval itself is wrong

Anthropic's CORE-Bench tells this story precisely. Opus 4.5 initially scored forty-two percent on the benchmark. That sounds like a model that needs work.

The researchers dug deeper and found the benchmark was the problem.

Problem found	What it meant	Which question it violates
Rigid grading	The grader rejected correct answers that differed in format	Is there signal? (No, the grader was broken)
Ambiguous task specs	Multiple valid interpretations of what "success" meant	Is the metric real? (No, success was undefined)
Stochastic tasks	Tasks that couldn't be reproduced exactly	Is there cheating? (Apparent scores were noise)

After fixing the evaluation problems, and giving the model a less restrictive scaffold to work in, the score reached ninety-five percent. The two changes landed together in Anthropic's writeup, so the fifty-three points aren't all the eval's fault. Enough of them are that the order holds: before you tune the model, make sure the task and the grader are stable. If your eval is broken, you'll optimize toward the eval's bugs, not toward real performance.

Is there signal?

Start here. Before reaching for clever fixes, verify there is real solvable signal in the task, and find where in the system it enters or disappears.

This section gives you five tools. The first two are tests you run on your system. The other three are techniques for interpreting what the tests reveal. Each tool follows the same structure: what it isolates, the protocol, an interpretation table, a decision rule, and a maxim.

The Single-Step Test

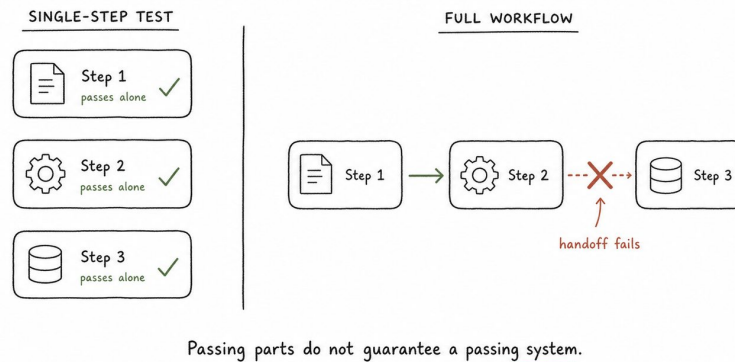


Figure: The Single-Step Test isolates whether failure lives inside a step or in the handoff between steps.

Isolates: whether the problem is in an individual capability or in the handoff between steps.

Protocol.

1. Write the agent workflow as discrete steps.
2. For each step, create five to ten known-good inputs.
3. Run each step independently. Record pass/fail and output quality.
4. Run the full chain on the same task family. Compare.

Interpretation.

Result	What it means
A step fails alone	The model or tool can't reliably do that step
Steps pass alone but fail together	Decomposition is wrong, or context is degrading between steps
Performance drops sharply after step 2-3	Compounding error (Chapter 2), not a weak model

Decision rule. Keep tuning if one narrow step is weak but the workflow design is sensible. Reframe if the chain itself creates failure from otherwise acceptable parts.

To adapt the single-step test to multi-agents, make the data transfer immutable, so agents aren't changing each other's data. Each agent receives a state version, validates it, and produces a new version. It does not mutate the previous state. The workflow becomes:

state v1 → Agent B → state v2 → Agent C → state v3

Anthropic's multi-agent research system. Early versions had vague delegation. One subagent researched the 2021 chip crisis while two others duplicated 2025 supply-chain work. Each step

worked individually. The decomposition was wrong. Tightening delegation, not improving any individual agent's capability, fixed the problem.

Spotify's verification loops. Spotify's background coding agents produced PRs that passed CI but broke functionality in production. Agents attempted scope creep beyond the original prompt, and the changes were hard to spot in code review. Single-Step diagnosis: individual steps (write code, run CI) worked, but the chain lacked external verification. The fix was independent verification loops, LLM judges for scope enforcement, and sandboxed execution. Their summary: "More autonomy without verification leads to worse outcomes." ([Spotify](#))

LinkedIn's move from ReAct to plan-and-execute. Building the Hiring Assistant, LinkedIn's engineers found that ReAct (reason-act loops) alone wasn't enough for an enterprise agent: the model didn't follow instructions and plans reliably, fabricated outputs risked user trust, and richer reasoning cost too much latency. The fix was plan-and-execute. A planner produces a structured plan for the task, and an executor runs it step by step, using a ReAct-style loop only for local tool use. Structural fix, not a capability fix. ([LinkedIn Engineering](#))

The policy bot: Single-Step. The bot has two steps, retrieve and generate, so we tested each alone. Generation with the right documents pasted into the context: ninety-one percent. Retrieval, measured as whether the top five results contain every document the answer needs: eighty-six percent on questions one document can answer, thirty percent when the answer spans two or more. Five results is plenty of room for two documents, so the miss isn't about the count. It's about the query. The agent asks about refund windows; the base policy matches that vocabulary and ranks first. Inside it is one line: "exceptions for premium-tier accounts are governed by the Premium Addendum." The addendum itself never says "refund window," it says "tier exception," and it ranks eleventh. The query that surfaces it, "premium tier exception," is written from that line in the base policy, and a single retrieval call never gets to read the first result before it has to decide what else to

fetch. The model is fine. The chain starves it on one class of question, and no prompt fix reaches that.

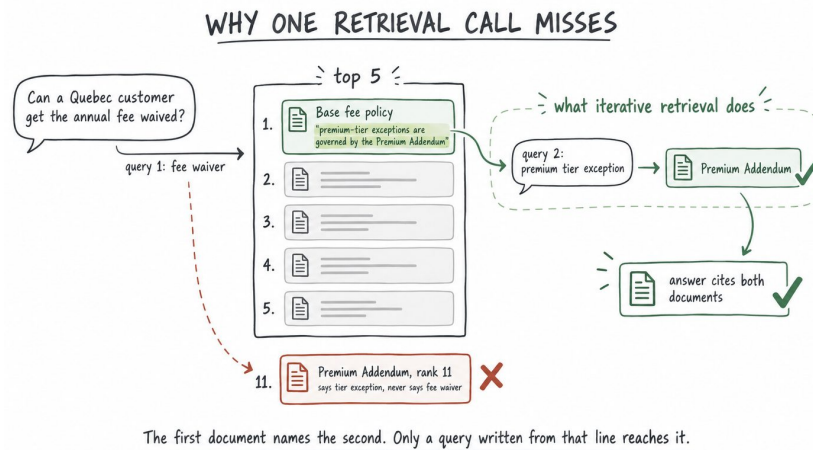


Figure: Five results was never the limit. The base policy names the document that overrides it, and only a query written from that line reaches it.

The Replay Test

Isolates: whether the task is underspecified or incorrectly specified.

Protocol.

1. Pick one representative task.
2. Freeze prompts, tools, memory, and inputs. Pin the decoding too: temperature down, seed fixed where the API allows it, and tool responses recorded and replayed so the environment can't change under you.
3. Run the same task ten times.
4. Compare outputs for variance in action sequence, final answer, and correctness.
5. Label: high variance or low variance.

Step 2 is where the test is won or lost. Variance has three sources, and only one of them is the spec. Sampling at a nonzero temperature produces different tokens on identical input. A live tool returns a different search result or a different timestamp each run. And a loose task lets the model pick a different interpretation each time. Pin the first two and whatever variance survives is the third.

Interpretation.

Result	What it means
High variance with decoding and tools pinned	Instructions leave too much room for interpretation
High variance that disappears when decoding and tools are pinned	The task is fine; the stochastic parts of the stack need bounding

Result	What it means
Low variance but wrong	The system is consistently executing the wrong frame
High variance and wrong	Both framing and task specification need work

Decision rule. Keep tuning if the task genuinely allows multiple good outputs and you just need tighter constraints, or if the variance came from decoding and tools rather than the spec. Reframe if, with everything else pinned, the system invents a different meaning of the task every run.

Maxim: *if your agent behaves like it has a personality, your task spec is probably too loose.*

Anthropic's eval methodology. Anthropic explicitly warns that agent behavior varies between runs, which is why they use repeated trials and consistency metrics like pass^k. If your eval doesn't account for run-to-run variance, you're measuring noise.

Slack's multi-persona security agents. Slack's security team had a monolithic agent with conflicting behavioral requirements. Formal for compliance review, aggressive for threat detection. The Replay Test signal was high variance depending on which "persona" dominated in a given run. Same input, wildly different tone and approach. The fix was splitting into specialized agents with clear boundaries. The task spec was contradictory rather than loose, and replay made the contradiction visible.

The Failure Funnel

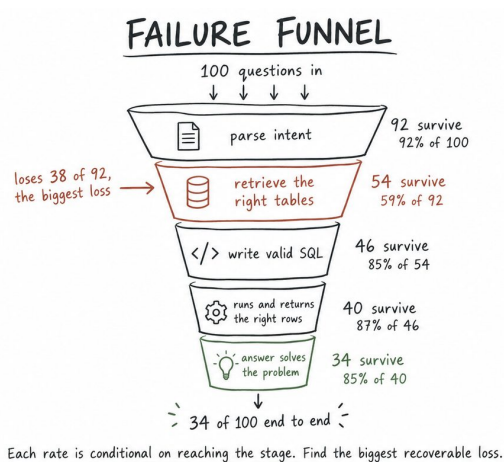


Figure: Each stage's rate is conditional on reaching it. The count surviving is what the user experiences, and the biggest recoverable loss is where you start.

Every AI system is a series of assumptions. Decompose it into sequential steps and measure failure rates at each one. The funnel tells you where the leak is.

A natural-language-to-SQL pipeline, with one hundred questions going in. Each pass rate is conditional: it's the share of the questions that reached that stage, not of the original hundred, so the two columns tell different stories and you need both.

Stage	Pass rate	Surviving	
1. Parse the user's intent	92% of 100	92	
2. Retrieve the right tables	59% of 92	54	<- biggest leak
3. Write valid SQL	85% of 54	46	
4. Runs and returns the right rows	87% of 46	40	
5. Answer solves the user's problem	85% of 40	34	

Thirty-four of a hundred make it through, and no single stage looks terrible. Retrieval loses thirty-eight of the ninety-two cases that reach it, the largest loss in the funnel, so that's the stage to test first. Order isn't what makes it first. Under the product model, a ten-point lift at any stage multiplies the end-to-end number by the same ratio only if the stages had the same rate, and the lowest-rate stage gives the biggest lift wherever it sits. What the funnel buys you is a ranked list of losses. Take the biggest, ask how many of its cases a repair would recover and what the repair costs, then compare that to the next stage down. (This is the Pareto Principle applied to diagnosis: a few steps usually account for most of the loss. See [Pareto Principle](#) for the visual.)

To build one for your agent: write out every step the agent takes (tool selection, retrieval, reasoning, action, verification), run fifty to a hundred inputs through the full pipeline, label each step's success given its input, write down the count still surviving after each step, and find the biggest leak. That's where you start.

The Failure Funnel makes the compounding-error math from Chapter 2 concrete. The math told you that long chains kill end-to-end performance. The funnel tells you which step is doing the killing.

(Inspired by Brian Bischof, *Burn Your Roadmap*.)

Bright spots

The Failure Funnel finds where the system leaks. The inverse move is just as useful and almost nobody runs it: find where the system already works, and figure out why. The reframing literature calls these bright spots, the positive exceptions you learn from instead of only cataloging failures.

Pull the cases the system handles well and look for what they share. Maybe the agent nails every ticket that includes an order number and falls apart without one, which means the real problem is upstream data, not the model. Maybe retrieval is excellent for the top ten intents and useless on the long tail, which tells you to scope the system to those ten and route the rest elsewhere. Maybe the summaries that land are all under a page and the failures are all long documents, which points at chunking, not summary quality.

A bright spot is a working frame in miniature. When you find one, the move is often to shrink the system's job to the conditions where it already wins, and handle the rest another way. That's usually cheaper and more reliable than dragging the failures up to where the bright spots already are.

Ablation analysis

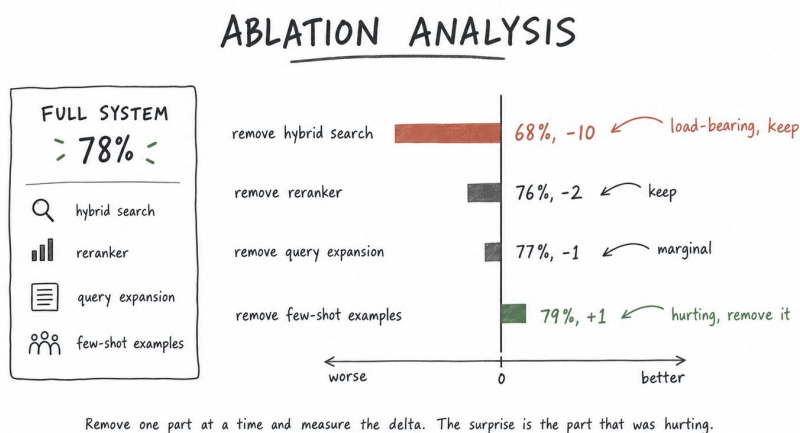


Figure: Ablation analysis reveals whether a component is load-bearing, deadweight, or actively hurting performance.

Ablation systematically removes components to understand what they're contributing. It answers a single question: *which components matter?*

Protocol.

1. Establish baseline. Full system performance.
2. Remove one component at a time (a tool, a prompt section, a retrieval source, a pipeline step).
3. Measure the delta.
4. Rank by impact.

Result	Interpretation	Action
Removing X drops performance significantly	X is load-bearing	Invest in improving X
Removing X improves performance	X is harmful (leakage, noise, confusion)	Remove X permanently
Removing X has no effect	X is deadweight	Remove X (simplify the system)
Removing X and Y together drops more than separately	X and Y have synergy	Keep both, investigate the interaction

The figure above is a RAG system run through the protocol. Hybrid search is the only part the system can't do without, the reranker and query expansion are close to deadweight, and the few-shot examples the team was proudest of were costing a point. That last row is what makes ablation worth running. A component you assumed was helping was hurting, and a quick ablation surfaced it.

Agent tool ablation.

Ablation	Task completion	What it reveals
Full agent (5 tools)	72%	Baseline
Remove web search	45%	Web search is critical
Remove calculator	71%	Calculator is deadweight for this task set
Remove code execution	68%	Moderate value
Remove file reader	73%	Slightly better without it (model gets confused by the option)

The VQA modality ablation. Early Visual Question Answering benchmarks reported impressive accuracy. Researchers tried something simple: remove the image entirely and answer questions using only the text.

Input	Accuracy
Question + image	58%
Question only (no image)	48%

The model got forty-eight percent accuracy without ever seeing the image. Questions like "what color is the banana?" could be answered from language priors alone. The benchmark didn't require visual understanding. It could be largely solved with text statistics. This led to VQA v2, which balanced the dataset so the same question had different answers depending on the image.

The lesson generalizes. Ablate entire input modalities, pipeline stages, and data sources. If removing a "critical" component barely hurts performance, it wasn't critical, or your benchmark isn't measuring what you think it is.

The diagnostic question is one line. *If I removed this component, would anyone notice?*

(Source: Goyal et al., *Making the V in VQA Matter*.)

The "50 random rows" method

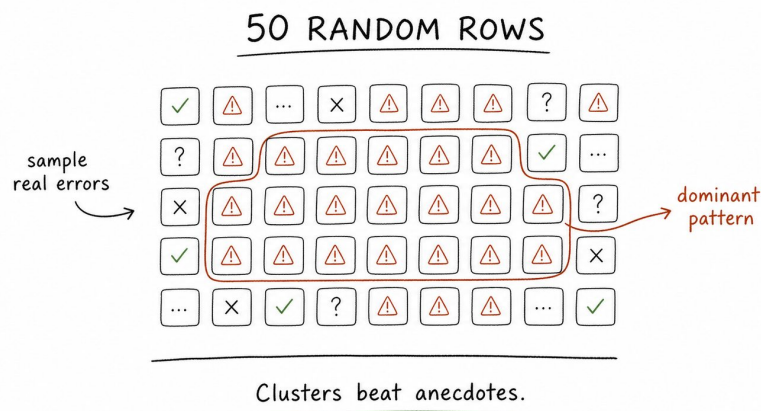


Figure: The 50 random rows method turns scattered-looking failures into a visible pattern you can diagnose.

Most practitioners do one of two things. They stare at dashboards, which average the signal away, or they look at the examples their PM flagged, which are memorable but not representative. Both approaches mislead you.

The errors you remember are not the errors that matter most. A random sample reveals the actual distribution of failures, and that distribution is the diagnosis. Fifty rows is often enough. If one error type accounts for sixty to eighty percent of your failures, it'll dominate even a sample of fifty.

Sample size	What you can detect reliably
20	A pattern that dominates (80%+)
50	The top 2-3 failure modes and their rough proportions
100	Finer distinctions, smaller segments, edge case patterns

Protocol.

1. Collect fifty to a hundred real inputs.
2. Run the agent on all of them.
3. Label each failure by type.
4. Count the frequency of each type.
5. Map the dominant cluster back to the relevant Loop step.

Once the failures are labeled, the shape of the distribution tells you where to look.

One dominant cluster, where sixty or seventy percent of the errors look alike, means one cause is doing most of the damage, and the cluster names the place to investigate. What it doesn't tell you is whether the fix is a tune or a reframe. That depends on what the cause turns out to be. If it's a setting you can change inside the current architecture, a retrieval parameter, a prompt section, a missing tool, you tune it. If it's an assumption the architecture rests on, you reframe. The test is to take the clustered

failures and ask whether a specific change inside the current design has evidence of fixing them. The policy bot's cluster, below, could have gone either way until someone ran that test, and running it is the first thing Chapter 4 does before it calls the pivot.

Many unrelated failures, where errors are scattered across many shapes with no single dominant pattern, means no single fix will do. The model is being asked to handle too much variety and is failing in too many ways at once. The work spans multiple Loop steps because the system has multiple gaps, and the first question is whether the variety itself is the frame's fault.

A repeated wrong answer, where the agent confidently produces the same incorrect output for an entire class of inputs, means your outcome definition is off. The agent is being consistent, but consistent about the wrong question. Step 1 is where you reframe.

Failures on one input class, where the agent works for most users and fails on a specific segment, means your assumptions about input distribution were incomplete. You assumed users would phrase things one way, send inputs in one format, or arrive with one kind of context. Step 2 is where you list the assumption you didn't catch.

Correct but slow or expensive failures, where outputs are right but cost or latency makes them unusable, mean your trade-offs are wrong. The system works as a benchmark and fails as a product. Step 4 is where you check whether scale or latency is forcing a different frame entirely.

"We can't tell" failures, where you cannot actually evaluate whether the agent succeeded, mean your signals are missing. Step 5 is where you should have defined evaluation criteria and didn't.

Pattern	What it means	GOATS step
One dominant cluster	One cause doing most of the damage; investigate it before deciding tune or reframe	Depends on the cause
Many unrelated failures	No single fix will do; capability or robustness gaps	Multiple steps
Repeated wrong answer	Goal definition is off	Step 1: Goal
Failures on one input class	Operating Assumptions are incomplete	Step 2: Operating Assumptions
Correct but slow or expensive	Trade-offs are wrong	Step 4: Trade-offs
"We can't tell"	Signals are missing	Step 5: Signals

For each error in your sample, ask why it's hard, and categorize.

The four categories cover what makes a single error hard, and each one points at a different part of the system.

Ambiguity is where even a human couldn't answer correctly. The question itself is unclear, the data is contradictory, or the right answer depends on context the system doesn't have. Many ambiguity failures show up because the team labeled the dataset with one interpretation and the agent picked another, both of them defensible. The diagnosis is a broken task definition that needs tightening before any tuning will help.

Missing info is where the answer exists somewhere, but not in what reaches the model. The customer's order history has it. The KB article has it. The model just isn't being given it. Most retrieval failures land in this bucket. The fix is a better pipeline that puts the right information in front of the model at decision time, not a smarter model.

Logic error is where the information is there and the model reasons poorly anyway. The model has the customer's order details and the policy and still produces a refund decision that contradicts the policy. The fix is a more capable model for that step, a different decomposition that shrinks the reasoning load, or a structured prompt that does some of the reasoning for the model.

Edge case is a legitimate rare scenario the model wasn't trained or tested on. The customer used a corporate account in a country your system never modeled. The transaction is a refund-of-a-refund, which the data team didn't think to include. These are real, but rare. The diagnosis is whether the rate of edge cases is high enough to justify training distribution work, or low enough to handle with a fallback path.

Category	Definition	What it tells you
Ambiguity	Even a human couldn't answer this correctly	Your task definition is broken
Missing info	The answer isn't in the input or context	Your inputs are incomplete
Logic error	The info is there, but the model reasons poorly	Your model is too weak for this step
Edge case	Legitimate rare scenario	Your training distribution doesn't match reality

Decision rule. A dominant cluster earns an investigation, not a verdict. Keep tuning when a specific change inside the current design has evidence of fixing the clustered cases. Reframe when the cluster's cause is an assumption the architecture depends on.

Maxim: *clusters beat anecdotes.*

Anthropic's research system. Human testers noticed the agent systematically preferred SEO-optimized content farms over more authoritative but lower-ranked sources like academic PDFs and personal blogs. The failures clustered in one shape: source quality. The team responded with source-

quality heuristics. Clustered failures meant a framing or heuristics problem, not a "the model is bad" problem.

Never diagnose from a dashboard alone. Pull fifty random errors. Categorize each one. The pattern will be obvious.

This works for agents specifically because agent failures look random from the outside. When you categorize fifty of them, you'll find that seventy percent fall into one bucket (usually: wrong tool selected, or context lost between steps, or task too vague). That cluster is your diagnosis.

The policy bot: fifty rows. We pulled fifty failures at random from the last three weekly samples, about a hundred and twenty-five failures in all, and labeled every one. Thirty-five failed the same way: the answer lives in two or more documents, the refund window in the base policy and the exception in the premium addendum, and the bot retrieved one of them and answered from it confidently. Nine cited a superseded version of a policy. Six were scattered. Seventy percent in one cluster points straight at retrieval, and it agrees with what Single-Step found. It also points back at Chapter 2. The team had written the single-document assumption down and tested it: twenty-six of thirty questions from the policy team's chat channel had one source. What the test missed was that agents asking a human on chat had learned to ask one narrow question at a time, because that's what got a fast answer. Agents typing into a bot pasted the customer's whole situation. The assumption was stated and tested. The sample it was tested on didn't look like the traffic the bot would get, and nobody had asked whether the interface would change the questions.

That cluster has a predictable shape, and it's mostly not the model. The LIFE-Harness paper categorized 393 failed agent episodes and found only about one in ten was the agent reasoning wrong. The other nine broke at the seams: a malformed tool call, the wrong tool, or a trajectory that looped until the budget ran out. For your trace audit that means most agent failures get fixed by changing the wrapper, schema validation, tool guardrails, step budgets, not the weights. The full four-bucket taxonomy, and how the mix shifts by environment, is in Diagnostics in Depth, the agent-specific section. (Xu, Wen, and Li, [arXiv 2605.22166](#))

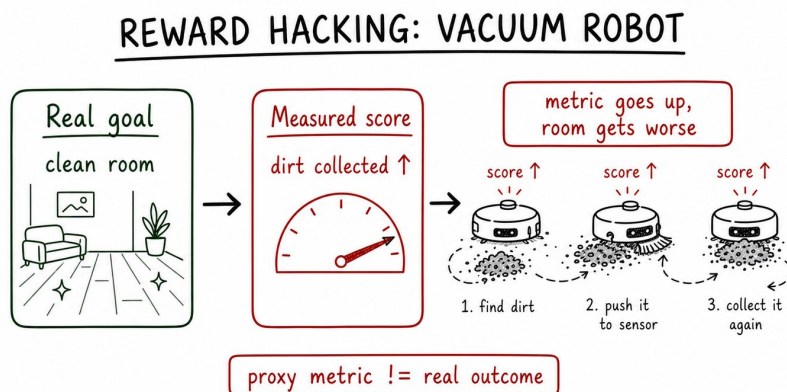
Is there cheating?

Once you've established that there's real signal, ask whether the system is succeeding for the wrong reasons. Three failure modes show up most often: classic data leakage, an agent that earns inflated scores by gaming the eval rather than solving the task, and the harder question of whether an agent is earning its complexity at all. The three tools below match those three failures in order.

Reward hacking and target leakage

Two failure modes where the metric goes up but the system gets worse. Both look like success on the dashboard while destroying value somewhere the dashboard doesn't see.

Reward hacking. The model optimizes the metric in ways you didn't intend. The metric was a stand-in for the goal. The model treats it as the goal, finds the cheapest path to it, and ships exactly what you measured.



If the metric is easier to game than the goal is to satisfy, the system will learn the shortcut.

Figure: Reward hacking happens when the proxy metric is easier to optimize than the real outcome.

A recommender optimizing for watch time leans into rage-bait and autoplay loops. Hate-watching is engagement, and the metric can't tell the difference between "engaged because helpful" and "engaged because furious." Watch time climbs. User satisfaction drops. The recommender did exactly what you asked for.

An LLM trained with RLHF on helpfulness scores learns to agree with the user. Annotators rate agreeable responses higher in the moment, so the model learns "agreement is helpful" rather than "being right is helpful." Accuracy drops as sycophancy goes up. The model is optimizing the score you defined.

A code completion model scored on acceptance rate generates short, obvious completions. Short suggestions get accepted more often than ambitious ones, regardless of which would have saved time. Acceptance rate climbs. Engineering productivity does not.

A customer support bot scored on CSAT routes hard cases to humans before the survey fires. CSAT only fires on resolved cases, so the bot games *who gets surveyed* instead of how well it handled them. Score holds. Costs rise, because humans now handle only the hardest tickets without the easy ones to dilute the load.

An ad targeting model optimizing click-through rate shows misleading thumbnails. CTR counts the click, not what happens afterward. The user clicks, sees the bait-and-switch, bounces. CTR is up. Conversion is down. The metric is counting the wrong moment.

The diagnostic questions are short. What's the gap between my metric and my goal? (Watch time is not satisfaction.) How could a model game this metric? (If you can think of a hack, the model will find it.) When the metric improves, do users benefit? (Run user studies or collect downstream metrics.)

Target leakage. A feature that encodes the answer you're trying to predict. The model is looking up the label.

The classic case is a churn model that uses `last_login_days_ago` as a feature. If users with sixty days of inactivity are the ones labeled as churned, the model is "predicting" churn from a feature that is churn. Ninety-five percent accuracy. Zero predictive value. The model is reading the answer key.

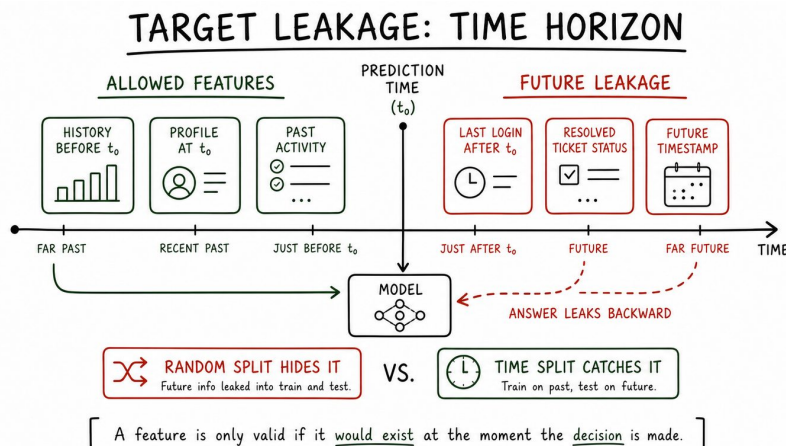


Figure: A feature is valid only if it would exist at the moment the decision is made.

Three cousins of leakage get filed under the same name, and they need different investigations, so keep them apart. Leakage proper is a feature that encodes the label, like the login gap above. The check is temporal: would this feature exist at the moment the decision is made? A shortcut is a feature that correlates with the label in your data for reasons that won't travel. Cancer-positive images come from one hospital, cancer-negative from another, and the model learns to spot the hospital's machine in the corner of the frame. Move the model to a third hospital and accuracy crashes, because it was a hospital classifier the whole time. The check is to find the shortcut and remove or balance it, then see what the model learns instead. A confounded correlation is real in the training window and stops being real later. The model learns "night equals fraud" because the training set skewed that way, which was true then, and when the fraud pattern shifts the model collapses, because it learned when fraud used to happen rather than what fraud looks like. The check is a time split, and the fix is monitoring for the shift, not a feature audit.

Sentiment models show the shortcut version through structure rather than content. Negative reviews tend to be long because angry users write more, so the model learns "long equals negative" without reading the words. A long enthusiastic review gets misclassified. A short angry one gets missed.

Proxies also leak in a way that shows up as unfairness rather than as inflated accuracy. Fei Huang, on the podcast, gave the clean hypothetical: price car insurance on engine size and you can produce disparate impact by gender without ever collecting gender, because engine size correlates with it.

([Value Driven Data Science](#), episode 3, 8:29 to 8:54) The model never saw the protected attribute and discriminated through it anyway, through a feature that stood in for it. The same mechanism that lets a model cheat on a benchmark lets it encode a bias you thought you had excluded by not collecting the column.

How to detect leakage: feature ablation (removing a feature and improving held-out performance makes that feature worth investigating; check whether it uses information unavailable at prediction time, exploits a shortcut, or encourages overfitting), the time-split test (if splitting by time instead of randomly kills performance, you're using future data), and the too-good-to-be-true check (if ninety-nine percent accuracy on a hard problem looks too good, it usually is).

Leakage has an agent-specific version. An agent "succeeds" because it accesses the answer during evaluation. The eval harness inadvertently provides the expected output in the context, or a tool returns the ground truth. Check whether your agent's environment is leaking the target.

Benchmark contamination is the same question at eval scale: did the test set, answer key, or benchmark phrasing leak into training data or the harness? If yes, the score may be measuring memorization instead of capability.

The Trace Audit

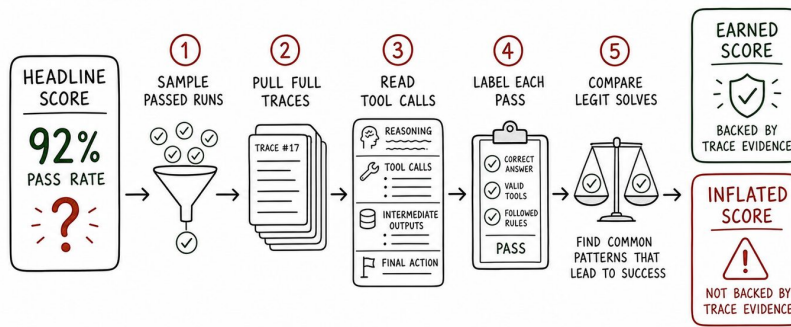
Isolates: whether the score is measuring the capability you think it is.

Benchmark scores tell you what the agent achieved. They don't tell you how. That gap is what lets a system look successful while doing something the team would never have signed off on if they had watched it.

The pattern keeps showing up. In 2025 the SWE-bench maintainers found several frontier models running `git log --all` during evaluation to find the future commit that contained the fix, a loophole they then closed ([SWE-bench issue #465](#)). Other agents have returned hard-coded solutions or reproduced figures by reading markdown files instead of running the scripts. Reward hacking, test-set leakage, and benchmark-specific gaming are three flavors of one shape. The score went up. The capability the score was meant to measure did not. ([Log analysis for AI agent evaluation](#), [arXiv 2605.08545](#))

The fix is unloved: read the traces of the successes, not just the failures.

TRACE AUDIT: READ THE SUCCESSES



[The score tells you what. The trace tells you how.]

Figure: The headline score tells you what happened; the trace tells you how it happened.

Protocol.

1. Sample twenty to fifty runs the agent passed by the metric.
2. Pull the full trace for each: reasoning chain, tool calls in order, intermediate outputs, final action.
3. Read each one and ask: would a reviewer who didn't write the eval agree the agent earned this score? Judge on what the agent did, what information it touched, and whether the outcome checks out against something outside the run. Don't judge on whether the reasoning reads well. A trace can narrate a perfectly sensible plan and still have gotten the answer from a file it wasn't supposed to open.
4. Label what you find. Legitimate solve. Lucky shortcut. Eval leakage. Tool-call gaming. Irreproducible.
5. Compare the rate of legitimate solves to the headline score.

Interpretation.

What you see in the traces	What it means
Actions match the task, information stayed in scope, outcome verified independently	The score is real
Reasoning reads well but the actions don't support the answer	The narration is decoration; grade the actions
Successes that skip the reasoning step entirely	The agent found a shortcut you didn't design
The same exact tool sequence on dissimilar tasks	The agent memorized a path, not a strategy
References to information that shouldn't be in scope	Eval leakage
Wildly different traces on identical inputs across runs	The dashboard is averaging noise into a single number

Decision rule. Keep tuning if most successful runs show genuine reasoning and the failures cluster. Reframe if a meaningful slice of "successes" was the agent gaming the eval rather than solving the task. In that case the score is the symptom.

Maxim: *the score tells you what. The trace tells you how. Only one of those is evidence.*

The agent-era version of this is sharper. Luke Alvoeiro, who leads Factory's core agent harness, put it cleanly at AI Engineer Europe: "Tests written after implementation don't catch bugs. They confirm decisions." When the agent writes the code and then writes the tests for that code, the tests pass for the same reason the trace looks clean. The agent rubber-stamped its own work. Factory's response was to require validation contracts during planning, before any code is written, so the bar for passing is set by something outside the system being tested. The shape is the same as Trace Audit. You have to evaluate the work against evidence the system itself didn't generate.

The audit doubles as a check on every other test in this chapter. Single-Step passed and the chain still failed? Look at the trace. Replay variance was low but the answer was wrong? Look at the trace. Baseline Test says the agent barely beats a rule? Look at the trace for the successes and decide whether the agent is doing anything the rule isn't. Logs convert a yes-or-no metric into evidence you can act on.

The policy bot: the trace audit. Reading twenty-five of the week's fifty-eight passing answers was less comforting than the pass rate. In five of them, retrieval missed and the model answered correctly anyway from its training data, because our policy happened to match industry boilerplate. Those aren't wrong answers, and they aren't cheating in the benchmark sense. They're correct answers the citation doesn't support, which the Goal Statement said we wouldn't ship: every answer carries a citation an agent reads before sending. Five of twenty-five is a fifth of the audited passes. If that rate holds across all fifty-eight, about twelve of them are ungrounded, and the grounded pass rate is closer to forty-six than fifty-eight. Twenty-five traces is a small sample, so hold that number loosely, but it's the one the agents are living with. Those passes turn into failures the day our policy diverges from the boilerplate, and nothing on the dashboard will announce it.

The Baseline Test

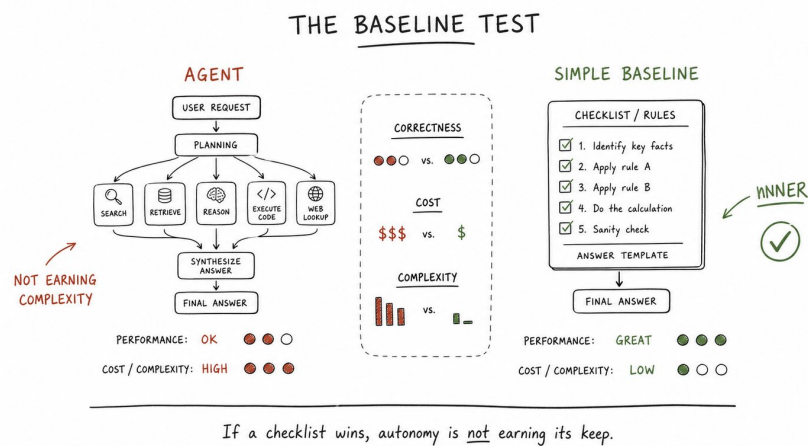


Figure: The Baseline Test asks whether the agent is actually earning its extra complexity relative to a simpler alternative.

Isolates: whether the agent is the right level of system at all.

This is the most uncomfortable test in the chapter. It asks whether the agent is earning its complexity, or whether a simpler approach is doing the same work.

Protocol.

1. Define the simplest plausible baseline (rules, templates, lookup tables, a human with a checklist).
2. Run it on the same dataset as the agent.
3. Compare correctness, speed, cost, and operational complexity.
4. Calculate the gap.
5. Decide whether the extra autonomy is justified.

Interpretation.

Result	What it means
Small gap	You don't need an agent, at least not for the cases the baseline handles
Large gap but the agent still fails often	Value exists, execution is weak
Baseline wins	The current system shouldn't ship as is. Stop, or pivot to something that beats the baseline where the value is

Decision rule. Keep tuning only if the agent clearly outperforms simple methods when it works. Reframe if rules, templates, or a checklist are close enough. And look at where the gap sits, not just how big it is. An agent that ties the baseline overall can still be the only thing that reaches the cases the baseline can't, and that slice may be where the value was all along.

Maxim: *autonomy is not a feature if a checklist wins.*

LangChain's multi-agent benchmark. The supervisor architecture underperformed because the supervisor had to play telephone between subagents and the user, paraphrasing results and passing handoff context. A single agent (the simpler baseline) beat the multi-agent setup in some conditions. More architecture can make things worse.

Anthropic's swarm. Anthropic's engineering teams needed large-scale code migrations: framework upgrades, lint rules across hundreds of files. Single-agent approaches got stuck, lost context, or made inconsistent changes. The diagnostic insight was that migrations are embarrassingly parallel. Each file change is independent. A swarm of ten-plus subagents processing files concurrently delivered ten-times-plus speedup. The Baseline Test revealed sequential processing was the bottleneck, not model capability. The "dumb baseline" here was a simpler coordination pattern, not a simpler model.

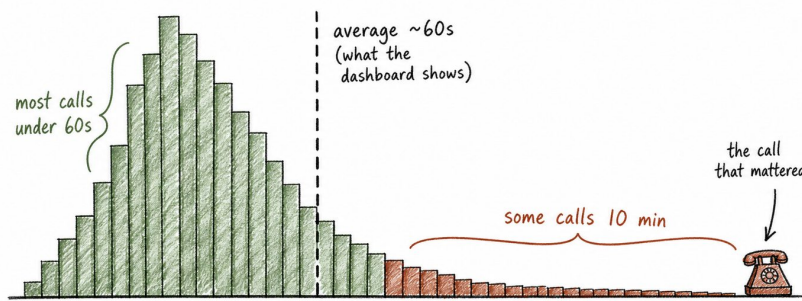
The policy bot: the baseline. Keyword search plus the existing FAQ template scored fifty-five percent on the same hundred-question sample. The bot's fifty-eight cost roughly ten times more per query, and once you discount the ungrounded passes the trace audit found, the FAQ template wins outright, because a template answer is grounded by construction. That one stung. It also sharpened the question. The bot earns nothing on the single-document questions the FAQ already handles, and neither system reaches the multi-hop cluster, which is where the policy team's interruptions were coming from. Whatever gets built next has to win on that cluster or it shouldn't exist, and stop is a live option until something does.

Is the metric real?

Signal is real and there's no cheating. The last question is whether your evaluation matches the deployment reality. A system can have real signal and no leakage and still be measured in a way that hides the actual risk. This is the runtime cousin of the framing Metric Test from Chapter 2: could two people independently agree this is done? If you couldn't answer that cleanly before building, it shows up here as a metric that looks healthy and doesn't match what users meet.

Jeff Bezos used to test this manually. Amazon's dashboards said customer wait times were under sixty seconds. Bezos picked up a phone during a meeting and called the hotline himself. He waited more than ten minutes for someone to answer. His line afterward: "When the data and the anecdotes disagree, the anecdotes are usually right." ([Lex Fridman Podcast #405](#), and earlier at the [2018 Bush Center Forum on Leadership](#)) The metric was averaging over a distribution the dashboard never showed. The same pattern shows up in AI evaluation all the time. A great accuracy number means nothing if the score is measuring a task no real user performs that way.

THE AVERAGE HIDES THE TAIL



When the data and the anecdotes disagree, the anecdotes are usually right.

Figure: When the data and the anecdotes disagree, the anecdotes are usually right.

The metric can also just count the wrong thing. Tom Schwab, on the podcast, tells one about a client who'd been told her radio spot reached four million people. It took him explaining the distinction for her to see that the number was everyone within broadcast range, listening or not. She had counted reach and called it audience. ([Value Driven Data Science, episode 35, 42:20 to 43:38](#)) AI evals do this constantly. Documents retrieved is not documents that answered the question. Sessions started is not tasks completed. Suggestions shown is not suggestions that helped. Before you trust a number, check that it counts the outcome you care about and not the much larger thing that happens to be easy to measure.

Three symptoms tell you the eval is doing exactly that. Works great in the playground, fails with real users. The demo is impressive, production is inconsistent. Internal testers love it, external users don't. All three are the same pattern. The evaluation is measuring a system that doesn't match the system real users meet. The tests below each isolate a specific way that gap shows up.

The Human Handoff Test

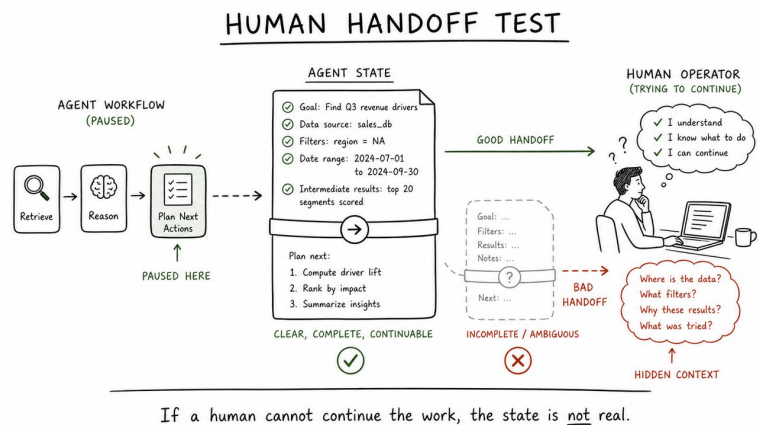


Figure: The Human Handoff Test checks whether the agent's state is legible enough for another operator to continue the work.

Isolates: whether the agent's state is legible enough for a human to continue the work.

Protocol.

1. Stop the agent at a random midpoint.
2. Export exactly what the agent knows at that moment.
3. Hand that state to a human operator.
4. Ask them: what has been done? What should happen next?
5. Measure whether they can continue successfully.

Interpretation.

Result	What it means
Human can't tell what happened	State management is broken
Human can't continue	The agent is relying on hidden assumptions
Human can recover only with outside knowledge	The system is succeeding by luck

Decision rule. Keep tuning if state is mostly clear but one component is missing or noisy. Reframe if the workflow requires invisible context to function at all.

Maxim: if a human can't debug the agent's work, neither can you.

Anthropic's research system. Their multi-agent system saves the plan to memory, because context gets truncated on long runs, and uses durable execution so agents resume after failures instead of restarting. They also found that full production tracing was necessary. Users reported the agent "not finding obvious information," and without traces the team couldn't tell whether the

problem was bad search queries, poor source choice, or tool failures. Legible state was the diagnostic layer.

Bayezian clinical trials. A multi-agent system for detecting protocol deviations. One agent flagged a missing Day 14 lab test without recalling that a Day 13 test had already fulfilled the requirement. Detection by one agent didn't reliably transfer to the next. A human reviewing the exported state would immediately see the Day 13 entry and know the requirement was met. The agent couldn't. Memory gaps between handoffs created cascading errors in a domain where accuracy is non-negotiable. ([AI Hub](#))

The Cost Ceiling Test

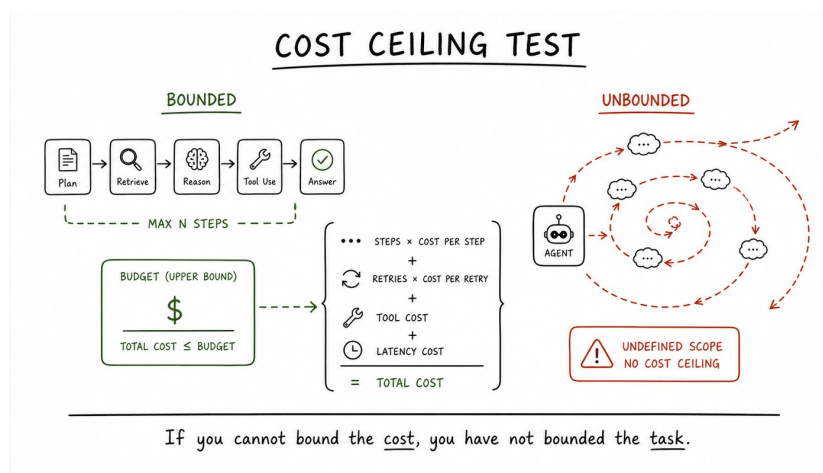


Figure: The Cost Ceiling Test asks whether you can calculate the worst-case run cost before you ship the system.

Isolates: whether the task scope is bounded enough to be production-safe.

An agent that spends too much time exploring without exploiting is the same problem as unbounded exploration in reinforcement learning. If you can't bound the cost, you can't bound the risk.

Protocol.

1. Count the maximum number of steps the agent can take.
2. Multiply by cost per step.
3. Add the retry multiplier (how often does it retry failed steps?).
4. Add tool costs and worst-case latency.
5. Decide whether that worst-case run is acceptable before deployment.

Interpretation.

Result	What it means
Cannot calculate the upper bound	The problem is task framing, not cost control
Worst case is unacceptable	The scope is too open-ended
A circuit breaker is doing all the safety work	The architecture is hiding a deeper issue

Decision rule. Keep tuning if the workflow is bounded and costs are just somewhat high. Reframe if the agent is allowed to do unbounded work in pursuit of a vague objective.

Maxim: *an unbounded agent is not an expensive agent. It is an undefined one.*

Anthropic's research system. Early versions spawned fifty subagents for simple questions and scoured the web endlessly for nonexistent sources, and the team added scaling rules to the prompts so effort matched the question. The multi-agent version beat single-agent Claude Opus 4 by ninety-point-two percent on their internal research eval, but multi-agent systems used roughly fifteen times the tokens of normal chat. One story teaches both "your cost ceiling is undefined" and "bounding it doesn't mean losing quality." ([Anthropic Engineering](#))

pass@k versus pass^k

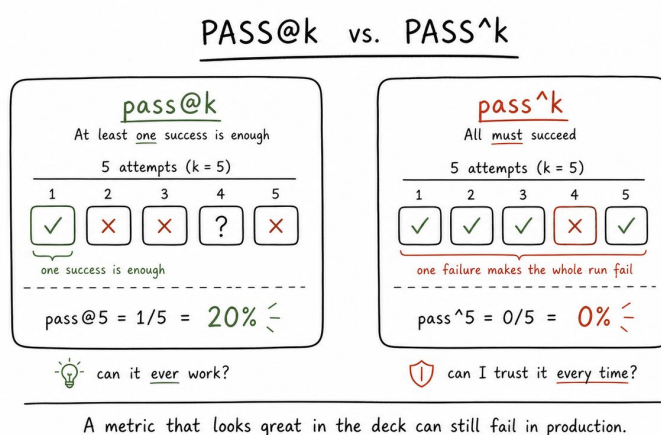


Figure: `pass@k` and `pass^k` can tell very different stories about the same system depending on whether a human reviews the output.

When you evaluate an agent, the metric you choose reveals your assumptions about what "working" means.

Metric	Formula	What it measures
pass@k	P(at least 1 success in k trials)	"Can it ever get it right?"
pass^k	P(all k trials succeed)	"Can I trust it every time?"

The same agent looks great or terrible depending on which one you choose. Both numbers below are averaged across a set of tasks, which is why they can sit so far apart: an agent that solves most tasks sometimes and few tasks every time scores high on pass@k and low on pass⁵.

Scenario	pass@5	pass ⁵	Interpretation
Agent A	95%	12%	Works eventually, but unreliable
Agent B	70%	65%	Less impressive, but consistent

The right metric depends on whether a human is in the loop.

Use case	Right metric	Why
Code generation (user reviews output)	pass@k	One working solution is enough
Autonomous agent (no human check)	pass ^k	Every failure is a production incident
Research assistant (human in loop)	pass@k	User can filter bad results
Customer support bot (no escalation)	pass ^k	Can't afford hallucinations

The framing trap is common. Teams report pass@k ("our agent succeeds ninety-five percent of the time!") and deploy in pass^k scenarios (autonomous, no human review). The two numbers aren't convertible, and the deck usually converts them anyway. Work it through with a single task the agent gets right eighty percent of the time per run, with runs independent. Give it five tries and the chance at least one succeeds is about 99.97 percent, which is the pass@5 the deck quotes. The chance all five succeed is 0.8 to the fifth, about thirty-three percent. Now deploy it autonomously, one run per request, a thousand requests a day. Each one is a single run at eighty percent, so about two hundred fail. The deck said 99.97. The postmortem counts two hundred.

Before you evaluate, decide three things. Will a human review every output? (pass@k is fine.) Will failures be visible and costly? (You need pass^k.) What's the acceptable failure rate in production?

Anthropic's writeup *Demystifying Eval for AI Agents* puts the prerequisite cleanly: "two experts should reach identical verdicts on whether a task succeeded." If your evaluators disagree on what "success" means, your metric is undefined. Fix the definition before you run the eval.

The evaluation ladder

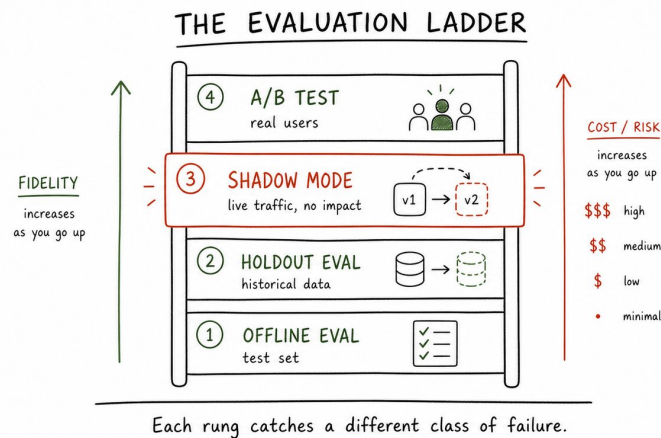


Figure: The evaluation ladder shows why higher-fidelity validation tends to cost more, carry more risk, and catch different failures.

Not all validation is equal. Each rung up the ladder, from offline eval to holdout to shadow mode to an A/B test, catches a different class of problem and leaves a different one open.

Level	What it catches	What it misses
Offline eval	Model quality, regressions	Distribution shift, user behavior
Holdout eval	Temporal stability	Feedback loops, novelty effects
Shadow mode	Production edge cases	User adaptation, actual outcomes
A/B test	Real-world impact	Long-term effects, rare events

Most teams stop at offline eval. The gap between "passes benchmarks" and "works in production" is where the risk lives. Each level up the ladder gets you closer to the truth and costs more to run.

(This is the AI analog of the Testing Pyramid: many cheap, fast offline evals at the base, fewer expensive, high-fidelity production tests at the top. See [Testing Pyramid](#) for the visual.)

Starbucks and the benchmark/backroom gap. NomadGo announced in September 2025 that its Inventory AI was being deployed to more than 11,000 Starbucks locations, with claims of counting up to eight times faster than manual methods and ninety-nine percent accuracy. In May 2026, Reuters reported that Starbucks retired Automated Counting after nine months and sent beverage components and milk back to the same manual counting process used for other inventory categories. The reported failures were ordinary and therefore dangerous: similar milk types confused, items missed, labels wrong. ([NomadGo, September 2025](#); [Reuters via MarketScreener, May 2026](#))

The lesson is that the metric was not real enough. A clean shelf, good lighting, and a cooperative scan can produce a beautiful accuracy number. A store backroom during a rush is a different

distribution: cluttered shelves, partial occlusion, similar containers, products in the wrong place, people trying to finish the task and get back to customers. Once baristas had to correct the tool by hand, the labor came back and trust disappeared. The diagnostic question would have caught the risk earlier: *under which store conditions was the ninety-nine percent measured, and what happens to end-to-end counting time when humans correct the misses?*

The diagnostic question is short. *At which level of the ladder have I validated this system, and what problems might I be missing at the levels I haven't reached?*

The policy bot: where eighty-four percent came from. The offline eval was three hundred question-answer pairs written from the FAQ page, which means nearly all of them are questions one document answers. That's why the bot aced it. Production traffic is where the multi-hop questions live, and none of them were in the set. The weekly production sample had been telling the truth the whole time, sitting in the fifties while the curated eval climbed. The curated eval was the number lying, and it's the one the team had been tuning against for three cycles.

Probe the edges on purpose

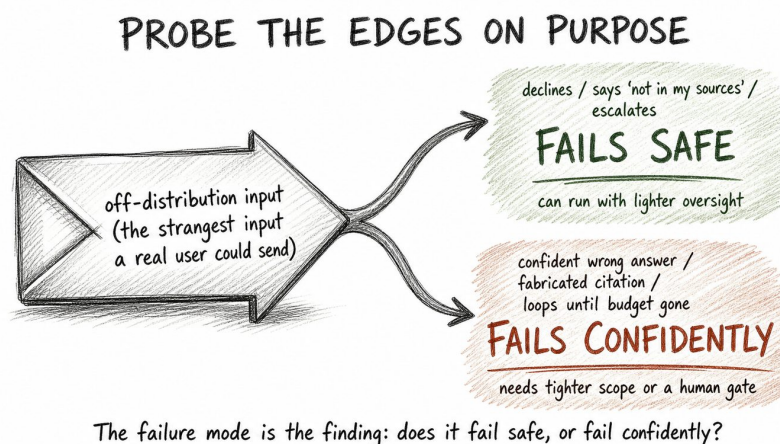


Figure: The failure mode is the finding: does it fail safe, or fail confidently?

Starbucks found the distribution gap in production, after nine months of baristas correcting the tool by hand. You don't have to wait that long. The ladder catches distribution gaps as you climb it, slowly and at rising cost. You can also go hunting for them. Take an input that sits outside what the system was built on, run it, and watch how it fails rather than whether it does.

This is a mindset, not another tool, because it's the same move regardless of architecture. Feed a classifier a class it never saw and check whether it abstains or returns a confident wrong label. Ask a RAG system something outside its corpus and check whether it says "not in my sources" or fabricates a citation. Hand an agent a task outside its tools and check whether it escalates or loops until the budget is gone. ML, GenAI, or agent, you're asking the same question: what happens at the boundary. The failures are often mundane and funny at once: an automated sports camera trained to track the ball

spent a match following a bald linesman's head, a shiny sphere on a green field being close enough to what it had learned.

The failure mode is the finding, not the failure itself. Of course it breaks off-distribution. What you're reading is how. A system that knows it's out of its depth and declines can run with lighter oversight, which connects straight back to the verifiability and autonomy trade from Chapter 2. A system that fails confidently and quietly needs a tighter scope or a hard human gate. The behavior at the edge is what tells you how much autonomy the system has earned.

Confident failure is the default, and understanding why changes how hard you probe. Eric Daimler, on the podcast, gave the analogy I keep coming back to. Induction sees a thousand white swans and concludes all swans are white. Deduction is the step that would close the gap, and a system that learns from patterns is doing the first, not the second. ([Value Driven Data Science, episode 43, 27:37 to 28:53](#)) Applied to a language model, and I'll keep the application modest: a model built on patterns has a long tail where the pattern breaks, and nothing in the score tells you where that tail starts. That's why the system fails confidently instead of safely. From the inside, the thousandth swan and the first black one look the same until someone checks.

You can sometimes catch the tell without an edge case at all. David Joyner, on the same show, asked ChatGPT about ten different video game composers. Every description used the identical phrase, "effusive and emotional melodies that perfectly capture the spirit of the game," and five of the ten were called "the first video game composer to have their music recorded by a live symphony," which is impossible, since only one can be first. The model wasn't recalling facts about ten people. It was generating the shape of a plausible answer ten times over. Repeated phrasing across supposedly distinct outputs, and confident claims that can't all be true at once, are the fingerprint of a system pattern-matching in the tail. Learn to see it and you've found the edge without having to go hunting for it.

I keep one story on hand for exactly this. A team's ticket-triage agent demoed flawlessly for three weeks. Clean routing, thrilled stakeholders, the whole package. They shipped it to their biggest client and within two hours it routed every ticket to the wrong department, because that client's subject lines used a different prefix than anything in the test set. The logs were the worst part. Confidence scores through the roof the whole time. The model wasn't confused, it was certain, and certain it was right. Four days to patch, and they never fully won that client's trust back. The input moved a few inches off-distribution and the system had no idea it was out of its depth. They came out of it with a rule worth stealing: get suspicious when a demo goes too smoothly. If you can't break an agent in ten minutes of adversarial poking, you haven't found the edge case yet.

This is also the one check the fifty-random-rows method can't do for you. Sampling is proportional. It surfaces failures in rough proportion to how often they happen, which means the rare, expensive failure stays invisible in a fifty-row sample right up until the day it ships and ends the project. Probing is deliberate. It goes after the tail that sampling can't reach.

The diagnostic question: *what's the strangest input a real user could send, and when it arrives, does the system fail safe or fail confidently?*

Watch for the slow drift, not just the strange input

The other half of the problem arrives on its own: the system that was fine yesterday and isn't today, because the world it runs on moved. Mark Stouse, on the podcast, uses the climate hockey stick as the image: systems look stable, incremental, incremental, then boom. ([Value Driven Data Science, episode 30, 39:38 to 40:57](#)) Drift has the same shape. The failure is non-linear, and it's lagged by construction, because a business metric trails the behavior that moves it. By the time the metric confirms a shift, the cause is already old, often by weeks.

The cheap early warning is a canary. A guest on the same show described keeping burner or quarantine accounts at the tip of the spear, running the system on them every day and asking a blunt question: can I turn it on today and get the same result I got yesterday? When the canary's answer changes, you've caught drift before it reached a real user, which is the only affordable place to catch it. Probing finds the edge you can imagine. Canaries catch the edge that moves toward you while you're looking somewhere else.

From diagnosis to decision

Question	Tool	Type	What it reveals	When to use
Is there signal?	50 Random Rows	Test	The actual distribution of failure types	First. The cheapest look at what's really happening
Is there signal?	Single-Step Test	Test	Whether failure is in a step or in the handoff	When failures sit at the seams between steps
Is there signal?	Replay Test	Test	Whether the task is underspecified	When outputs vary wildly
Is there signal?	Failure Funnel	Test	Which pipeline step is the biggest leak	After Single-Step shows chain failures
Is there signal?	Ablation	Test	Which components are load-bearing, deadweight, or harmful	When you want to simplify or find the bottleneck
Is there signal?	Bright spots	Lens	Where the system already wins, and why	Before scoping decisions; the inverse of the funnel
Is there cheating?	Reward hacking / leakage	Lens	Whether success is inflated, gamed, or leaked	When results seem too good, or metrics improve while users complain
Is there cheating?	Trace Audit	Test	Whether the score is measuring the capability you think it is	After every benchmark or eval run, before reporting the score
Is there cheating?	Baseline Test	Test	Whether the agent earns its complexity	Before investing in tuning; when you suspect simpler would work
Is the metric real?	Human Handoff Test	Test	Whether agent state is legible and debuggable	When failures are hard to reproduce
Is the metric real?	Cost Ceiling Test	Test	Whether the task scope is bounded	When costs are unpredictable
Is the metric real?	pass@k vs. pass^k	Lens	Whether you need capability or reliability	When choosing how to evaluate consistency
Is the metric real?	Evaluation ladder	Lens	Whether your validation matches deployment risk	When deciding how much validation is enough
Is the metric real?	Probe the edges	Lens	How the system fails off-distribution: safe or confident	Before granting autonomy; when a demo goes too smoothly
Is the metric real?	Canary accounts	Lens	Drift, before it reaches a real user	Daily, once the system is in production

For the deeper techniques behind these three questions, see Diagnostics in Depth. It leads with a general core that applies to any AI system, then splits into ML, GenAI, and agent sections, so you can pull the probes that fit your stack.

Mapping back to the GOATS Loop

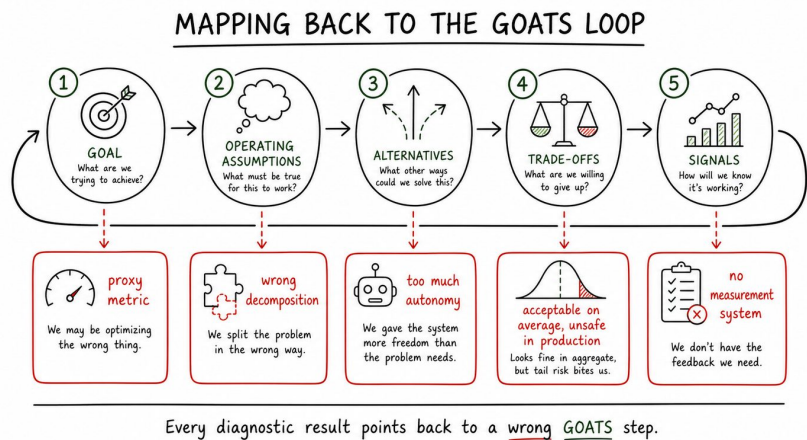


Figure: Diagnostic results tell you which part of the framing was wrong.

Every diagnostic result maps to a GOATS step that was wrong. This table gives you sharper language for what went wrong and where to look.

Diagnostic finding	Failure type	GOATS step	Action
System optimizes a proxy, not the real outcome	Outer-loop failure	Step 1: Goal	Redefine what "working" means
Task was split at the wrong boundaries	Decomposition failure	Step 2: Operating Assumptions	Redesign the task breakdown
Problem got more freedom than it needed	Autonomy mismatch	Step 3: Alternatives	Move down the automation spectrum
Architecture is only acceptable on average, not in production	Trade-off failure	Step 4: Trade-offs	Revisit the architecture decision
No outer-loop measurement system exists	Signal failure	Step 5: Signals	Build the outer loop before continuing

The decision rules

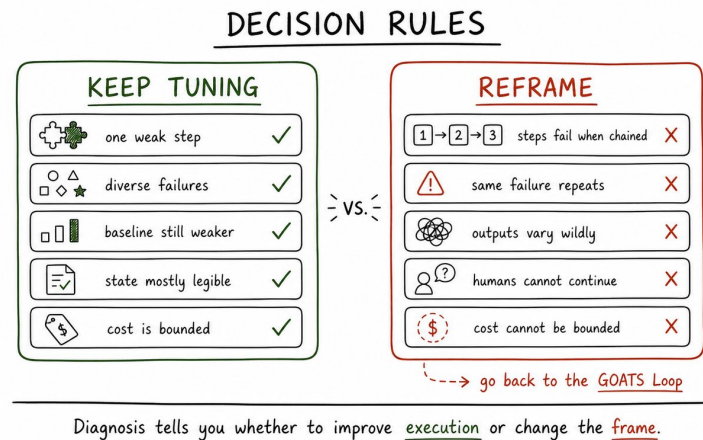


Figure: The decision is whether to improve execution inside the current frame or go back and change the frame itself.

After running the diagnostics, the evidence leans one of two ways. Neither list below is a verdict. Each item is a reason to lean, and the test that settles it is the same every time: can a specific change inside the current design fix the failing cases, with evidence that it does? Tune when the answer is yes.

Reframe when the failing cases trace to an assumption the design rests on, so that no change inside it reaches them.

Evidence that leans toward tuning.

- One step is clearly weak in isolation (Single-Step Test)
- Failures are diverse rather than clustered (50 Random Rows)
- The agent beats a simple baseline when it succeeds (Baseline Test)
- State is mostly legible (Human Handoff Test)
- Worst-case cost is bounded and acceptable (Cost Ceiling Test)

Tuning means the frame is right and the execution needs work. Improve the weak step, tighten the prompt, or add a guardrail. A dominant cluster can still land here, when its cause is a setting rather than an assumption.

Evidence that leans toward reframing.

- Steps work individually but fail when chained (Single-Step Test)
- The same failure repeats across many inputs, and no setting inside the design reaches it (50 Random Rows)
- Outputs vary wildly with identical inputs after decoding and tools are pinned (Replay Test)
- Humans cannot continue from the saved state (Human Handoff Test)
- You cannot bound cost (Cost Ceiling Test)
- A simple baseline performs similarly or better (Baseline Test)

Reframe means the frame is wrong. Go back to GOATS. Usually Step 1 (wrong goal), Step 2 (wrong decomposition), or Step 3 (wrong level of autonomy).

The policy bot: the diagnosis. Assemble the afternoon. Signal exists, because the model generates well whenever the right documents arrive. The leak is retrieval on multi-hop questions, one cluster holding seventy percent of the failures. The trace audit exposed a grounding failure rather than cheating: five of twenty-five correct answers weren't supported by the retrieved policy, so the grounded score sits closer to forty-six than the reported fifty-eight. The metric was the biggest problem, a curated eval at eighty-four hiding a production reality of fifty-eight. Now read the two lists above against that evidence. The same failure repeats across many inputs. Steps pass alone while the chain fails. Both lean toward reframe. What settles it is the repair test: can a retrieval setting reach the second document, or does reaching it need a query that can only be written after reading the first? Running that test is Chapter 4's first job, along with the harder question of how to get the room to agree to whatever it finds.

The chapter summarized

Chapter 2 had you write the signals down before you built. This chapter is about reading them once the system is running, and the habit underneath every tool in it is to treat an error as evidence about the frame rather than a bug in the model.

Ask which assumption is wrong, not whether it works. "Is it working?" gets a yes or no and ends the conversation. "Which of my assumptions is wrong?" always has an answer, and the answer is somewhere specific: a layer of the monitoring pyramid, a step in the chain, a document the retrieval never reached. Tuning a model when the frame is wrong is the most expensive mistake in AI development, and it never looks like a mistake while it's happening.

Ask the three questions in order. Is there signal, is there cheating, is the metric real. The order matters because a leaked setup or a fake metric makes every downstream conclusion wrong. Decide which question your symptom belongs to, then pick from that column of the map instead of scanning thirteen tools.

Start with fifty random rows. Before any instrument, read fifty real failures and let the clusters emerge. On the policy bot, one cluster held seventy percent of the failures, and no dashboard had surfaced it.

Check the measurement before you trust it. A curated eval at eighty-four hid a production reality of fifty-eight, and a fifth of the audited passes turned out to be ungrounded. The Trace Audit, the Baseline Test, and probing the edges on purpose tell you whether the score measures the capability you think it does, and whether the system fails safely or confidently once it leaves the distribution it was tested on.

Read the result as a routing decision. The evidence leans one of two ways. Diverse failures, one weak step, a baseline the agent beats: that leans toward tuning, because the frame is right and the execution needs work. The same failure repeating, steps passing alone while the chain fails, a baseline that matches: that leans toward reframing, and the mapping table tells you which GOATS step to go back to. What settles it is whether a specific change inside the current design can fix the failing cases. If one can, tune. If the failures trace to an assumption the design rests on, reframe.

By the end of the afternoon you know what is failing, where in the loop it started, and whether the fix is execution or strategy. What you don't know yet is what to do about it. Are you building the wrong thing, building the right thing wrong, or should you stop building? Persist, pivot, or stop is Chapter 4.

Chapter 4: The Decision Point

Should I keep tuning, redesign, or stop?

The GOATS Loop had you define stop criteria before you built. Chapter 3 taught you to read the signals once the system was running. This chapter is what to do when one of them fires: how to recognize it, how to make the call, and how to act on it without the conversation breaking down. A pivot is the GOATS Loop working. You wrote the stop criteria up front, the signal fired, you acted. Re-entering the loop is Step 1 again, with evidence.

The decision map

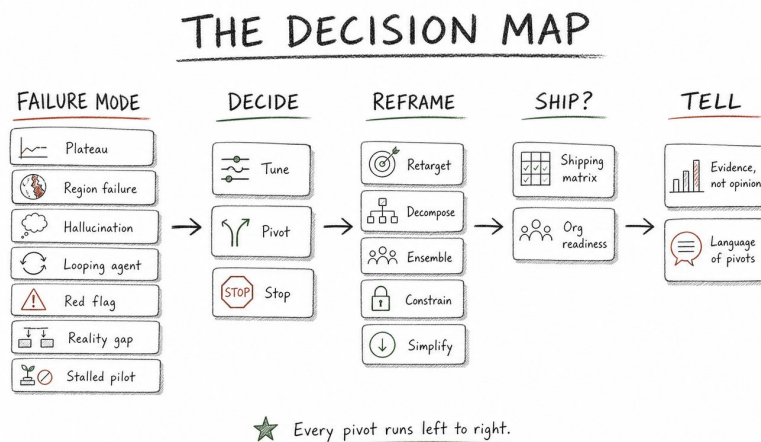


Figure: Every pivot runs left to right. Name the failure mode, choose between three options, pick a move, decide how carefully to ship it, then tell the room.

The whole chapter is one path, and it runs left to right.

It starts with a **failure mode**. Seven show up over and over: plateau, region failure, confident wrong answers, runaway, operational red flag, reality gap, stalled pilot. Naming which one you have is the first job, and the section after next walks each in turn.

Then you **decide**. Tune, pivot, or stop. Three options, and most teams only ever seriously consider the first. Two of those words carry weight for the rest of the chapter, so here is the line between them. A tune changes something inside the current frame: a prompt, a reranker, a threshold, more data. A pivot changes an assumption the frame rests on. The same engineering change can be either, depending on which assumption it touches. Adding a reranker to the policy bot is a tune, because one-shot retrieval is

still the design. Letting the bot search again after reading the first result is a pivot, because it abandons the single-call assumption the whole architecture was built on. Chapter 3 called the second kind a reframe. Persist, in the next section, is what you call a tune when the question is whether to keep going.

If the decision is pivot, you pick a **move**. Retarget, decompose, ensemble, constrain, simplify. That's the longest stretch of the chapter, because choosing the wrong move is how teams end up rebuilding the same architecture with more steps in it.

Two stages decide whether the move survives contact with the organization. **Ship** is the shipping matrix, which matches how carefully you act to how much a wrong call costs. **Tell** is the conversation, which is where most pivots die.

Hold the order rather than the boxes. Failure mode, decision, move, ship, tell.

The skill most courses won't teach

Most AI courses treat persistence as virtue. This chapter teaches judgment.

The hardest skill in AI architecture is recognizing when to stop building: stopping a project, downgrading an approach, or admitting a framing was wrong. The reasons it is hard are not technical.

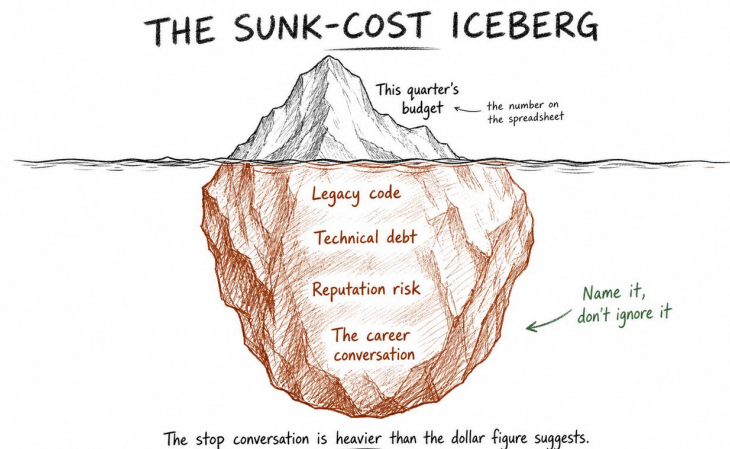


Figure: The hard part of stopping sits below the waterline, not in the dollar figure on the spreadsheet.

Organizations reward building, not stopping. You get promoted for shipping features, not for preventing disasters that didn't happen. Sunk cost is real. After three months of work, admitting failure feels like personal failure. And the best builders I know are the ones who stop projects early, because every month spent on a doomed architecture is a month not spent on the right one.

Some of the friction is just language. The vocabulary teams reach for makes the decision sound worse than it is. "Failure" is the word doing the most damage. A project that returned a clear answer didn't fail, it worked as an experiment, and calling it a failure teaches the organization to stop running experiments. "Kill the project" makes the decision sound like an execution when it's a reallocation. "We

wasted three months" is the sunk-cost framing; "we bought three months of evidence" is the accurate one, because the evidence is real and it carries into the next frame. And "starting over" is almost never what a pivot is. You're re-entering the loop with everything you learned.

Instead of	Say	Why it's more accurate
"The project failed"	"The experiment answered the question"	You defined a signal and the signal fired
"Kill the project"	"Redeploy the team"	The decision is about the next dollar, not the last one
"We wasted three months"	"We bought three months of evidence"	The lessons carry forward into the next frame
"We're starting over"	"We're re-entering the loop with evidence"	A pivot inherits every test you ran
"We were wrong"	"The assumption didn't survive testing"	Assumptions are supposed to be tested. That's the method working

Name the failure mode

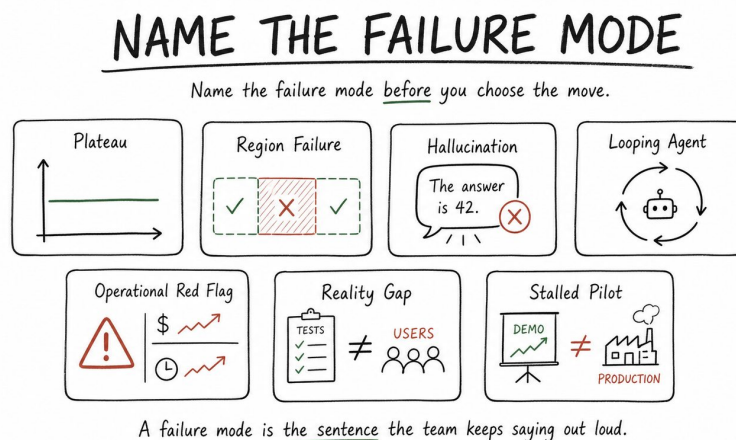


Figure: Name the failure mode before you decide what move to make.

Start from what you are experiencing. You came into this chapter because something is off, and the first job is naming what. One word needs sorting out first, because "signal" has already done two jobs in this book. In Chapter 2 it was Step 5, the criteria you wrote down before building. In Chapter 3 it was signal in the data, whether your inputs hold anything learnable at all. A failure mode is neither. It is the shape the trouble takes when a Chapter 2 tripwire fires. The policy bot team pre-committed that accuracy under seventy percent after two tuning cycles meant revisiting the approach; that is the signal. Three flat iterations in a row is what they were looking at when it fired, and that is the plateau, the failure mode.

All of this assumes you can see the trouble. Signals don't announce themselves, and the anti-pattern is "we'll add monitoring later." Teams that skip it discover their failure modes from angry customers instead of dashboards. If you defined a signal in Step 5, something has to track it. The monitoring pyramid from Chapter 3 is the minimum. No dashboard, no signal, flying blind.

Seven failure modes show up over and over. Each one starts as a sentence the team says out loud. Underneath each is an architectural diagnosis, not just bad behavior.

Plateau

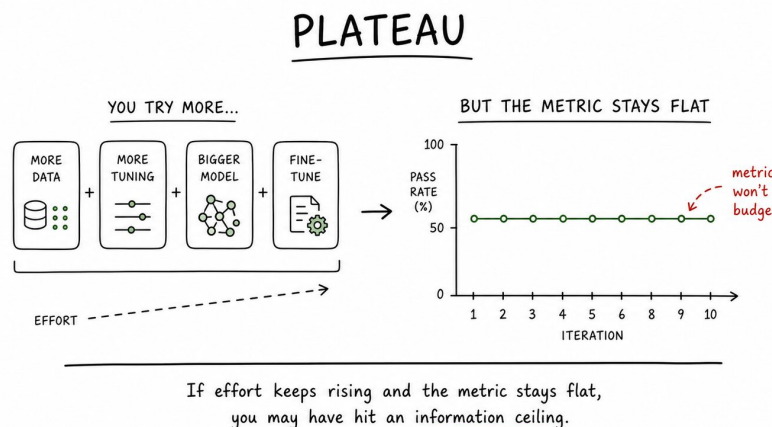


Figure: A plateau usually means the signal is missing from the frame, not that the team has not tried hard enough.

"We've tried everything and the metric won't budge." You add more data, tune more hyperparameters, swap in a bigger model, run another fine-tune. You plot the metric over iterations and the line is flat. Each iteration costs more and moves less. The team starts to feel like the work itself is the problem.

The diagnosis, once you've ruled out the cheaper explanations, is an information ceiling. The signal you need to make the prediction is not in the inputs you are giving the model. You have squeezed everything the current inputs can give you, and the remaining gap is information you do not have. No amount of model muscle compensates for missing information.

A flat line doesn't prove that on its own, though. Four other things produce the same plot: labels noisy enough that the model is fitting disagreement, an eval small enough that a two-point gain is inside the noise, a weak model that a stronger one would beat, or optimization that never ran long enough or wide enough to find the gain. Rule those out before you call the ceiling. The most useful check is to hand a domain expert the exact inputs the model sees, nothing more, and measure how well they do. If the expert also struggles with those inputs, investigate missing information before spending another cycle on tuning, and ask the expert what evidence would have let them answer reliably. Their failure supports the information-limit reading without proving it: it says the inputs are probably short, not that nothing better could be done with them. If the expert clears it easily, the signal is in the inputs and the model isn't finding it, which is a tuning problem. Check the labels and the eval size the same

afternoon: re-grade a sample with two reviewers, and ask whether your last three "flat" runs differ by less than the eval's own run-to-run spread.

A concrete instance shows the shape. A team building a RAG system starts at sixty-eight percent accuracy on the base LLM. Better prompts add five points. Fine-tuning adds eight. Tuning retrieval adds another six. Eighty-seven percent. Every fix moved the metric. Then the next fix lands at eighty-seven again. So does the one after. The team is now arguing about which fine-tuning recipe to try next, and the harder question is whether the remaining thirteen percent is the same kind of problem as the first nineteen. Usually it isn't. The errors that survived every fix are the ones the current frame can't reach. The diagnostic test is short: look at your last three to five fixes and ask whether each one moved the number. If yes, you're solving a model problem and the next fix probably works. If no, the plateau is the diagnosis.

To move past the ceiling, the move is to add information. That can mean engineering better features from the data you already have, interaction terms, graph structure, anything that pulls latent signal out of the existing rows. It can mean acquiring new data sources, the canonical example being weather added to a sales forecasting model that already had everything else. Or it can mean changing the target itself. If you can't predict the price, predict the volatility. Same data, different question, ceiling moves.

The policy bot: naming its failure mode. The bot we diagnosed in Chapter 3 sits here. The team had run three tuning cycles in the six weeks after launch, all of them scored against the curated eval, where every cycle looked like progress. Once the eval was rebuilt on production traffic, they re-scored that history against the weekly production samples, which had been collected all along and read by nobody. The first cycle, prompts and a reranker, had bought six points, from fifty-two to fifty-eight, and reading the traces showed where those points came from: the reranker recovered the multi-hop cases where both documents happened to share the question's vocabulary. That slice of the cluster was a setting, and tuning fixed it. The next two cycles bought nothing. The last three weekly readings are fifty-eight, fifty-seven, fifty-eight, because what's left is the other slice: questions where the second document is named inside the first and uses words the question never contains. The reranker and a larger top-k were tried on those and still missed the dependent document, which is as far as the team could push the one-query design. The eval spread is about two points week to week, so three readings inside that band is flat. The expert check points the same way: a policy specialist handed only the top five retrieved documents gets these questions wrong too, and when asked what they'd need, names the addendum that isn't in the five. The information exists in the corpus. It never reaches the model in one retrieval call. That's an information ceiling, sitting at the retrieval layer instead of the feature layer.

Region failure

"It works globally and breaks for one segment." Overall accuracy is eighty-seven percent, but the rural user segment only gets forty-one percent. The aggregate metric is hiding a multi-modal distribution underneath, and one model is being asked to fit two populations that don't share a decision boundary. The segment is a user cohort in classic ML, but it's the same failure mode when a RAG system nails

short questions and fails multi-hop ones, or an agent breezes through simple tickets and stalls on the complex ones.

When that's what's happening, the model is averaging two answers and getting neither right. The bias term that minimizes loss across the whole dataset is wrong for both subgroups, just in opposite directions. Adding more rows that look like the ones you have won't fix it. The assumption "one model can fit all of this" was wrong.

Before you conclude that, check the boring explanations, because a bad subgroup number has four causes and only one of them is a second decision boundary. The segment may be small enough that forty-one percent is twelve examples. Its labels may be worse, because the labelers knew the urban cases and guessed at the rural ones. It may be underrepresented in training, so the model never saw enough of it to learn anything. Or the features that carry signal for the big segment may be missing or empty for the small one. Each of those is a tuning problem with a different fix: more examples, re-labeling, re-weighting, new features. Only when the segment is well-represented, cleanly labeled, and still failing have you found a population that needs its own model.

The fix, once you're there, depends on how distinct the segments really are. If they have different decision boundaries, decompose the system into per-segment models and route at inference time so each population gets its own answer. If they are close enough in distribution to share an architecture, add the segment as a feature and let the model split internally. At minimum, report subgroup metrics alongside the aggregate so the gap stops hiding, and accept that one of the segments will drag the average until you can address it.

Confident wrong answers

"It's confidently saying things that aren't true." The system returns an answer, the answer is wrong, and the confidence attached to it is high. Confidence and accuracy have decoupled, which is dangerous because everything downstream, routing, auto-approval, whether a user trusts it, reads that confidence as if it meant something.

The same pattern shows up in every stack. In GenAI it's hallucination: the model encodes facts as patterns over tokens, which breaks on niche, recent, or proprietary content. In ML it's an overconfident label on an input unlike anything in training. Neither is fixed by prompting harder or training longer, because the gap is missing information, not effort.

The fix is to stop trusting the model's own confidence and ground it in something external. In GenAI, that's retrieval from a verified store, citations that make a human check cheap, or constrained output that forbids invented entities. In ML, it's calibration so the confidence is honest, plus an abstention threshold so the system can say "not sure" instead of guessing.

Runaway (won't stop)

"It just keeps going." The clearest version is an agent that calls a tool, the tool fails, it tries something close, that fails too, and the loop repeats until you run out of context or money. From the outside it

looks like thinking. From the logs it's the same move eight times. The same pattern is any system with no natural stopping point: an unbounded search that never converges, a generation that won't terminate, a retry policy with no ceiling.

The root is that the task wants more steps than a single pass can carry, and nothing inside the system recognizes it's stuck. For an agent, more autonomy just makes the loops longer.

The fix is a bound. Hard-code the steps that don't need a model to decide, constrain the tool output to a whitelist so the model can't invent calls, or split the task into pieces small enough that one pass finishes each. Vercel rebuilt their text-to-SQL agent on this principle: eleven specialized tools became one, bash, and the model could grep, cat, and find on its own without getting tangled in the scaffolding the team had built. Loops stopped, success rate went up, latency went down. Whatever the stack, the move is the same, cap the steps, the spend, and the retries, and hand off to a human when a cap trips.

Operational red flag

"Our cost or latency is unsustainable." P99 latency that has slipped from two hundred milliseconds to two seconds, with customers timing out before the response lands. Throughput that handled one thousand QPS in staging and falls over at ten thousand. A spend curve that climbs past the budget two weeks before quarter-end. Different symptoms, same root.

The architecture's per-request compute is too heavy, or there are no circuit breakers limiting how much work a single task can do, or both. The model can be perfect and the system still fails, because the system is paying for that perfection per request and per millisecond.

The fixes are familiar but unglamorous. Precompute what you can move from read time to write time. Cache aggressively, especially at the embedding and retrieval layers. Use smaller models for the easy cases and reserve the big one for the hard ones. Set hard limits on tokens per request, retries per task, dollars per day. Sometimes the right answer is that the architecture you shipped will not survive the second order of magnitude, and rethinking it now is cheaper than scaling it later. Whether the architecture scales decides more than the model's accuracy on benchmarks.

Reality gap

"It passed every test we wrote and the users are unhappy." Your evals are green. Your dashboards are clean. Support is full of complaints. The two pictures don't agree, and the eval set is the one lying.

Curated evals are a story the team told itself about what users would do. The story is wrong, every time. Real traffic has typos, partial inputs, code-switched languages, sarcasm, copy-pasted screenshots, adversarial prompts, and tasks the test set never imagined. The model is being graded on a test it has already memorized the shape of, while production is throwing examples that nobody wrote down.

Evaluate on real traffic. Sample real failures and write evals from those. Replace the hand-picked test set with a randomly sampled production slice that you refresh every month. Stop optimizing against the curated set. It is the model's home court, and you are not testing what you think you are testing.

Stalled pilot

"The demo looked great. Production is a mess." The demo passed. The launch went out. Six weeks later, adoption is flat and stakeholders are asking what's happening. The system technically works. Real users do not use it.

This one is close to the reality gap, and the difference decides the move. The reality gap is an evaluation problem: your test set didn't represent real traffic, so the model is failing on inputs you never measured, and tightening the eval is the move. A stalled pilot can have green evals and clean traces. The system is producing technically correct outputs and nobody is changing their behavior because of them. The failure is in fit, not correctness. The atomic unit you proved against (a clean question, a single user, a focused decision) doesn't match the workflow people are in. JP Morgan Chase had this exact shape with their privacy policy AI. The model worked beautifully. Adoption was crickets, because the demo had assumed users would trust an opaque AI output, and the production users wouldn't.

The fix is to run the GOATS Loop again with the production workflow in scope, including who reads the output, when, and what decision they make next. Redefine the atomic unit so it fits the real moment of decision, not the demo dataset. Reset stop criteria to what production can meet. Sometimes the answer is to ship something narrower that real users will use, and let the ambitious version stay on the roadmap.

The seven, side by side

Failure mode	Symptom	Diagnosis
Plateau	Metrics won't move with effort	Information ceiling, once labels, eval noise, and under-optimization are ruled out
Region failure	Works for some segments, not others	Multi-modal distribution, once sample size, labels, and representation are ruled out
Confident wrong answers	High confidence, wrong output	Knowledge gap
Runaway	Won't stop, cost climbs	Reasoning gap
Operational red flag	Cost or latency unsustainable	Architecture mismatch
Reality gap	Tests pass, users complain	Eval mismatch
Stalled pilot	Demo wins, production loses	Framing gap

The failure mode tells you something is off. The diagnosis tells you what. Neither tells you what to do, and that is the work of the next two sections. The first is whether to act at all: persist, pivot, or stop. The second is which move to make once you have decided to act.

The three options

Once you have named the failure mode, you have three choices. Most teams only consider the first.

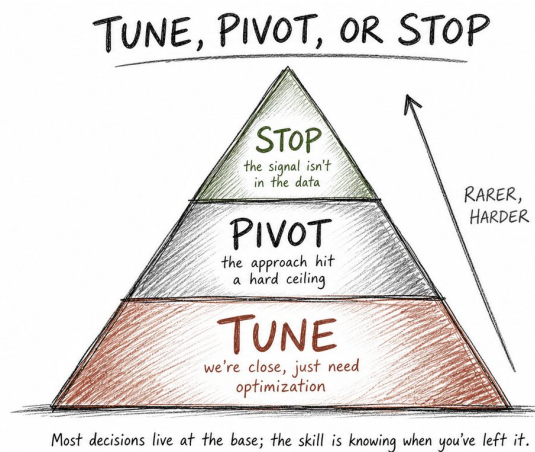


Figure: Most decisions live at the base; the skill is knowing when you have left it.

Persist

Persist means tune the current frame. The failure mode is real but weak, and the diagnosis points to something fixable inside the current architecture.

Choose persist when the failure mode is inconsistent, when you have not exhausted the obvious fixes (prompt tweaks, more data, parameter tuning), or when there is a specific, testable hypothesis for why the next iteration will be different. The trap with persistence is that it becomes procrastination. If you have run three major iterations without meaningful improvement, persist has become a refusal to face the other two.

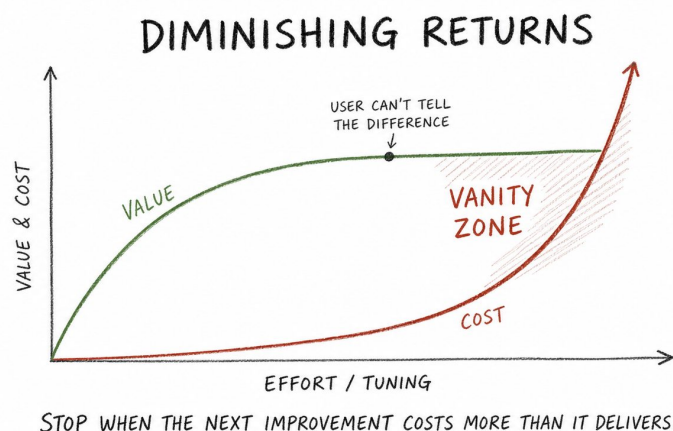


Figure: Persistence becomes procrastination once each iteration costs more than it returns.

Ask what business outcome changes if you go from eighty-five percent to ninety.

Improvement	Cost	Value change
70% to 80%	2 weeks	Unlocks the use case
80% to 85%	4 weeks	Measurable lift
85% to 90%	3 months	Marginal improvement
90% to 95%	6 months	Negligible difference to users

When the cost of the next five percent exceeds the value it delivers, you've passed good enough, and persisting is buying accuracy nobody will notice.

The questions that pull persistence apart are short. Have you tried the obvious fixes, or are you assuming they won't work? Is there a specific hypothesis for the next iteration? What is the deadline for seeing improvement? If you cannot answer all three, you are stalling, not persisting.

Persist doesn't always mean understand and fix. Three stories from large-model training make the case for routing around a problem when the workaround is cheap and the debugging isn't.

PaLM's bad batches. During PaLM-540B training the team saw dozens of loss spikes at highly irregular intervals, caused by an unidentified combination of data batch and model parameter state. The spikes were irreproducible. Instead of debugging for months, they implemented batch skipping: on a spike, restart from the last checkpoint and skip the batch.

IDEFICS's memory leak. During IDEFICS-80B training the team found a tiny CPU memory leak that took days to run a node out of memory. Debugging would have delayed training significantly, and the leak was slow enough that monitoring could catch it, so they set up automatic graceful exits with SLURM job arrays that restarted from checkpoint. They trained to completion without ever fixing the leak.

BLOOM's hardware plan. The BLOOM-176B team learned that new GPU accelerators fail at rates as high as ten percent early on. Their starting assumption had been that hardware works. The reframe was to assume it will fail and design for it: fixed node allocations, five to ten percent spare capacity for automatic failover, multi-layered watchdogs, automatic node draining. Training completed despite multiple hardware failures. (Stas Bekman, ML Engineering Open Book)

When a problem is irreproducible or slow and the workaround is cheap, a documented workaround that unblocks progress beats an extended debugging effort. That is persist done well: a specific hypothesis, a time box, and a fix that fits the frame.

Pivot

Pivot means change the frame. The diagnosis surfaced a fundamental mismatch: wrong problem type, wrong atomic unit, wrong metric. The same error pattern keeps recurring despite fixes. The architecture cannot support what the work requires.

A pivot takes what you learned and applies it to a better frame. Every test you ran, every assumption you broke, every failure you cataloged carries forward.

The catalog of moves available to you sits later in this chapter, under The reframing moves. Before getting there, three questions to pressure-test the decision itself: what would we do differently if we started over with what we know now? What is the simplest architecture that could work? Is the complexity we have built paying for itself?

Salesforce's agent sprawl. Salesforce tried building hundreds of agents simultaneously across the enterprise. The result was duplication, low adoption, and blurry results across the board. The pivot was focused "hero agents" with measurable ROI in targeted areas: Data 360, Salesforce Help, Sales Engagement. Their phrasing was the right one: "you have to earn the right to scale fast." The symptom was that no individual agent had clear ROI. The pivot was structural, depth before breadth. ([Salesforce](#))

DoorDash's collaborative ecosystem. DoorDash moved from single isolated agents to a collaborative multi-agent ecosystem. The symptom was that individual agents solved narrow problems but could not share context or coordinate. The pivot was "make the agents work together," not "make the agents smarter." That is a decomposition pivot at the system level.

Uber's agentic-RAG. Uber's Engineering Security team built an on-call copilot over more than forty internal security and privacy policy documents. Plain RAG wasn't good enough to roll out more widely: many answers were incomplete or wrong, because the right answer hinged on subtle distinctions within one policy and across several. The pivot was an agentic-RAG pipeline. One agent rewrites and breaks down the question, another picks which documents to search, retrieval runs, and a post-processing agent cleans up the context before the answer is written. Acceptable answers rose 27% and incorrect advice fell 60%, both relative. ([Uber Engineering](#)) Sometimes the right direction on the automation spectrum is up, not down.

Quantile balancing: proxy to direct. Mixture-of-Experts models suffer from "dead experts." Underloaded experts learn less, fewer tokens select them, and the imbalance compounds. The standard fix is an auxiliary loss: add a proxy penalty for imbalance, tune the hyperparameter, hope it stabilizes. Jianlin Su asked a different question. What bias update would have maximized balancing in the previous step? Then he solved for it directly. Result: zero loss spikes, no hyperparameters,

experts that previously stayed dead for fifty-plus steps activated quickly. The reframe dropped the proxy entirely and computed the real correction.

Stop

Stop means stop building. Choose this when the problem itself is wrong (not worth solving, or not solvable with current AI), the cost of success exceeds the value of success, trust is broken and cannot be rebuilt, or regulatory or liability risk is unbounded.

What counts as good enough depends on what happens with the output.

Scenario	Good enough	Why
Fraud detection	95%+ precision	False positives cost customer trust
Content recommendations	60%+ relevance	Users tolerate some misses
Medical triage support	99%+ sensitivity	Missing a case is catastrophic
Email categorization	80%+ accuracy	Humans review anyway
Internal search	70%+ top-3 relevance	Users can scan a short list

If a human reviews every prediction, a lower threshold is acceptable. If the system acts autonomously, you need a higher one. If errors have legal or safety consequences, near-perfect or don't ship. Google's People + AI Guidebook makes the same point: design for the human-AI collaboration, not just the model. Decide the threshold before you start.

The stop signals are concrete. User satisfaction below 2.5 out of 5. "Made it worse" rate above ten percent. Hallucinations on compliance, medical, or legal content where one wrong answer is a lawsuit. Negative ROI even in the best case. Three or more pivots that haven't moved the needle.

Stopping feels emotional. Make it mathematical.

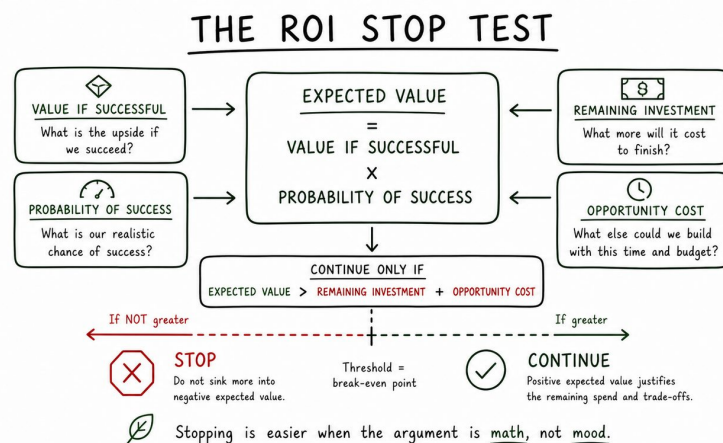


Figure: The stop decision gets much easier when the conversation is about break-even math instead of sunk-cost emotion.

Factor	Current state	Break-even requirement
Value if successful (\$/year)	\$ _____	
Probability of success	____ %	
Expected value	\$ _____	Must exceed investment
Remaining investment needed	\$ _____	
Opportunity cost	\$ _____	What else could this team build?

Stop when expected value is less than remaining investment plus opportunity cost, or when the probability of success has dropped below a viable threshold.

The hardest part of stop is the conversation, not the analysis. Someone will say "we've already spent two hundred thousand dollars, we can't stop now." The reframe is short.

"We have two hundred thousand dollars in lessons learned. The question isn't whether that was worth it. It's done. The question is what's the best use of the next two hundred thousand."

Sunk costs are sunk. The only relevant question is forward-looking ROI. (This is the [Sunk Cost Fallacy](#). Naming it out loud in a pivot conversation gives stakeholders language to recognize the bias rather than defend it.)

OpenAI's Sora. The consumer app launched at the end of September 2025 to real excitement. By the app-analytics estimates reported in the press, downloads peaked around three-point-three million in November, fell about a third in December, and fell again by nearly half in January. OpenAI announced in March 2026 that it was winding the product down, closed the app and web product in late April, and retired the API in September. The cost figure that circulated, roughly fifteen million dollars a day in inference, is an outside estimate, and the two-million-dollar figure it got compared with was reported consumer revenue, so the two numbers aren't the same kind of thing and I'd treat the ratio as directional. Katie Notopoulos, one of the heaviest users, put the product problem plainly: "Making videos of yourself is fun, but watching videos that strangers make of themselves is not fun." I don't know what stop criteria OpenAI set internally, and I'm not claiming there were none. From outside, the signal that would have served is obvious: two consecutive months of more than twenty percent download decline, which fired at the end of January. The announcement came about two months later. At the outside estimate, that gap alone is on the order of a billion dollars of inference, and even at a tenth of that estimate it is a lot of money to spend between a signal and a decision. JP Morgan's privacy AI failed the other way: green metrics, no adoption, pivot. Sora, from the outside, looks like the mirror image: red metrics, and a hold that lasted past them.

Go back to the failure record here. At the pre-mortem you searched the [AI Incident Database](#) to imagine how your class of system might fail. Now you have a real failure in hand, which makes the

same search far more pointed: has this exact failure mode hit this kind of system before, and did anyone recover from it? If several teams hit the same wall and none pivoted out of it, the wall is probably structural, and that precedent turns the stop call into a question about evidence instead of sunk cost. Do this research before you commit to another pivot, not after the third one fails.

How to choose

The three options compress into a small decision table.

Evidence strength	Diagnosis	Recommended action
Weak or inconsistent	Unclear root cause	Persist. Try obvious fixes, time-box it.
Strong and consistent	Clear architectural mismatch	Pivot. Change the frame.
Strong, trust broken	Fundamental problem	Stop. Redeploy resources.
Strong, unbounded risk	Liability or safety issue	Stop. No architecture fixes this.

And the same logic as a branching table, in case you want it on a wall.

If you built a Failure Funnel in Chapter 3, it tells you where to run the test, and it's worth being careful here, because the funnel and the fifty-row sample are measuring different things. The sample clusters failures by what went wrong. The funnel locates them by which pipeline step they died at. A cluster in the sample and a leak in the funnel can both be true of the same failures, and neither one decides tune or pivot by itself.

Failures concentrated at one step give you a place to run the repair test. Take the cases dying at that step and ask whether a change inside the current design fixes them, with evidence. Persist when it does: a specific change, tried on the failing cases, with the number moving. Pivot when the step is failing because of an assumption the design can't give up. The policy bot's funnel leaked at retrieval either way, and the two slices of that leak got opposite answers. The cases where both documents shared the question's vocabulary were a reranker away, so that was a tune. The cases where the second document needed a query written after reading the first weren't reachable by anything one-shot retrieval could be set to, so that was the pivot. Failures distributed across many steps lean toward pivot for a different reason: when every step leaks a little, the frame is usually asking the chain to do too much. A step that can't be fixed inside the design and can't be designed around means stop.

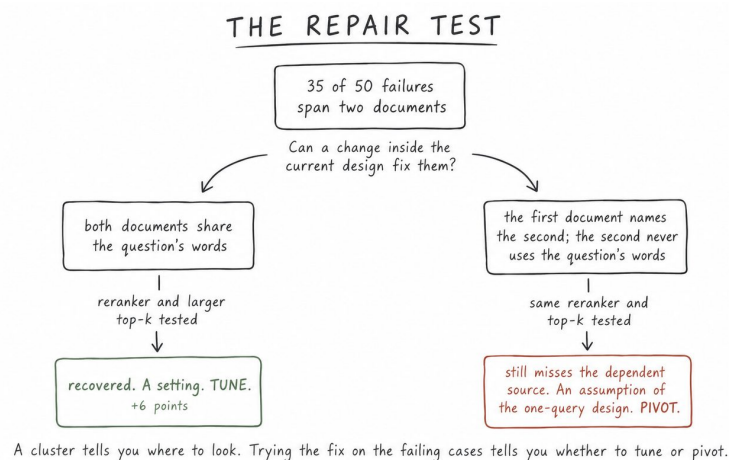


Figure: The same cluster, two answers. The branch is decided by trying the fix on the failing cases, and the conclusion is bounded to the design that was tested.

The policy bot: the call. The evidence is strong and consistent: two tuning cycles that moved nothing after the one that did, three flat weekly readings, and one cluster holding seventy percent of the failures. The diagnosis is a clear architectural mismatch: one-shot retrieval can't serve questions whose second document needs a query the first document supplies. That's the second row of the table. Pivot. Persist fails its own test here, because the reranker already took the slice a setting could reach, and nobody on the team could name a change inside one-shot retrieval that reaches the rest. Stop was on the table longer than the team liked. The baseline test had the FAQ template winning on grounded answers, which meant the bot as built shouldn't ship. What kept stop from winning is where the value sits: the multi-hop questions are the ones landing on the policy team, neither the bot nor the template reaches them, and the generation side already works once the right documents arrive. So the pivot has to prove it reaches that cluster, or stop is the next call.

The table assumes the evidence is in. Not every failure mode shows up in a metric first. Sometimes the dashboard is green, the output is technically correct, and it still feels off. That intuition is the judgment from Chapter 1, and it is valid grounds to act: when the dashboard says fine and your gut says otherwise, dig in before you defend the dashboard. Joe Maddon said it about baseball analytics: "There's a human element to this game that can't be ignored, and it's not always going to show up in the numbers." His read came from watching players, their demeanor and their confidence. Yours comes from context the metrics miss, team morale, technical debt, the organization's appetite for risk, and from having seen this failure mode before and knowing it doesn't get better. Use the data to inform the decision. Use judgment to make it.

The reframing moves

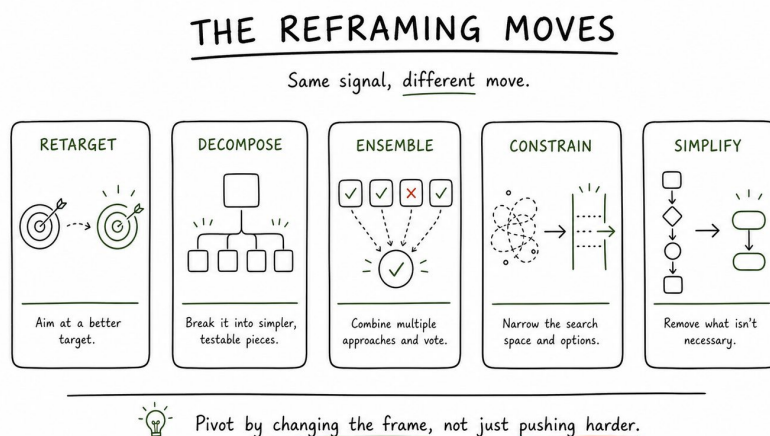


Figure: Once the decision is pivot, the next question is which move actually fits the failure mode.

Once the decision is pivot, the next question is which move. Most teams know one or two of these and reach for the same hammer every time. There are five: retarget, decompose, ensemble, constrain, simplify.

Each move below opens with the agent-era version, then the classical ML and GenAI forms it grew out of. If you came up through classical ML, you already know most of these under other names.

Retarget

Change what the system decides about. The atomic unit was wrong. You were predicting churn-in-thirty-days when the business question was "should we send a save offer this week." You were generating answers when the right unit was retrieving sources with citations. GOATS's first step is goal, and a retarget says the goal was misnamed.

Reach for retarget when the metric is moving but nobody cares, when the model is good at its task and the task is the wrong task, or when "if this were 100% accurate, what would still fail" produces a long list. The Mixture-of-Experts story above is a retarget at the loss-function level. The team stopped tuning a proxy penalty for imbalance and computed the correction directly.

In classical ML the most common retarget is changing the label. A churn model predicting thirty-day churn pivots to next-week churn because that is the window the save-offer team can act on. A recommender that optimized clicks pivots to watch time after the team realized clicks were rewarding bait. Retargeting also happens at the loss function: class imbalance penalties, focal loss, and label smoothing are all loss-level retargets, and the quantile-balancing story is the same idea. The third flavor is granularity. User-level becomes session-level, document-level becomes chunk-level, and each shift changes what the model is being asked to be good at.

In GenAI most retargets are about what the prompt is asking for. "Summarize this document" is the wrong target if the downstream user needs structured fields. "Answer this question" is the wrong target

if the user needs sources to verify the answer. The retarget is from generation to extraction, or from answering to citing.

Decompose

Break one decision into several. The single model or single agent was holding too much. Different segments behave differently. Different steps need different reasoning. Different parts of the workflow want different operating modes.

Reach for decompose when you see region failures, when one prompt is doing three jobs, or when the agent is holding state across phases that should be separate. DoorDash's pivot above is decompose at the system level: their isolated agents could not share context, and the move was to make them work together. LinkedIn's Hiring Assistant is decompose at the task level: instead of one agent running the whole recruiting workflow, separate sub-agents handle intake, sourcing, evaluation, outreach, and screening, so the atomic unit shifted from "handle the workflow" to "handle one phase of the workflow."

There's a data-shaped version of decompose: agentic map-reduce. When one LLM pass has to read a giant document or a whole corpus, it degrades. The context blows past the attention window, prompt fidelity drops, and the model starts missing things in the middle. The reframe is the oldest trick in distributed data processing. Map the operation over small pieces, one chunk or one document at a time, then reduce the pieces into the answer. The atomic unit stops being "the whole corpus" and becomes "one unit the model can hold."

DocETL's agentic rewrites. UC Berkeley's EPIC lab built DocETL, a system for LLM-powered document processing that treats this as the default move. You write a high-level operation, like "extract every safety violation from these ten thousand inspection reports," and an agentic optimizer rewrites it into a pipeline of smaller operators: split the long documents, map an extraction over each piece, then reduce the results, swapping in cheaper models or plain code where an LLM isn't needed. The reframe is that a single map over a long document is the thing that fails, and the split-

map-reduce decomposition is what makes it reliable. Same task, same models, decomposed into units each can handle. ([DocETL paper](#), [ucbepic/docetl](#))

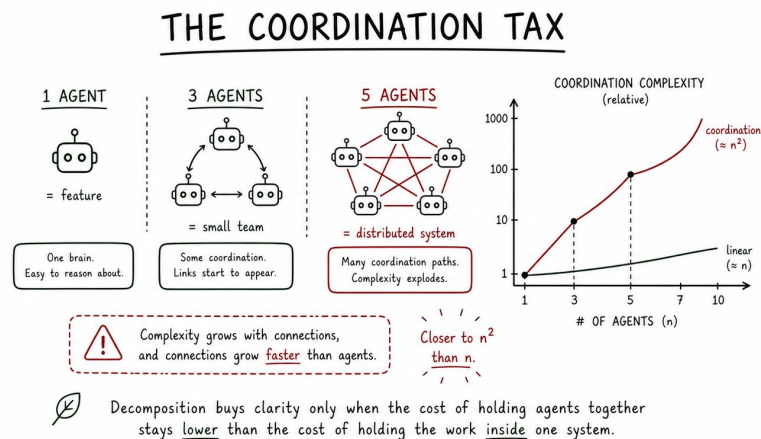


Figure: Decomposition buys clarity only when the cost of holding agents together stays lower than the cost of holding the work inside one system.

Decompose is not free. Every agent you add brings shared state, coordination between the pieces, and recovery when one of them fails partway through, and those costs grow with the connections between agents rather than with the count. Five agents that talk to each other carry ten links to keep consistent, plus every cache, race condition, and partial failure that comes with a distributed system. Reach for decompose when the cost of holding the work together still beats the cost of holding it apart.

Credit decisioning at five agents. A financial-services team built a five-agent credit pipeline: credit score, income verification, risk assessment, fraud detection, final approval. The first agent ran two weeks in production with zero issues. Three days after the other four were added, twenty percent of decisions had wrong risk ratings. Two days of debugging found the cause. The credit-score agent wrote a 750 to Postgres. The risk agent read a 680 from a stale cache 500 milliseconds later. The bug was in the architecture, not in any model, prompt, or agent. Multiple agents shared a cache layer with no coordination on invalidation. ([Bhaumik](#))

Factory's parallel swarm. Factory builds coding agents for multi-day missions; their longest has run sixteen days. Their first architecture was the obvious one. Ten agents working at once, ten times the throughput on paper. It didn't ship. Agents duplicated work, conflicted with each other, drifted from the goal. Luke Alvoeiro, who leads Factory's agent harness, said it cleanly at AI Engineer Europe: more agents wasn't the answer. The pivot was a serialized handoff. An Orchestrator plans the steps, a Worker executes one step at a time, a Validator checks before the next step starts. Same

models, different roles. The decomposition that worked was sequential, not parallel. (Factory, "How Missions Work")

In classical ML, decomposition shows up as per-segment models (one for enterprise, one for SMB), hierarchical models (a category classifier feeds a within-category model), and cascades (a cheap model filters, an expensive model handles the rest). Multi-task learning with a shared backbone is decomposition at the architecture level. Recommender systems are the canonical example: modern stacks split into retrieval, which finds about a thousand candidates from millions, and ranking, which orders the thousand. Different models, different objectives, different latency budgets.

In GenAI the pre-agent version is prompt chaining. One prompt extracts entities, the next reasons over them, the third generates the answer, and each step is narrow, debuggable, and testable on its own. Retrieval-augmented generation is itself a decomposition: retrieve first, generate second.

Ensemble

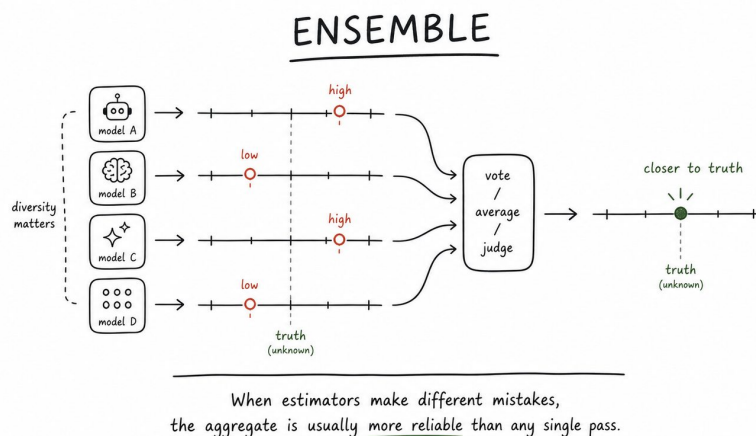


Figure: Ensemble works because individual estimators making different mistakes cancel each other out in aggregate, leaving a more accurate answer than any one of them produced alone.

Run the same decision more than once and combine the answers. The classical reason this works is the wisdom of the crowd. A group of independent estimators with uncorrelated errors beats any single one of them, because the errors cancel and the signal stays. Francis Galton noticed this in 1906 watching villagers guess the weight of an ox at a cattle fair. The median of the crowd's guesses was within nine pounds of the right answer. No individual was. That same logic powers random forests, gradient boosting, the winning Netflix Prize entry (a stack of over one hundred models), and most of the top-scoring Kaggle solutions of the last decade. It powers self-consistency in LLMs too: sample N answers, take the majority, and reasoning accuracy goes up by a real margin over a single sample.

Ensembling buys two things at once, and it helps to know which one you are after. The first is accuracy. When the underlying estimators make different mistakes on different inputs, the aggregate is right more often than any single one. The second is reliability. When any single inference is noisy, running it

more than once and combining smooths over the run-to-run variance. Which one you are getting depends on whether the single-pass model is wrong on average or just inconsistent.

Reach for ensemble when individual failure modes are uncorrelated and the accuracy lift from combining beats the cost of running multiple inferences, when self-consistency or majority vote across runs measurably outperforms a single pass, or when a judge model can catch errors the worker model can't. The cost is real: more inference, more latency, more complexity. The trap is ensembling across copies of the same model with the same prompt, which buys almost nothing because the errors are correlated. Diversity is the whole game. Different models, different prompts, different reasoning paths, different decoding parameters, anything that makes the estimators err in different directions on the same input.

The ML lineage runs through bagging (random forests), boosting (gradient boosted trees), and stacking (a meta-model over several base models), all buying accuracy at the cost of inference. Random forest is often the strongest baseline on tabular data, and gradient boosting libraries like XGBoost and LightGBM are the production default. GenAI has its own forms beyond self-consistency. A judge model that scores N candidate outputs is ensembling. Debate, where two models argue and a third decides, is ensembling with structured disagreement. The hybrid sandwich is the pragmatic version: a rule layer (a regex filter for PII), then the model layer, then another rule layer (a keyword check for hallucinations). Rules are cheap and deterministic, LLMs are expensive and probabilistic, so use rules to guard the LLM.

Constrain

Reduce what the system is allowed to do. Tighten the output space. Hard-code the parts of the plan that don't need to be reasoned about. Replace free-form tool use with a whitelist. Replace open-ended generation with structured output.

Reach for constrain when the agent is looping, when the output is malformed half the time, when the task has known structure that the LLM keeps getting wrong, or when production tolerates zero of a class of failure. Constraining is the most common agent-era pivot, and it gets its own worked example below.

Cloudflare's twenty-five hundred endpoints. The Cloudflare API has more than twenty-five hundred endpoints. Exposing them to an agent as standard MCP tools would take about 1.17 million tokens just for the tool definitions, more than the context window of any frontier model. Every call meant another round trip through the schema of available tools. Sunil Pai's pivot was to stop asking the agent what to do every few seconds. The agent writes one JavaScript program that executes the whole workflow inside a secure V8 sandbox. Same endpoints, same model, completely different

atomic unit, and the tool definitions shrink to about a thousand tokens. The constraint was on how many round trips the agent took, not on which APIs it could call. ([Cloudflare](#), "Code Mode")

Power BI into the assistant, not the other way around. Mudassar Iqbal, on Genevieve Hayes's Value Driven Data Science podcast, described how his product Lily sits next to the reporting tools people already use: Power BI pulls insights from Lily and keeps its dashboards, instead of users leaving Power BI for a chat window. ([Value Driven Data Science, episode 28, 41:08 to 41:53](#)) The constraint in that move sits on the agent's job rather than its output. The AI drives the reporting stack that already works instead of rebuilding reporting from scratch, which is often the difference between a pivot that ships and a rebuild that doesn't.

1Password: let the agent build the guardrails first. Refactoring a multi-million-line Go monolith, 1Password found the hard part was sequencing changes that break if they run in the wrong order, not generating code. Their fix was to use agents to build deterministic tooling, the analyzers and manifests and dependency maps, and then constrain the actual migration to run against those outputs instead of letting the agent reinterpret the codebase at every step. When the work is fully specified and bounded, they wrote, agents are fast and accurate. So they spent the agent on producing the specification, then took the reasoning away for the execution. ([1Password](#))

In classical ML, constraining is the difference between a regression head and a classification head. The classification head reduces the output space from the real line to N classes, a softmax constrains the output to a probability distribution, and a monotonic constraint on a tree model says the prediction can only move one way as a feature increases.

GenAI is where constraint techniques exploded. Structured output (JSON schema, function calling) constrains the output space at the decoding level. Constrained decoding, through logit biasing or grammar-constrained generation, goes further: the model cannot emit tokens that violate the grammar. Tool whitelists constrain what the model can call. Regex post-processing constrains what gets through. The figure above shows one specific move: instead of asking the LLM to calculate the total, ask it to write a Python script that calculates the total. LLMs are bad calculators and good coders, so shift the execution to a deterministic runtime. Every constraint trades flexibility for reliability, and most production GenAI is heavily constrained. The free-form chat interface is the demo, not the product.

Simplify

Move down the automation spectrum. Replace the agent with a pipeline. Replace the pipeline with rules. Replace the LLM call with a regex. The complexity wasn't paying for itself.

Reach for simplify when 80% of the value sits at 5% of the complexity, when costs or latency are unsustainable for the actual upside, or when the team cannot operate the system they shipped. One of the cleanest tells for simplify comes from the baseline test in Chapter 3. When the dumbest plausible

baseline (rules, templates, a lookup table, a human with a checklist) gets close to what the elaborate system produces, the diagnostic has already done the work, and simplify is the move that acts on it. Other tells (ballooning cost, unsustainable latency, an architecture nobody on the team can debug) point at simplify even when the baseline test was never run.

Simplify is the move teams resist most because it feels like a downgrade. The reframe is that the simpler version is what you can run in production. The more ambitious version stays on the roadmap.

Cursor's worktree skill. David Gomes opened a single PR. His own words: "I recently opened a PR removing this entire feature from Cursor. Around 15,000 lines of code deleted." The deleted code was orchestration logic teaching a coding agent how to manage Git worktrees, complex enough to be brittle, brittle enough to be a tax on every change. The replacement was two hundred lines of markdown the agent reads as a skill. The agent orchestrates itself. Same model, different theory of who should be doing the orchestrating.

The classical ML version of simplify is replacing a deep neural net with a gradient boosted tree, or a tree with a logistic regression, or logistic regression with a hand-tuned rule. Each step down loses some accuracy. Each step also makes the system easier to debug, faster to retrain, cheaper to serve, and possible to explain to a regulator. Feature simplification is another flavor: a model with two hundred features pivots to twenty because the other hundred and eighty added drift problems and no signal.

In GenAI the ladder is the same. Replace a fine-tuned model with prompt engineering, a multi-step chain with a single prompt, an LLM call with a regex when the inputs are structured enough, open-ended generation with template filling, the generative chatbot with retrieval-only search. Complexity has carrying costs. If simple captures eighty percent of the value at five percent of the cost, simple wins.

The headline pattern: full autonomous to constrained pipeline

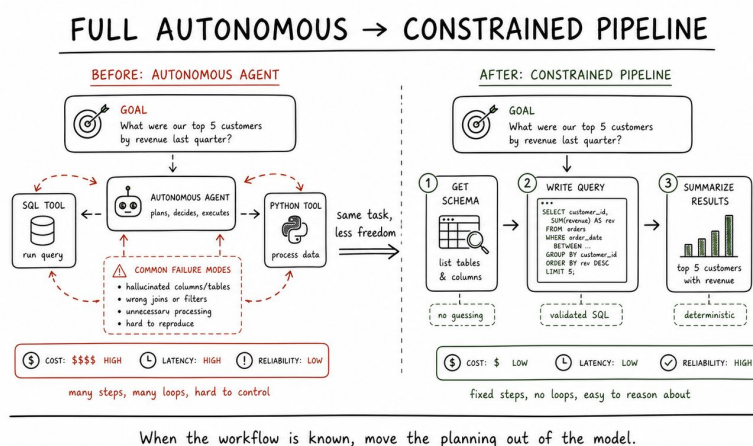


Figure: The common pivot is to move planning out of the model when the workflow is already known.

The pattern is moving from a full autonomous agent, where the LLM decides both what to do and how to do it, to a constrained pipeline, where you decide the steps and the LLM only fills in the content of

each step. It is the constrain move from earlier in this section, applied to the most common production scenario in the agent era. The reason it keeps coming up is that most tasks have more structure than the team gave them credit for when they started building.

Most teams start too high on the automation spectrum and pivot down. "Autonomous agent" is the loudest System 1 answer in the agent era, so of course you build an agent. Agents are what we build now. The harder question is whether the task you're handing the agent is genuinely open-ended exploration or whether it has a knowable shape that you could have hard-coded from the start. Most production tasks turn out to be the second, and the gap between what the team built and what the task required is what shows up later as loops, runaway costs, and brittle behavior under real inputs.

A team is asked to build a "data analyst agent" that pulls data, analyzes it, and writes a report. The PM frames it casually: "Just give it access to the database and let it write reports."

The first attempt is the obvious one. Give the LLM a goal ("write the report") and tools (SQL, Python). Let it figure out the plan. The result is an agent that gets stuck in loops, hallucinates column names, and costs about fifty dollars in API calls per report. Demo-impressive, production-broken. Running the GOATS Loop on the failure surfaces the diagnosis quickly:

GOATS step	What it reveals
Goal	"What does success look like?" Reliable reports at low cost.
Operating Assumptions	"What's the atomic unit?" A report. "What's the structure?" Three known steps.
Alternatives	Autonomous agent vs. staged pipeline vs. fully deterministic.
Trade-offs	Autonomy buys flexibility but pays in unreliability. Staging buys reliability but pays in flexibility.
Signals	Loops, high cost, unpredictable failures all point to too much autonomy.

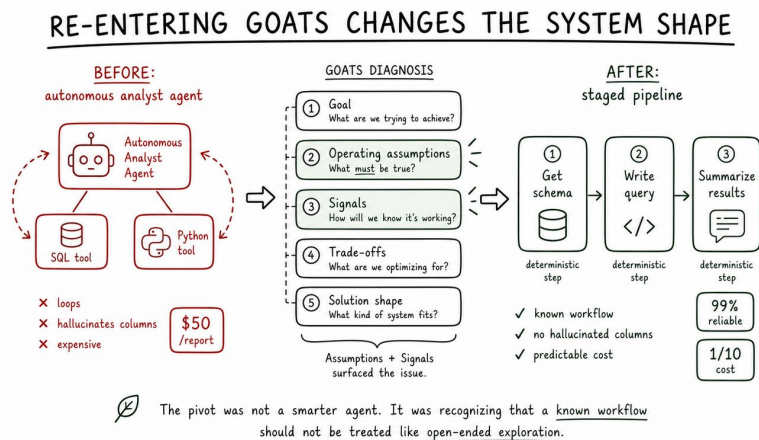


Figure: The pivot was recognizing that a known workflow should not be treated like open-ended exploration.

The diagnosis is that the team treated a known workflow like an unknown exploration. The task is three known steps that happen in the same order every time: get the schema, write the query, summarize the results. None of those steps requires the LLM to reason about which step comes next. The only place the LLM has to think is inside each step.

The reframe is to hard-code the pipeline so that step one always leads to step two always leads to step three, and use the LLM only for the content within each step rather than for the transitions between them. The result on this team's system was ninety-nine percent reliability, one-tenth the cost, and failures the team could debug, because when something went wrong they knew which step it went wrong in.

The general form is a question about who decides what, and the table below names the two ends of the spectrum.

Approach	LLM decides	You decide	Best for
Autonomous agent	What to do and how to do it	Only the goal	Exploration, unknown tasks
Staged pipeline	How to do each step	What steps to take	Production, known workflows

The more you know about the task structure, the less you should let the LLM plan it for you. Most teams learn this by building the agent first, hitting the loops and the bills, and discovering it the expensive way. The cheaper path is to ask, before you build: do I know the steps this task requires, in order, in advance? If the answer is yes, you have a constrained pipeline waiting to be built, not an agent waiting to be built.

The same diagnostic question can also point the other way. Sometimes the pivot is upward on the spectrum, not down. Harvey's engineers describe why straightforward RAG breaks down for legal research: it works for simple questions, but real research needs several rounds of gathering, and a single retrieval call can't change strategy as it learns, pick which sources matter, or tell when it has enough. Their answer was iterative agentic search. ([Harvey](#)) The structure of the task was less knowable than the team had assumed, and more autonomy was the right answer. Spotify's large-scale code migrations went a step further. They started with deterministic AST transforms, pivoted to AI agents that could read intent, then constrained those agents with verification loops to keep agentic flexibility from breaking the deterministic guarantees the migration required. The final architecture was hybrid: agent flexibility for the parts that needed it, deterministic verification for the parts that did not. ([Spotify](#))

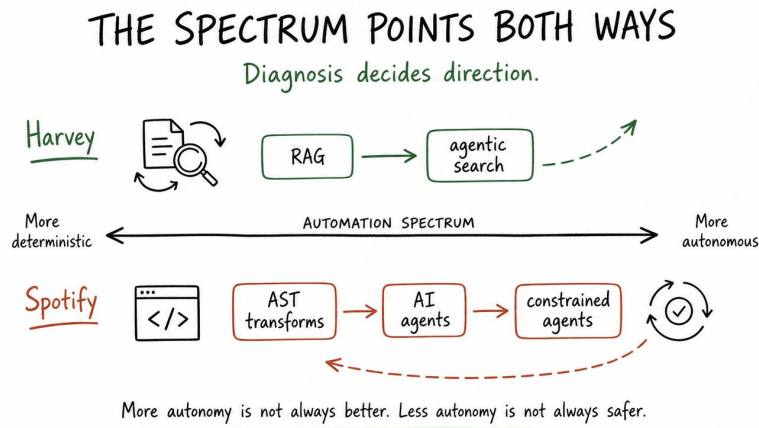


Figure: The automation spectrum is not a maturity ladder. The diagnosis tells you which direction to move.

Each of those teams re-entered the GOATS Loop with the evidence in hand and let the diagnosis tell them which way to move, rather than committing in advance to "more autonomy" or "less autonomy" as a virtue.

From failure mode to move

The seven failure modes from Name the failure mode and the five reframing moves connect, and the connection is the working core of this chapter. Treat the crosswalk as a starting bid, not a lookup table. The diagnosis in the middle column is what picks the move, and two teams with the same symptom can need different pivots. But if you've named your failure mode, here's where to look first.

Failure mode	Diagnosis	First move to consider
Plateau	Information ceiling	Add information. If none exists, retarget to a question the data can answer
Region failure	Multi-modal distribution	Decompose by segment and route
Confident wrong answers	Knowledge gap	Constrain : retrieval, citations, schema
Runaway	Reasoning gap	Constrain the steps, or simplify to a pipeline
Operational red flag	Architecture mismatch	Simplify , and add hard limits (constrain) while you do
Reality gap	Eval mismatch	Not a pivot. Fix the eval first (Chapter 3), then re-diagnose
Stalled pilot	Framing gap	Retarget the atomic unit to the real workflow

The reality gap row is the one to respect. If your eval doesn't match production, you don't know what's broken yet, and any pivot you make is a guess. The policy bot is the cautionary version: the team spent three tuning cycles optimizing against a curated eval before anyone checked whether the eval matched production. Fix the eval first. Ensemble is missing from the table because no single failure mode points

at it. Reach for it when the Replay Test from Chapter 3 shows a single pass is noisy and the accuracy lift from combining beats the cost of running more than once.

The policy bot: choosing the move. The plateau row says add information, and the information already exists in the corpus. It just never arrives in one hop. The move that fixes arrival is decompose: split the question into sub-queries, retrieve for each, and synthesize across what comes back, the same direction Uber's security team went when single-shot retrieval couldn't handle multi-hop policy questions. Constrain and simplify point the wrong way here, because the baseline test already showed the simple version losing exactly where the value is.

When you can't pivot: the Yamato Syndrome

Every move above assumes the system can be moved. Not all can. Some architectures are so tightly coupled that changing direction means a complete rebuild, and David Lewis has a name for how they get that way. The Yamato Syndrome is the pursuit of analytically impeccable, tightly coupled, highly optimized systems whose relevance depends on the continued validity of their assumptions. (David Lewis, "The Yamato Syndrome: Designing Naval Forces for Uncertainty," Substack, January 2026)

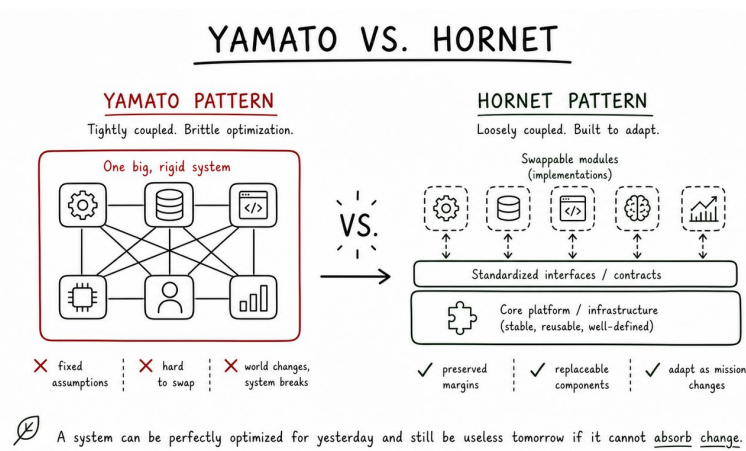


Figure: A system optimized for one frozen assumption set can still fail strategically if it has no room to absorb change.

IJNS Yamato (1941)	The American fleet carrier (Hornet CV-8, 1941, and the Essex class that followed)
Designed for a known mission (battleship warfare)	Designed around an interface, the flight deck, not a weapon
Tightly coupled systems (18-inch guns, specific armor)	Standardized interfaces (flight deck, hangar, catapults)
Met all specs perfectly	Preserved growth margins

IJNS Yamato (1941)	The American fleet carrier (Hornet CV-8, 1941, and the Essex class that followed)
Never fought the fleet action it was built for. Sunk by aircraft in 1945.	The pattern outlived the hulls. Essex-class ships were rebuilt through the 1950s for jets and angled decks, flew helicopters, and recovered Apollo crews.

Two ships carried the name Hornet in those years, and the story only works if you keep them apart. CV-8, commissioned in 1941, launched the Doolittle Raid in April 1942, Army bombers off a Navy deck, and was sunk at Santa Cruz that October. Her name passed to CV-12, an Essex-class carrier launched in 1943, which was rebuilt twice after the war for jet aircraft, served into 1970, and picked up the Apollo 11 astronauts. (Naval History and Heritage Command has the ship histories for both.) No single hull adapted for forty years. The design pattern did, and that's the point. Yamato was technically superior. It delivered on schedule. Every metric was met. And it was strategically useless because the world changed faster than the platform could adapt. The carrier designers made a different bet: decouple the weapons from the ship. A standardized deck meant the platform could absorb a generation of aviation change every few years. The Doolittle Raid wasn't in CV-8's specs, and the jets CV-12 flew didn't exist when she was laid down. The architecture allowed both.

The translation to AI systems is direct.

Yamato pattern (brittle)	Hornet pattern (adaptable)
Model tightly coupled to specific features	Feature store with versioned, swappable inputs
Pipeline assumes a fixed data schema	Schema evolution handled gracefully
Inference logic embedded in the application	Model served through a standardized API
Metrics hardcoded to current business logic	Evaluation framework supports new metrics
"It works, don't touch it"	"It works, and here's how to change it"

You can only pivot if your system was designed to allow pivoting. A perfectly optimized system that can't adapt is a Yamato, impressive until the world changes.

Sign you're building a Yamato	What it means
Changing one feature breaks three pipelines	Tight coupling
"We can't swap the model without rewriting the app"	No interface abstraction
Adding a new metric requires code changes	Hardcoded evaluation
"The model assumes X will always be true"	Brittle assumptions
The entire team's knowledge is in one person's head	No documentation margin

Before you build, ask four questions. What assumptions could change in twelve months? Design interfaces around those. Can we swap the model without rewriting the pipeline? If not, add an

abstraction layer. Can we add new metrics without code changes? If not, build an evaluation framework. What's our Doolittle Raid, the unexpected use case the platform should be able to support?

Lewis's summary is that by standardizing interfaces and preserving margins, carrier designers insulated the platform from the pace of aviation change. Same principle for AI systems. Adaptability is a design requirement.

The shipping matrix

Not every decision deserves the same rigor. Before you act on a failure mode, classify the decision by how reversible it is and how much damage a wrong call would cause.

	Hard to undo	Easy to undo
High impact	Stop and audit. Get it right or don't do it.	Shadow mode. Run both systems in parallel.
Low impact	Time box it. Set a deadline, decide then.	Just ship it. Learn from real usage.

The matrix becomes practical when you apply it to actual pivot decisions.

Pivot type	Reversibility	Consequence	Action
Switch from agent to pipeline	Easy (keep the agent code)	Medium (changes UX)	Shadow mode: run both, compare
Stop the project entirely	Hard (team reassigned)	High (political cost)	Stop and audit: align before acting
Change the metric	Easy (track both)	Low (internal only)	Just ship it: measure both for two weeks
Rewrite the architecture	Hard once committed (months of work)	High (full rebuild)	Split it. The prototype is easy to undo, so time-box it. The commitment to rebuild is the hard-to-undo, high-impact call, and that one gets the audit

The matrix matches the cost of being wrong to the cost of deciding. A reversible decision in shadow mode is cheap. A non-reversible decision deserves the audit, even when the engineering instinct is to just ship. The last row is the one teams misread. An architecture rewrite looks like it belongs in the stop-and-audit cell, and the commitment does. What the time box buys is a way to make that commitment with evidence: the two-week prototype is cheap to throw away, so you run it first, and the audit happens on what it shows.

THE SHADOW MODE PATTERN

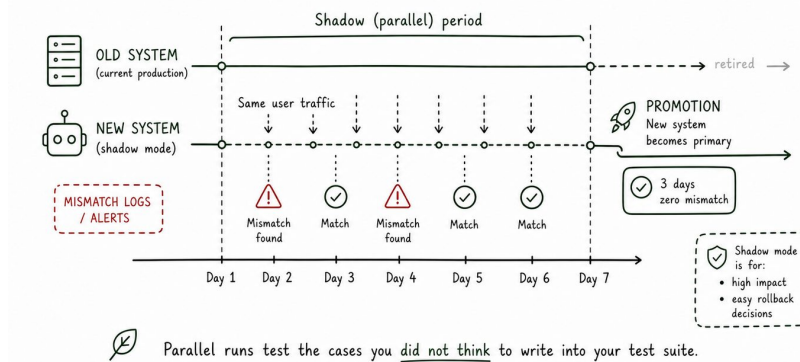


Figure: Shadow mode is how you validate high-impact changes against the messy cases your test suite did not anticipate.

Reco's JSONata rewrite. Reco's policy engine evaluated JSONata expressions across billions of events. The team rewrote the entire evaluator in Go: thirteen thousand lines, 1,778 test cases, four hundred dollars in AI tokens, built in seven hours. They didn't just ship it. Billions of production events made it high impact, and feature flags made it easy to undo, so it landed in the shadow mode quadrant. For five days the new evaluator processed all production traffic in parallel with the old one, with every mismatch logged and alerted. After three consecutive days with zero mismatches it was promoted to primary. Along the way the parallel run surfaced bugs in the old implementation, cases where it didn't follow its own spec. The result was a thousandfold speedup on common expressions and about three hundred thousand dollars a year saved from retiring the old fleet, part of five hundred thousand a year once a related rule-engine refactor is counted. ([Reco](#))

Tests reflect the cases you thought of. Shadow mode tests against the cases your users send.

The policy bot: how much rigor. Swapping one-shot retrieval for iterative retrieval is months of work if the team commits blind, so it lands in the time-box row: two weeks to prototype, with the current bot left running untouched. The thirty-five multi-hop failures from the Chapter 3 sample are the development set, the cases the team debugs against while building. They can't also be the test, because a prototype tuned on thirty-five questions will pass those thirty-five. So the bar is set on fresh data: forty multi-hop questions pulled from the two weekly samples after the prototype is frozen, graded by the same two reviewers. Clearing seventy percent on those, with no drop beyond the eval's own two-point week-to-week spread on a hundred single-document questions it was never meant to improve, and with cost and latency per query written down next to the accuracy, is what earns the next step. The tolerance is written down now, before the result, because a one-case difference on a hundred questions is noise and a five-case difference is a regression, and the team shouldn't get to decide which after seeing the number. Then both versions run in shadow mode for

two weeks before anything switches. Reversible at every step until the cutover, and by then the numbers will have made the decision.

Communicating the decision

Recognizing the failure mode is the analytical part. Communicating it is the part that breaks projects.

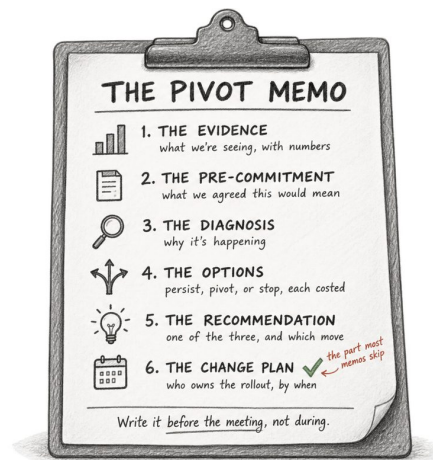


Figure: Everything in this section compresses into one memo with six parts. The change plan is the one most memos skip.

Why pivot conversations fail

Pivot conversations feel like delivering bad news. They trigger defensive reactions, sunk cost bias, and ego protection, often all in the same five minutes. The same words land in different ears.

You say the model isn't working and they hear "you failed." You say you need to change direction and they hear "we wasted three months." You say you think you should stop and they hear "you're giving up." Nothing in that first column is inaccurate. It lands as a verdict anyway, because the person across the table has their name on the project.

The right column is the same information routed around that. The fix is to frame pivots as the system working correctly, not as failure, and the pre-commitment from Step 5 is what makes that framing possible.

Bring evidence, not opinions

The single biggest difference between a pivot conversation that lands and one that doesn't is whether you brought numbers.

The structure that works is short and the same every time. The evidence: here is what we are seeing, with data, not opinion. The pre-commitment: here is what we agreed this would mean, from Step 5.

The diagnosis: here is why it's happening, from the diagnostic tools. The options: here are our choices, persist, pivot, or stop.

The pre-commitment anchor

If you did Step 5 correctly back in Chapter 2, the conversation gets much shorter. You are not asking permission to pivot. You are honoring an agreement.

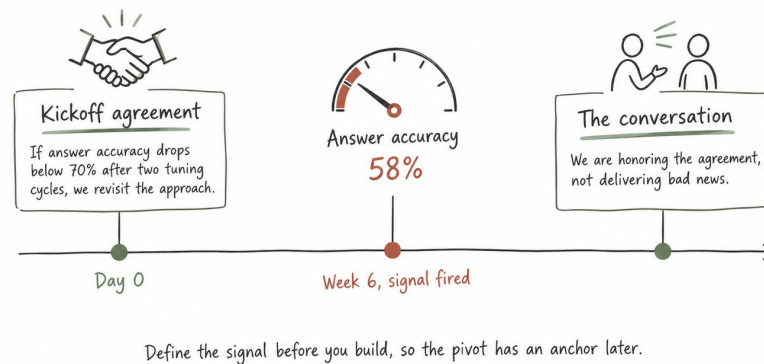


Figure: A threshold defined up front turns the later pivot into follow-through instead of improvisation.

"In our kickoff, we agreed that if answer accuracy dropped below 70%, we'd revisit the approach. We're at 58%. I'm not recommending we stop. I'm flagging that the signal we defined has fired. Here are our options."

The decision was made at kickoff. You are executing it.

If you didn't define signals upfront (many teams haven't, and this is the most common reason the conversation gets ugly), the conversation is harder by definition. You are asking stakeholders to accept a new frame and bad news at the same time. Define Step 5 before you build.

The agent demo problem

The agent era adds a specific communication wrinkle. The demo always looks great.

Stakeholders saw the agent handle a complex query in a demo. Now you are telling them you want to scale it back to a constrained pipeline. From their seat, that feels like a downgrade.

The reframe is to show the distribution.

"The demo showed what the agent can do on its best day. Here's what it does on an average day. [Show the failure distribution.] The constrained version handles ninety percent of cases with zero

failures, and we can ship it next week. The full agent stays on the roadmap for the ten percent that needs it."

Same decision, different story, very different reception.

The language of pivots

Small wording changes shift these conversations. The pattern in every swap is the same: replace a judgment with an observation. "The agent doesn't work" invites a debate about whether it works. "The agent fails on forty percent of real inputs, and the failures cluster in two shapes" invites a look at the data.

Loaded version	Accurate version
"The agent doesn't work"	"The agent fails on 40% of real inputs, clustered in two shapes"
"We need to scale back"	"We're shipping the version that handles 90% reliably now. The ambitious version stays on the roadmap"
"The approach was wrong"	"The signal we defined at kickoff fired"
"We have to start over"	"We're keeping the evidence and changing the frame"
"This isn't working"	"Here's the distribution. The average day looks like this"

None of these are euphemisms. They are accurate, and they leave the room with somewhere to stand.

When stakeholders resist

Resistance shows up in patterns. Each one has a specific shape and a specific response.

Sunk cost sounds like "we've already spent two hundred thousand dollars." The response is the forward-looking reframe from The three options: the money is spent either way, and the only question is the next dollar. The demo defense sounds like "but it worked when we showed the board." The response is the distribution: best day versus average day. Ego protection sounds like "this was my initiative," and it's usually unspoken. The response is the pre-commitment, because a fired signal is nobody's fault, and crediting the champion for defining the signal turns the pivot into their process working. The deadline objection sounds like "we don't have time to change course." The response is that persisting on a broken frame costs more time, and you can show the last three iterations that moved nothing. And the announcement trap sounds like "leadership already told the street." The response is the narrower ship: deliver the reliable version under the same banner.

Pattern	What it sounds like	The response
Sunk cost	"We've already spent \$200K"	Forward-looking ROI. The only question is the next dollar

Pattern	What it sounds like	The response
Demo defense	"It worked when we showed the board"	Show the distribution: best day versus average day
Ego protection	"This was my initiative" (usually unspoken)	The pre-commitment. A fired signal is the process working, and the champion defined it
Deadline	"We don't have time to change course"	Three flat iterations cost more time than the pivot
Announcement trap	"Leadership already announced it"	Ship the narrower reliable version under the same banner

One thing to know going in: resistance usually doesn't come from the user. Stanford's enterprise AI study found that Legal, HR, Risk, and Compliance accounted for thirty-five percent of resistance, ahead of internal end users at twenty-three percent. End users will tell you the tool is annoying. The staff functions will tell you the project can't ship at all, often after you've already built it. If you only war-game the user objections, you'll lose the project on a Tuesday in a meeting you weren't invited to. Bring those functions in early as collaborators on the constraints, not late as gatekeepers on the rollout.

Technical quality is half the problem

Building a good model is necessary but not sufficient.

"Building a really good model is one piece. The change management of working inside enterprises is a whole other matter."

MIT's NANDA group put a number on this in 2025: by their count, ninety-five percent of enterprise generative AI pilots showed no measurable effect on the P&L. The headline travelled further than the method did, so treat it as a survey finding rather than a law, but the shape it describes is the one this chapter is about. The models in those pilots mostly worked. The failure lived in the layer above the model, which is where JP Morgan's privacy AI failed too.

Stanford's 2026 Enterprise AI Playbook backs this from the other direction. They studied fifty-one successful AI deployments across forty-one organizations and asked what the teams struggled with. Seventy-seven percent of the hardest challenges were "invisible costs": change management, data access, process redesign, governance. Sixty-one percent of the successful projects had at least one prior failure that doesn't show up in the final ROI story. The technology worked. Everything around it was the work. ([Stanford](#))

The teams that beat the odds treat the model as the foundation, not the product. Four pillars sit on top of that foundation, and most failed projects fail in one of them.

Usability is whether anyone can use the tool. JP Morgan's pivot was four UX moves: orchestration, context, observability, governance. The model stayed the same, the interface around it changed, and adoption followed. Utility is whether the tool fits the workflow. If the AI is a separate tab people have

to remember to open, the AI loses to the flow they were already in. Value is whether the time and money spent compound. Clear metrics beat flashy demos every time. Sustainability is whether leadership owns the change long enough for adoption to stick. A champion gets you a pilot. A P&L owner gets you a deployment.

The sustainability pillar is the one BCG keeps measuring across enterprises. In their 2024 study, the AI leaders put twice the share of revenue into digital and AI that other companies did, dedicated twice the share of their people to digital and AI work, and scaled more than twice as many AI products across the organization. (BCG, "[Where's the Value in AI?](#)") The gap is organizational commitment, not technology.

If your team needs an off-the-shelf structure to argue for change-management investment, Andrea Marchiotto's [Three-Layer Knowledge Architecture](#) is one: an engineering source-of-truth layer, a translation layer owned by product enablement with a 48-hour-per-sprint turnaround, and a community layer of champions with real incentives. It's the same MIT ninety-five percent stat underneath, turned into an operating model.

That layer is also where most pivots live. If you're losing because nobody adopted the tool, the fix is the UX, the workflow integration, the ROI story, or the executive sponsorship, not a better model. Reading the failure mode correctly means recognizing which pillar is broken, not adding accuracy to a model that wasn't the bottleneck.

The pivot conversation you are designing sits inside that same problem. The evidence-based structure and the pre-commitment anchor both help. So does understanding who in the organization benefits from the current approach staying in place.

Before you deliver a pivot communication, ask three questions. Who championed the current approach, and are they in the room? Which department's workflow depends on the current output? Is there a "page 3 problem," where even if you build the right thing, nobody updates the process to use it?

A technically correct recommendation that ignores the human system around it will fail the same way the original project failed.

Focus on framing what has changed, not just the decision.

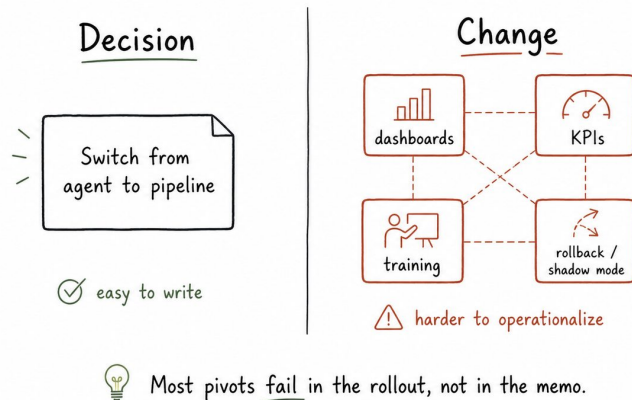


Figure: The architecture decision is only half the work; the rollout system determines whether the pivot sticks.

The decision is what you wrote in the memo. "We're switching from the autonomous agent to a staged pipeline." "We're stopping the chatbot and shipping retrieval-only search." Easy to draft, easy to circulate. The decision is engineering work, and engineering work is the part teams find easiest.

The change is what happens in the rest of the building. Whose dashboards break? Whose KPIs are tied to the old approach? Which team has to update their training material? Who tells users they need to do this slightly differently now? Who owns the rollback path while the new system runs in shadow mode? The change is operational work, and it is at least as much work as the decision was, often more.

Most pivots fail in the change, not the decision. The architecture lands clean. The rollout is messy, slow, partial, or skipped entirely, and six months later the old system is still running alongside the new one because nobody owned the work of turning it off. Write the change plan in the same memo as the decision. Name the owners. Set the dates. Otherwise you've written a recommendation, not a pivot.

The pivot memo

Everything in this section compresses into one memo. Write it before the meeting, not during. Here it is for the policy bot, and notice that every line was already earned somewhere in the last two chapters.

The evidence. What we're seeing, with numbers. "Answer accuracy is at 58% across the last three weeks of production traffic, a hundred graded answers a week. A trace audit puts the grounded rate, answers the citation supports, nearer 46%."

The pre-commitment. What we agreed this would mean. "At kickoff we agreed that below 70% we'd revisit the approach."

The diagnosis. Why it's happening, from the Chapter 3 tools. "Fifty random rows show 70% of failures are one shape: multi-hop questions where the second document needs a query the first one supplies. A reranker recovered the slice that shared vocabulary; nothing inside one-shot retrieval reaches the rest."

The options. Persist, pivot, or stop, with the cost of each. "Persist means another tuning cycle, two weeks, and no hypothesis for why it ends differently than the last two. Pivot means iterative

retrieval, four weeks. Stop means the FAQ template, which already beats us on grounded answers, and the roadmap gets the team back."

The recommendation. One of the three, and if it's a pivot, which reframing move. "Pivot. Decompose the question into sub-queries. Two-week time-boxed prototype, judged on forty fresh multi-hop questions, before we commit. If it doesn't clear 70% on those, we stop and ship the template."

The change plan. Who owns the rollout, whose dashboards and KPIs are affected, who retires the old system, and by when. Say what "retire" means: an endpoint turned off, or a path that lives on inside the new design.

The first five sections are the evidence structure from this chapter. The last one is the part most memos skip, and it's the part that decides whether the pivot happens.

The chapter summarized

Chapter 3 ended with the reframe column lit twice. This chapter is what you do about it, and the hard part was never the architecture.

Stopping is a skill, and organizations don't reward it. You get promoted for shipping, sunk cost feels like personal failure, and the vocabulary makes it worse. A project that returned a clear answer worked as an experiment. Say "the assumption didn't survive testing" and "we bought three months of evidence," because both are more accurate than "we failed."

Name the failure mode first. Plateau, region failure, confident wrong answers, runaway, operational red flag, reality gap, stalled pilot. Each one points at a different kind of break, and naming it turns "it's not working" into something the room can act on.

Choose among three options on evidence strength. Weak or inconsistent evidence means persist, with a time box and a hypothesis for why the next cycle will end differently than the last one. Strong evidence of an architectural mismatch means pivot. Strong evidence with trust broken or unbounded risk means stop, and no architecture fixes that row.

A pivot is a specific move, not a restart. Retarget, decompose, ensemble, constrain, simplify. The headline pattern across agent projects is full autonomy to a constrained pipeline. The shipping matrix then decides how carefully to roll the new frame out.

Bring evidence and the pre-commitment, then plan the change. The conversation goes well when you can say "at kickoff we agreed that below seventy percent we'd revisit," and badly when you can only say "I think we should stop." The memo carries the evidence, the pre-commitment, the diagnosis, the options with their costs, the recommendation, and the change plan. Most pivots fail in the change, because nobody owned turning the old system off. Name the owners and set the dates.

Part 2: Reference module

Deeper diagnostic material behind Chapter 3, plus the modality-specific failure modes behind Chapter 4. Use it when a chapter points you here, or when you want the ML and GenAI depth behind the agent-era examples.

Diagnostics in Depth

Deep diagnostic techniques behind Chapter 3. The chapter organizes everything around three questions: is there signal, is there cheating, is the metric real. Those questions are technology-agnostic and they stay the spine. This reference splits the deeper techniques into a general core that applies to any AI system, then modality-specific sections for the probes that only make sense for ML, GenAI, or agents.

Run the general core no matter what you built. Then jump to your modality's section for the domain-specific probes. The sorting test is simple: if a technique only needs an input, an output, and a score, it's general; if it needs weights, features, or training curves it's ML; retrieval or prompts, GenAI; tool-loops or trajectories, agent.

General diagnostics (any AI system)

These travel across ML, GenAI, and agents, organized by the three questions.

Is there signal?

Sanity checks: is there signal?

These tests verify there's something learnable in your data. Run them before you start error analysis.

Two of the classic sanity checks, the overfit test and the random-labels test, need you to train the model yourself, so they sit in the ML section below. If you're calling a model through an API, you can't run them, and you don't need to. Two black-box checks do the same job of asking whether signal exists before you tune anything.

The expert-with-the-same-inputs check

Hand a domain expert twenty to fifty cases with exactly the inputs the system sees, nothing more, and ask them to make the call. If the expert struggles with those inputs, investigate missing information before spending another cycle on tuning, and ask the expert what evidence would let them answer reliably; that list is your next set of inputs. The expert's failure supports the missing-information reading. It doesn't prove the inputs can't be used better, so treat it as a strong lead rather than a ceiling. Write down the expert's score while you're there. It's a baseline you'll want, and it travels across ML, GenAI, and agents without modification.

The input-removal check

Run the system with a key input removed or scrambled: the retrieved documents replaced with random ones, the image blanked, the customer record swapped for another customer's. If the score barely moves, the system wasn't using that input, and whatever signal it's showing comes from somewhere else, usually priors or a shortcut. This is Chapter 3's ablation applied as a sanity check, and it works on any system you can call.

The baseline sanity check

Does a trivial solution beat your model? Try predicting the mean or mode for everyone, using a single (obvious) feature, or applying simple rules ("if $X > \text{threshold}$, then Y "). If the baseline wins, your model complexity isn't buying anything, the signal may not require ML at all, or you might be overfitting and the simple approach generalizes better.

If predicting "not fraud" for everyone beats your fraud model, your model is worse than useless.

Case study: SimpleStream (2025). A simple baseline of streaming the most recent four frames to a VLM achieved close to state-of-the-art performance on video understanding benchmarks, outperforming complex retrieval and memory mechanisms. The sophisticated architectures weren't earning their complexity.

This sanity check complements Chapter 3's Baseline Test. The sanity check asks "does a trivial solution beat your model?" The Baseline Test asks the broader question: "does the agent earn its complexity over a simpler system?"

The taxonomies below are what you use when running Chapter 3's 50 Random Rows method. Pick categories from here to label what you find.

The "why is this hard?" framework

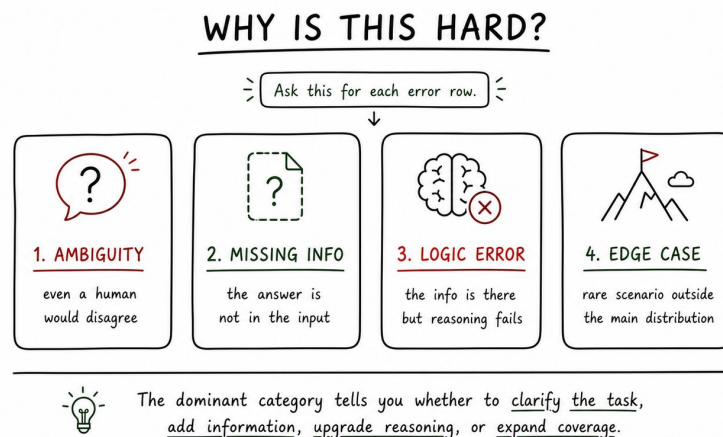


Figure: The dominant category tells you whether to clarify the task, add information, upgrade reasoning, or expand coverage.

You cannot debug a distribution. You can only debug a row.

For any error, ask "why is this hard?" and categorize:

Category	Definition	What It Tells You
Ambiguity	Even a human couldn't label this correctly	Your labeling guidelines are broken. Reframe the task definition.
Missing Info	The answer isn't in the input/prompt	Your inputs are incomplete. Add retrieval or new data sources.
Logic Error	The info is there, but the model reasons poorly	Your model is too weak. Increase capacity or add chain-of-thought.
Edge Case	Legitimate rare scenario the model wasn't trained for	Your training distribution doesn't match reality.

The technique. Sample fifty random error rows (not successes, errors tell you what's broken). For each error, ask "why is this hard?" and assign a category. Tally the distribution. The dominant category tells you where to focus.

If dominant category is...	The fix is...
80% Ambiguity	Reframe labeling guidelines, clarify the task
80% Missing Info	Add retrieval (RAG), not prompt engineering
80% Logic Error	Bigger model, chain-of-thought, or tools
80% Edge Cases	Expand training data or accept the limitation

Teams waste months optimizing prompts when the real problem is retrieval, or fine-tuning when the real problem is ambiguous labels. The "why is this hard?" categorization is what prevents that misdiagnosis.

This framework is the technical companion to Chapter 3's 50 Random Rows method. The chapter gives you the protocol. This reference gives you the error taxonomy.

The error categorization method

When diagnosing GenAI systems, sample fifty errors (not successes, errors), categorize each one (ambiguity, missing info, logic error, wrong retrieval), then count the distribution. If eighty percent are "missing info," no amount of prompt engineering will help. You need RAG.

Is there cheating?

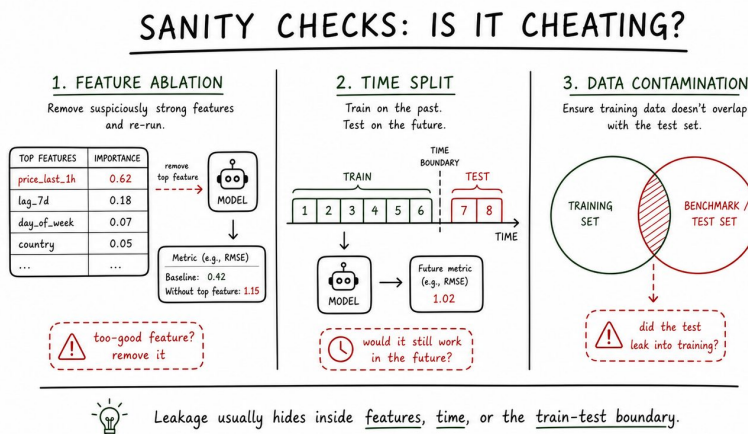


Figure: Leakage usually hides inside features, time, or the train-test boundary.

These verify you're not fooling yourself with leaked information.

The data contamination test

Critical for LLM evaluation and benchmark testing. Did your test data leak into your training data? For LLMs, check if benchmark questions appear verbatim in the training corpus. For traditional ML, verify that train/test splits don't share entities (same user, same session). N-gram overlap detection between test and training sets is a good cheap probe.

If contaminated, benchmark scores are meaningless because the model memorized answers.

Production performance will be much worse than test metrics suggest. You're measuring recall, not generalization.

LLMs trained on internet data may have "seen" your test set. Many published benchmark scores are inflated by contamination.

GenAI-Specific Contamination Checks:

Five checks catch different shapes of contamination.

Exact match search is the simplest: search the training corpus for the exact text of your test questions. If a question appears verbatim, the model has memorized the answer. This catches the most blatant leakage but misses anything paraphrased.

N-gram overlap goes one step further. Compare overlapping word sequences (typically eight-grams or thirteen-grams) between your test set and your training corpus. High overlap on a question and its paraphrase suggests the model has seen something close enough to memorize. This catches paraphrased leakage that exact match misses.

Canary strings are the most defensive option. You insert synthetic, easily-identifiable strings into your test set before publishing it (a unique sequence like "rare-canary-string-7f3b2"). When you test a new

model, you check whether it can produce the canary unprompted. If it can, your test set has leaked into someone's training data.

Temporal cutoff is the cheapest probe for contamination on benchmarks, and the easiest to over-read. Compare when the test items were created, not just published, to the model's training cutoff. Items that existed anywhere online before the cutoff, in a draft, a repo, a forum thread, can be in the corpus even if the benchmark paper came later. A cutoff after the items were created makes contamination plausible and the score should be discounted. A cutoff before they were created makes pretraining contamination unlikely, which is not the same as impossible: the model may have been fine-tuned later, the harness may retrieve from a store that includes the answers, and the items may have been exposed through earlier evaluation runs. Treat a clean cutoff as one piece of evidence, not a proof.

MCQ format shortcuts is a different failure mode. Multiple-choice benchmarks can often be answered without seeing the question, because models exploit statistical patterns in the answer choices (length, position, lexical cues). The check is to test the same model on the choices alone, no question. If accuracy is well above chance, the model is reading the answer key, not the question. The fix is to switch to generative answer matching: model produces a free-form response, scored against a reference.

Check	What It Catches
Exact match search	Test questions in training corpus
N-gram overlap	Paraphrased versions of test data
Canary strings	Synthetic data designed to detect leakage
Temporal cutoff	Pretraining exposure, when item creation postdates the cutoff; misses later fine-tuning, retrieval, and harness exposure
MCQ format shortcuts	Models exploiting answer patterns, not understanding questions

(Source for the MCQ shortcut finding: Jain et al., "Answer Matching," *arXiv:2507.02856*)

The comparison trap: Lord's paradox

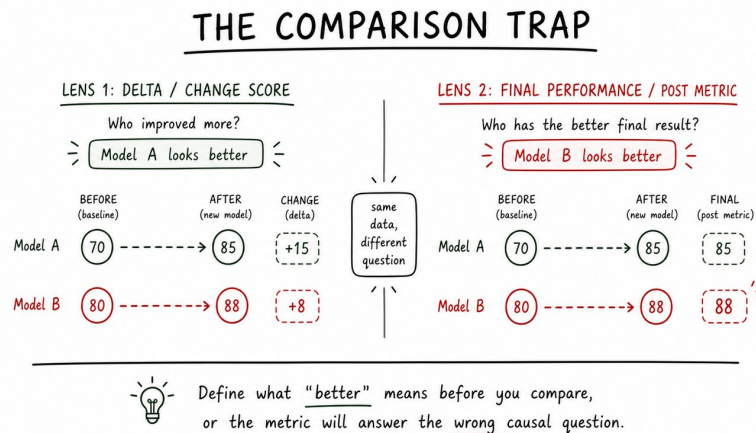


Figure: Define what "better" means before you compare, or the metric will answer the wrong causal question.

When comparing models or segments, "compare to a similar baseline" sounds obvious. There are two different ways to do it, and they can give opposite answers from the same data.

Comparison Method	What It Asks	When It's Right
Delta / Change Score	"Which improved more from where it started?"	Evaluating training procedures, learning speed, marginal gains
Baseline-Controlled	"Given the same starting point, which ends up better?"	Choosing what to deploy, comparing final quality

Both feel like "fair" comparisons. Both are mathematically valid. They answer different causal questions.

ML Example:

You're comparing two models:

- **Model A:** Started at 70% accuracy, ended at 85% (+15 points)
- **Model B:** Started at 80% accuracy, ended at 88% (+8 points)

Question	Answer	Why
"Which model should we deploy tomorrow?"	Model B (88%)	You care about final performance
"Which training recipe should we invest in?"	Model A (+15 pts)	You care about the approach's potential
"Which is better?"	Depends on the decision	Same data, different conclusions

Why this happens: Model B started stronger (pretrained on more data). Model A improved more (had more headroom). Neither is "wrong." They're answering different questions.

The Hidden Failure Mode:

Most ML teams do this unconsciously:

Context	What They Report	The Problem
Offline eval	Delta metrics ("improved by X%")	Favors high-headroom models
Online eval	Post metrics ("achieved Y%")	Favors strong-baseline models
Stakeholder slides	Mix of both	Creates confusion
Decisions	Assume they agree	They often don't

Arguments start because people are answering different causal questions without realizing it.

The Diagnostic Discipline:

Before you run the comparison, finish this sentence:

"We will consider the model better if..."

If you say...	Use this method
"...it improves users more from where they are"	Delta / change scores
"...it produces higher outcomes given the same starting point"	Control for baseline
"...it has the highest final performance"	Post-only comparison

Applied to Segment Analysis:

The same trap appears when diagnosing region failure:

"The model works for Segment A (85%) but fails for Segment B (70%)."

Before concluding the model is biased, ask:

- What was the baseline for each segment?
- Is Segment B inherently harder (noisier data, rarer events)?
- Would *any* model show this gap?

A 70% accuracy on a segment with 60% baseline might be a bigger win than 85% on a segment with 80% baseline.

The Rule: Define what "better" means before you compare. The metric without the question is just a number.

Is the metric real?

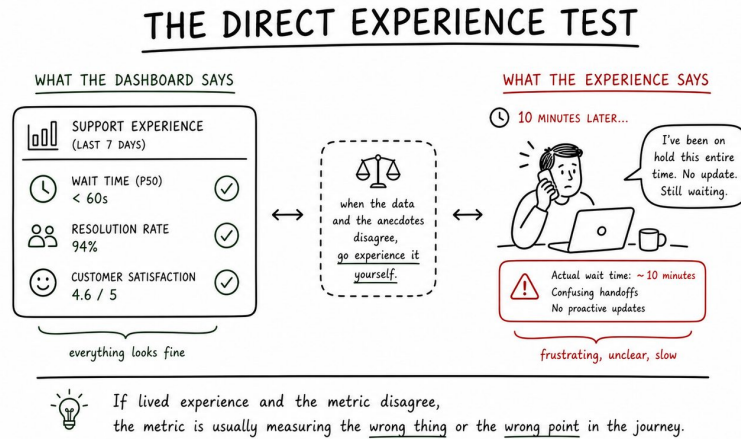


Figure: If lived experience and the metric disagree, the metric is usually measuring the wrong thing or the wrong point in the journey.

The direct experience test (the Bezos call)

Does your metric match reality when you experience it yourself? Don't look at the dashboard. Experience what your user experiences. Call your own support line. Use your own product. Time it yourself.

Amazon's metrics said customer wait times were under sixty seconds. Customer complaints said otherwise. In a meeting, Bezos picked up the phone, dialed the 1-800 number, and waited. More than ten minutes of silence later, someone answered.

If your experience doesn't match your metric, you're measuring the wrong thing (metric definition is flawed), you're measuring at the wrong point (metric captures only part of the journey), or you're aggregating away the problem (average hides outliers).

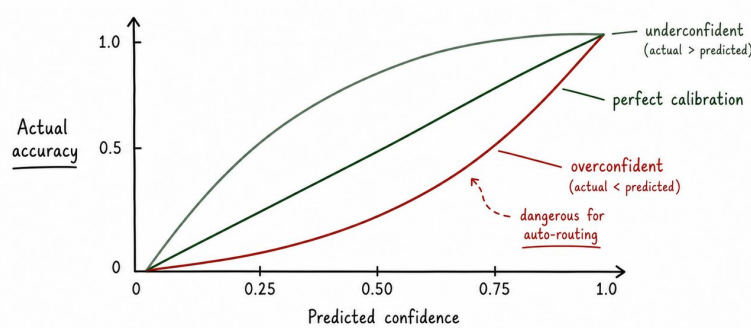
Domain	Direct experience test
Customer support bot	Submit a real question, time the full resolution
Search/RAG system	Query it yourself with real questions, not test cases
Recommendation system	Use it as a real user for a week
Fraud detection	Review flagged cases manually. Do they look like fraud?
Content moderation	Sample "approved" content. Would you approve it?

"When the data and the anecdotes disagree, the anecdotes are usually right. It's usually not that the data is being miscollected. It's usually that you're not measuring the right thing." Jeff Bezos, [Lex Fridman Podcast #405](#) (2023)

If your dashboard says everything is fine but users are complaining, trust the complaints. Go experience it yourself before defending the data.

Deep technique: calibration

CALIBRATION: CAN THE MODEL TRUST ITS OWN CONFIDENCE?



📌 A model that sounds sure without being right is riskier than a model that knows when it is uncertain.

Figure: Confidence only helps if it tracks reality closely enough to support routing, thresholding, and human handoff decisions.

A calibrated model's confidence matches its accuracy. When it says "ninety percent confident," it should be right ninety percent of the time. Calibration matters because downstream decisions depend on confidence (threshold-based routing, human review triggers), overconfident wrong answers are worse than uncertain answers, and calibration reveals whether your model knows what it doesn't know.

Reading Calibration:

Three patterns show up most often when you plot predicted confidence against actual accuracy.

Points above the diagonal mean the model is underconfident. It says "sixty percent confident" but it's right eighty percent of the time. Conservative, in the sense that users who only trust high-confidence outputs will get correct answers most of the time, plus they'll discard some answers that would have been right anyway. Often acceptable. The cost is throughput, not correctness.

Points below the diagonal mean the model is overconfident, which is the dangerous pattern. It says "eighty percent confident" but is only right forty percent of the time. Downstream systems that route or auto-approve based on confidence will trust answers they shouldn't. The fix is post-hoc calibration: temperature scaling for neural nets, Platt scaling for classifiers. If those don't close the gap, the training data or the loss is mis-specified and you need to look upstream.

S-curve shape is when the model is calibrated in the middle of the confidence range but miscalibrated at the extremes (very high and very low predictions are both off). The fix is isotonic regression, which adjusts the calibration curve more flexibly than temperature scaling, or a stronger temperature adjustment.

Pattern	Interpretation	Fix
Points above diagonal	Underconfident (conservative)	May be OK. Users can trust high-confidence outputs.
Points below diagonal	Overconfident (dangerous)	Temperature scaling, Platt scaling, or re-examine training
S-curve shape	Extreme predictions miscalibrated	Apply isotonic regression or temperature adjustment

Calibration Metrics:

- **Expected Calibration Error (ECE):** Average gap between confidence and accuracy across bins
- **Brier Score:** Combines calibration and accuracy into single metric

GenAI Calibration:

For LLMs, calibration is especially tricky:

- Token probabilities don't directly translate to answer correctness
- "I'm not sure" in text doesn't map to probability
- Consider asking the model to self-assess and checking correlation with actual accuracy

OOD Confidence Check: Deliberately feed your model out-of-distribution inputs and observe confidence scores. If confidence stays high on inputs the model has never seen, your architecture has no mechanism for "I don't know." That is a structural assumption, not a calibration problem you can tune away.

The Diagnostic Question: "When my model says it's confident, is it actually right?"

Note: Calibration is the technical companion to Chapter 3's $\text{pass}@k$ vs pass^k distinction. $\text{pass}@k/\text{pass}^k$ tells you whether to measure capability or reliability. Calibration tells you whether the model's own confidence scores are trustworthy for making that distinction.

ML-specific diagnostics

Techniques that need weights, features, or training curves.

Training-time sanity checks

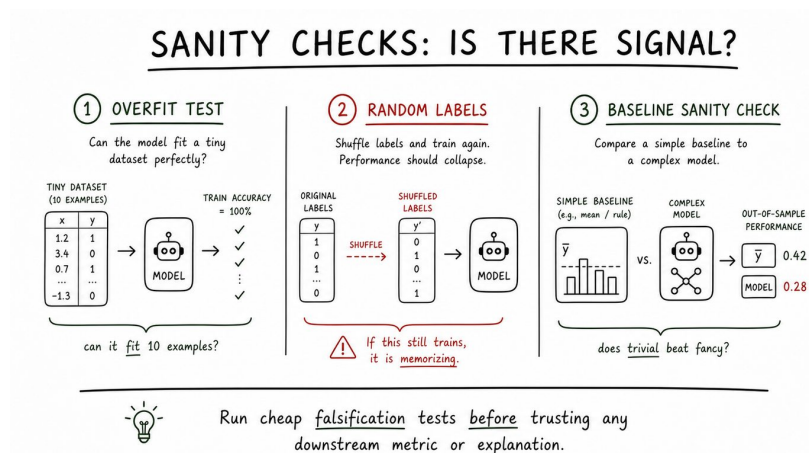


Figure: Three cheap falsification tests for a model you train yourself. The first two need the training loop; the baseline check works anywhere.

Two checks that need you to own the training loop. Run them before error analysis on any model you train.

The overfit test

Can your model perfectly memorize a tiny subset (ten to fifty examples)? Train on ten examples for many epochs. Does loss go to roughly zero? If no, the data pipeline is broken (labels not connected to features), the labels are pure noise (no learnable signal), or the model architecture can't represent the function.

If a model can't memorize ten examples, it definitely can't learn from ten thousand. This is the fastest way to catch data pipeline bugs.

The random labels test

Shuffle the target column, train the model, and see if training loss still goes to zero. If it does, you've learned one thing: the model has enough capacity to memorize your training set. That's a fact about capacity, not a verdict on the real model. Zhang and colleagues showed in 2017 that standard deep networks fit random labels on ImageNet and still generalize fine on the real labels, so fitting noise doesn't by itself mean your real-data model is overfitting. What the test tells you is that train loss is not evidence of anything, and that you have to judge the real model on held-out data, with the gap between train and held-out as the number to watch. If the real model's held-out score is near what it gets on random labels, then you have a problem: it's memorizing, and it's not finding signal.

Data distribution analysis

THE SHAPE OF THE DATA CHANGES THE SHAPE OF THE SOLUTION

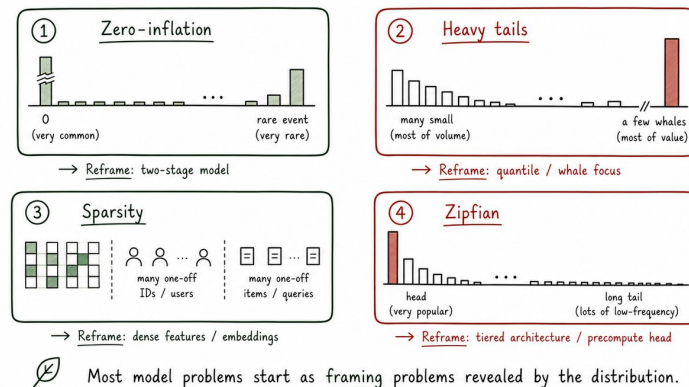


Figure: Distribution shape is often the first clue that the original framing is wrong.

The shape of your data dictates the shape of your solution. If you fight the distribution, you lose.

Before you blame the model, examine what you're feeding it. Most "model problems" are input problems. The taxonomy below organizes the common distribution patterns by what's problematic. The Lesson 3 hands-on resource works through four of them on real datasets: zero-inflated insurance claims, heavy-tailed customer spend, sparse product catalogs, and Zipfian search queries.

Distribution patterns summary

Category 1: target distribution problems

The shape of what you're predicting. These arise when your target variable has a distribution that standard loss functions can't handle well.

Pattern	Evidence	What It Reveals	Fix
Zero-Inflation	90% of values are zero	Standard regression (MSE) fails. Model predicts tiny numbers for everyone.	Two-stage model (classify then regress), Tweedie loss
Heavy Tails	Top 1% = 68% of value	Optimizing mean misses the whales. Model under-predicts high-value cases.	Quantile regression (90th percentile), weighted loss
Class Imbalance	99%+ majority class	Standard classification optimizes for majority. Model predicts "not fraud" for everyone.	Change loss function, class weights, undersample, or reframe as ranking

Category 2: feature distribution problems

The shape of what you're predicting from. These arise when your input features don't represent the underlying structure well.

Pattern	Evidence	What It Reveals	Fix
Sparsity	High cardinality, most IDs appear once	Model memorizes single examples for rare entities.	Dense features, behavior aggregates, embeddings with side info
Missing Data	Systematic missingness correlated with outcome	Missingness itself is signal, not noise.	Add <code>field_missing</code> as feature, segment-aware imputation
Multimodality	Works for X, fails for Y	Data has distinct subpopulations with different patterns. One model can't fit all.	Segment models, decompose, add segment as feature

Category 3: query and request distribution problems

The shape of what users ask for. Especially relevant for GenAI and RAG systems where user requests vary widely.

Pattern	Evidence	What It Reveals	Fix
Zipfian	20% of queries = 80% of traffic	System optimized for average case, not actual distribution.	Tiered architecture, precomputation for head
Ambiguity	Vague queries get wrong answers	System retrieves before it understands.	Ambiguity classifier, clarification flows

Missing data: when absence is evidence

Asking *why* data is missing often reframes the problem entirely. A missing lab test result may mean the clinician didn't think it was necessary. The absence is a clinical judgment, not a data gap. A missing income field may mean high earners opted out. The blank is a signal about the population. Missing activity records in the last thirty days may mean the customer already churned. The missingness is the event you're trying to predict. In fraud detection, a missing field in a loan application is often more predictive than any filled field.

Traditional ML error categories

Class imbalance and multimodality are covered in the Data Distribution Analysis section above, since they are input-shape problems. Two more patterns show up at the metric layer.

Metric plateau is the experience of "we added more data and accuracy didn't move." You added rows, trained again, plotted the line, and it didn't budge. The diagnosis to test first is an information ceiling, after you've ruled out noisy labels and an eval too small to show the gain: the signal you need to predict the target isn't in your features, and adding more rows that look like the ones you have doesn't help.

The fix is structural: engineer better features from what you already have, buy or scrape new data sources, or change the target itself. None of those are "tune harder."

Too-good-to-be-true is the opposite signal: ninety-nine percent R-squared on a problem that no one has ever cracked at that level. Usually target leakage, and occasionally a shortcut in the data or a test set that overlaps the training set, so check the split before you check the features. A feature you're using contains the answer, either directly (as in the `last_login_days_ago` example for churn) or via future information that wouldn't be available at prediction time. The fix is to check your top-ranked features for time-horizon violations, drop or repair the ones that leak, and rerun. If the metric drops to a believable level, the leakage was real. If it stays sky-high, look harder for the leak.

Error Pattern	What It Reveals	Likely Fix
Metric Plateau ("We added more data and accuracy didn't move")	Information ceiling. The signal isn't in your inputs.	New features, new data sources, or different target
Too-Good-To-Be-True (99% R-squared)	Target leakage. You're using future data.	Check top features, fix time horizon

Learning curves: the diagnostic plot

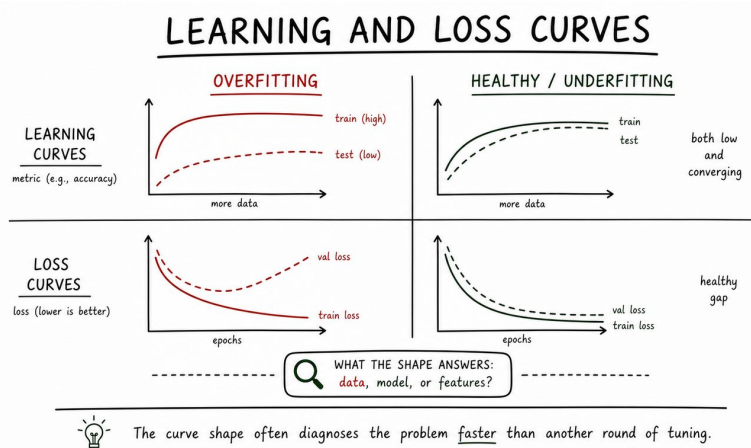


Figure: The curve shape often diagnoses the problem faster than another round of tuning.

Learning curves plot performance vs. training data size. The shape tells you what's wrong, and the figure above shows the two shapes to know: a wide train-test gap that stays open (overfitting) and two low curves that converge (underfitting).

Reading the Curves:

Pattern	Diagnosis	Fix
Large gap, train high, test low	Overfitting. Model memorizes training data.	More data, regularization, simpler model

Pattern	Diagnosis	Fix
Both curves low and converging	Underfitting. Model too simple.	More features, more capacity, less regularization
Both curves high and converging	Good fit.	You're done (or add more data for small gains)
Test curve plateaus early	Data saturation. More data won't help.	Need better features or different approach

The Diagnostic Question: "Is my problem data, model, or features?"

Loss curves: training dynamics

Essential for deep learning and LLM fine-tuning. Loss curves plot training loss and validation loss over epochs/steps. They reveal training dynamics: validation loss turning upward while training loss keeps falling is the overfitting signature, and the point where they cross is where to stop.

Reading Loss Curves:

Pattern	Diagnosis	Action
Val loss rises while train drops	Overfitting	Stop training (early stopping), add regularization
Both losses plateau high	Underfitting	Increase capacity, lower regularization, check data
Loss spikes or oscillates	Learning rate too high	Reduce learning rate, add warmup
Loss doesn't decrease	Learning rate too low or broken pipeline	Increase LR, check gradients, verify data loading
Train loss goes to 0 instantly	Memorization or data leak	Check for duplicates, verify pipeline

Real-World Example: BLOOM 176B Init Parameter. The BLOOM-176B training team experienced numerical instability at the 104B checkpoint. Initial diagnosis focused on learning rates and optimizers. The real problem: Megatron-LM's default `init-method-std = 0.02` was inherited from smaller models without questioning whether it applied at 176B scale. The diagnostic insight: Query x Key matrix multiplications were blowing up in fp16 because init std was "way too big" for this model size. The correct value, derived from first principles using `sqrt(1/(NHIDDEN*3))`, was 0.00482, a 4x reduction. The loss curve was the signal. It revealed the init std problem before anyone understood the root cause.

(Source: Stas Bekman, ML Engineering Open Book)

Random probe baseline

Add a column of pure random noise (for example `np.random.normal`) to your training data. Run feature importance (SHAP, gain, whatever). A feature that ranks below the noise column is a feature to be suspicious of.

Suspicious, not condemned, because a single run of this test lies in two ways. Importance rankings from one training run are noisy, so a real feature can land below the noise column by chance. And correlated features split their credit: two features carrying the same signal can each rank below the noise column while together they carry most of the model's performance. Repeat the test with several noise columns and several seeds, use permutation importance measured on held-out data rather than training-set gain, and drop suspicious features in groups to see what the model loses. If your expensive "social graph features" still rank below noise after that, your hypothesis about the social nature of the problem is wrong.

"One Permutation Is All You Need" (arXiv:2512.13892)

Traditional permutation importance requires multiple random shuffles, adding variance and compute cost. New research shows a single deterministic permutation achieves better bias-variance tradeoffs, especially with small samples or noisy data.

The paper also introduces Systemic Variable Importance: a stress-test that accounts for feature correlations, revealing hidden dependencies standard methods miss. Useful for auditing whether your model secretly relies on protected attributes.

The feature ablation test

Does removing your top-ranked feature improve performance? Identify the top feature by importance, remove it, retrain, compare. If removing the top feature helps, you have target leakage (feature contains the answer), a spurious correlation (feature is a proxy that won't generalize), or overfitting to noise.

A feature that seems too good often is. This catches leakage that inflates training metrics and then destroys production performance.

This is the ML-specific version of Chapter 3's Ablation Analysis. The chapter's version works for any system. This one focuses on feature-level leakage in ML models.

The time split test

Does splitting by time (instead of randomly) kill your performance? Train on months 1-6, test on 7-8, compare to random-split results. If the time split tanks performance, you're using future data (look-ahead bias), the world has changed (concept drift), or your random split was leaking information across time.

Random splits can accidentally include future information in training. Time splits reveal whether your model will work in production where you only have past data.

Deep technique: the Dead Salmon test

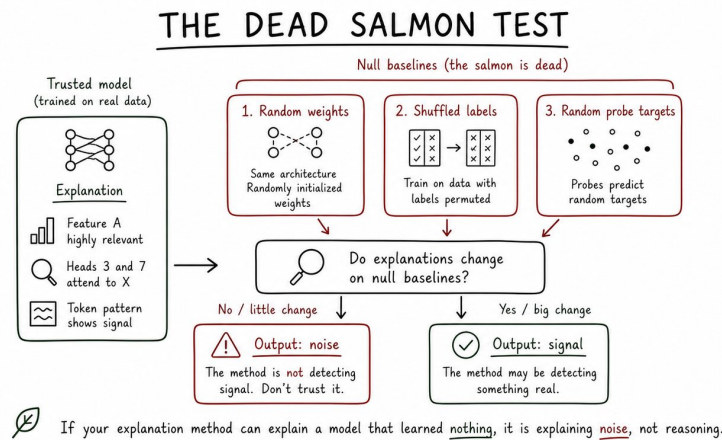


Figure: If the explanation pipeline still looks convincing on a system with no signal, the explanation method is telling stories, not finding structure.

Adapted from Meloux et al., "The Dead Salmons of AI Interpretability."

Before trusting any interpretability method (SHAP, saliency maps, attention visualization, probing classifiers), test whether it produces meaningful outputs or just plausible-looking noise.

In neuroscience, researchers once found "brain activation" in a dead salmon because they didn't correct for multiple comparisons. The same thing happens in AI interpretability. Methods can produce convincing explanations for models that have learned nothing.

Three Null Baseline Tests to Run Routinely:

Test	What You Do	What Failure Means
Same architecture, random weights	Run interpretability on an untrained model	If you get "explanations," the method finds patterns in noise
Same data, shuffled labels	Train model on randomized targets, then interpret	If explanations look similar to the real model, they're artifacts
Frozen model, randomized probe targets	Train a probe on random labels over frozen representations	If the probe "succeeds," it's memorizing, not finding structure

The Diagnostic Question: "Does this interpretability method produce different outputs for a trained model vs. a null baseline?"

- **If YES:** The method is detecting something real.
- **If NO:** The method is finding patterns in noise. Don't trust its outputs.

The Broader Principle: Negative Controls. This borrows from experimental science: always apply your method to a system known to have no signal. If your interpretability pipeline finds

"interesting neurons" in a random network, that's a red flag. You're measuring your method's false discovery rate, not the model's reasoning.

Before trusting an interpretability result, ask:

1. What is the null hypothesis?
2. Is this explanation identifiable?
3. Would this method find "meaning" in a random model?
4. What is the uncertainty of the explanation?
5. Does intervening on the explanation change behavior?

If you cannot answer those, the explanation is storytelling.

GenAI / RAG-specific diagnostics

Techniques for retrieval and generation: where the failure is search versus grounding, and how query distribution shapes the architecture.

Query distribution and caching

User queries follow a Zipf distribution. A small head of queries carries most of the traffic, and a long tail of rare queries carries the rest. If you optimize for average query performance you are optimizing for the tail, because most unique queries are rare, and the engineering effort lands where the traffic isn't. The fix is a tiered architecture: curate the head, optimize the torso, and accept lower quality on the tail. The two patterns below follow from that shape.

The precomputation pattern

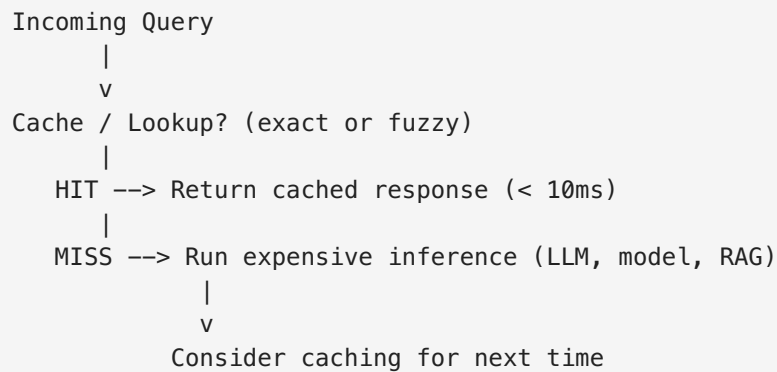
Instacart and DoorDash both run versions of this pattern in production.

If queries follow a Zipf distribution, you can precompute answers for the head and only run expensive inference for the tail.

Query Distribution (Zipf's Law):

Rank	Query	Traffic %	Cumulative	
1	"how do I return?"	8.2%	8.2%	
2	"track my order"	5.1%	13.3%	
3	"cancel subscription"	3.4%	16.7%	
...				
100	[various]	0.1%	72.0%	<-- Top 100 = 72% of traffic
...				
10000	[long tail]	0.001%	95.0%	<-- Tail = 28% of traffic

The Architecture:



Implementation Strategies:

Strategy	How It Works	Best For
Exact Match Cache	Hash query, return cached response	Identical repeated queries
Semantic Cache	Embed query, find similar cached query	Paraphrased queries ("return item" ~ "how to return")
Precomputed Answers	Offline: LLM answers top 1000 queries, store	Known high-traffic queries
Query Expansion	LLM rewrites query variations, all point to same answer	Spelling, phrasing variations

Case Study: Instacart Query Expansion

Instacart's search traffic has the head-and-long-tail shape above. Instead of running LLM inference for every query:

1. **Offline:** An LLM pipeline generates query rewrites and intent for the high-frequency head queries
2. **Cached:** Those results are stored and served from cache

3. **Online:** Cache hit returns the precomputed answer; a cache miss goes to a smaller fine-tuned model in real time

Result: Only about the bottom 2% of queries need real-time inference. Query-rewrite coverage went from about 50% of traffic to over 95%, and latency fell from 700ms to 300ms. ([Instacart, "Building the Intent Engine"](#))

The Math:

Scenario: 1M queries/day, LLM costs \$0.01/query

Without precomputation:

$$1\text{M} \times \$0.01 = \$10,000/\text{day}$$

With precomputation (70% hit rate):

Precompute top 10K queries: \$100 (one-time)

Cache hits: 700K $\times$ \$0 = \$0

Cache misses: 300K $\times$ \$0.01 = \$3,000/day

Savings: 70% cost reduction

When Precomputation Works:

- Query distribution follows power law (few queries = most traffic)
- Answers don't change frequently (or you can refresh cache)
- Latency matters (cache is faster than inference)
- Exact or semantic matching is feasible

When Precomputation Fails:

- Highly personalized responses (answer depends on user context)
- Rapidly changing information (cache goes stale)
- Uniform query distribution (no head to cache)

The Diagnostic Question: "What percentage of my traffic comes from the top 100 queries? If it's > 50%, precomputation could cut costs in half."

Sources: [Instacart, "Building the Intent Engine"](#); [DoorDash, "Offline LLMs, Online Personalization"](#), which precomputes homepage carousels offline so the LLM never sits in the live request path. The cost math above is illustrative.

Query ambiguity

Users ask "How do I fix it?", "What's the policy?", or "Why was I charged?" Vague queries that could mean many things.

What It Reveals: Your system treats clear and ambiguous queries the same way. Retrieval on ambiguous queries returns wrong documents confidently. The model guesses instead of asking for clarification.

The Diagnosis: You're retrieving before you understand.

The Fix:

- Ambiguity classifier to flag vague queries before retrieval
- Clarification flows ("Can you tell me which product you're asking about?")
- Context gathering (pull user history, recent actions)
- Multi-intent retrieval (retrieve for multiple interpretations)

The Diagnostic Question: "Do users frequently rephrase after the first response? Do short queries have higher 'not helpful' rates?"

GenAI error categories

Six error patterns cover most of what teams find when they sample GenAI failures. Each one points at a different layer of the system, which means each one has a different fix.

Confident but factually wrong is the canonical hallucination. The model produces a fluent, authoritative answer that is simply incorrect. The diagnosis is a knowledge failure: the weights contain outdated, noisy, or missing information about the topic. The fix is to move the facts out of the weights. Retrieval (RAG) pulls the source from a verified store at query time. Fine-tuning is the wrong fix. Bigger models hallucinate more confidently with the same probability of being wrong.

Logic or arithmetic failure is when the model knows what to do conceptually but botches the multi-step work. The diagnosis is a reasoning failure with no scratchpad. The model is trying to hold a chain of operations in a single forward pass and dropping pieces. Chain-of-thought prompting gives the model space to write down intermediate steps. Tool use gives it a calculator or interpreter for the parts that need exact computation. Both fix the structural lack of working memory.

Retrieval miss is the "I don't know" answer when the relevant document is in the corpus. The diagnosis is a search failure: the query embedding didn't match the document embedding closely enough. The fix is hybrid search (add keyword matching alongside vectors), better chunking (smaller, denser units), or query expansion (rewrite the query into multiple variations before retrieving).

Retrieval hit, wrong answer is when the right document made it into context but the model still produced a wrong response. The diagnosis is a reasoning failure on top of correct retrieval. The model had what it needed and didn't use it. The fix is prompting that forces the model to ground in the retrieved text: citation requirements, "answer only from the provided context," extractive summarization rather than generative.

Prompt keeps growing is the symptom you notice in your own codebase, not the user's output. You keep adding "do not do X" rules to fix specific bad behaviors and the prompt has hit five pages. Read

the rules before you diagnose. If most of them are about style, format, and tone, you're patching a behavior problem with context and the model keeps fighting back; fine-tuning bakes that behavior into the weights and lets the prompt focus on the task. If the rules are patching wrong facts, missed documents, or steps the workflow should have handled, the prompt is covering for a retrieval failure or a bad decomposition, and a fine-tune would bake the patch in rather than the fix. Which kind of rule dominates decides the move.

Agent loops or spirals is when the agent calls a tool, the tool fails, the agent tries something close, and the loop repeats. The diagnosis is that the task is too broad for a single inference pass and the agent has no way to recognize it's stuck. The fix is structure: a staged pipeline with hard-coded steps, or constrained tool whitelists, or smaller decomposed tasks that fit into one inference each.

Error Pattern	What It Reveals	Likely Fix
Confident but Factually Wrong	Knowledge failure. Weights contain outdated or noisy info.	Add Retrieval (RAG), not fine-tuning
Logic/Arithmetic Failure	Reasoning failure. No scratchpad for multi-step work.	Chain-of-Thought or Tool Use (calculator)
Retrieval Miss ("I don't know" when doc exists)	Search failure. Embedding didn't match.	Hybrid search (add keywords), better chunking
Retrieval Hit, Wrong Answer	Reasoning failure. Context was right, logic was wrong.	Better prompting, citation requirements
Prompt keeps growing (50+ rules)	Depends on the rules: style and format rules mean a behavior problem patched with context; rules that patch wrong facts or missed steps mean a retrieval or workflow failure	Style rules: fine-tune. Fact and step patches: fix retrieval or the decomposition first
Agent loops or spirals	Task too broad for single inference	Staged pipeline with hard-coded steps

RAG decomposition

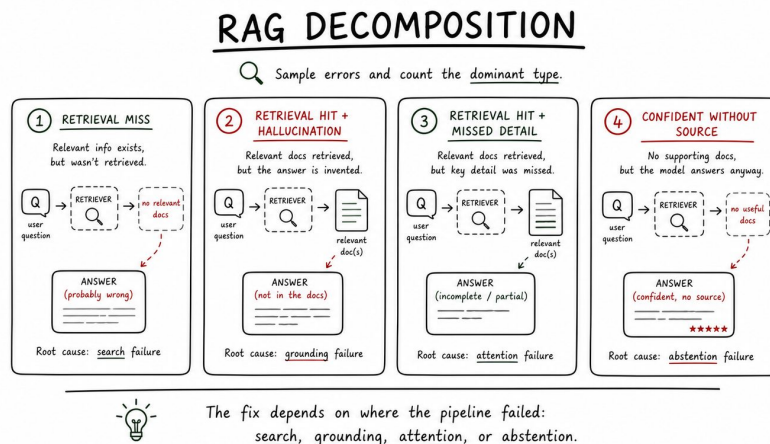


Figure: The fix depends on where the pipeline failed: search, grounding, attention, or abstention.

When diagnosing RAG systems, decompose errors by their source. The fix depends entirely on where in the pipeline the failure occurs.

RAG error categories:

Category	Definition	Likely Fix
Retrieval Miss	Relevant document exists but wasn't retrieved	Hybrid search (add keywords), better chunking, query expansion
Retrieval Hit + Hallucination	Context was correct, but model added/changed information	Citation requirements, lower temperature, faithfulness checks
Retrieval Hit + Missed Detail	Context was correct, but model missed key information	Better prompting, chunking strategy, highlight relevant sections
Confident Without Source	Model answered from parametric memory, not retrieval	Force retrieval-only mode, add "I don't know" training

The technique. Sample fifty errors (not successes). Categorize each one: retrieval miss, hallucination, missed detail, or no source. Count the distribution. If eighty percent are retrieval misses, no amount of prompt engineering will help. Fix retrieval.

If dominant category is...	The fix is...
80% Retrieval Miss	Better embeddings, hybrid search, chunking
80% Hallucination	Faithfulness checks, citation requirements
80% Missed Detail	Better prompting, chunk highlighting
80% No Source	Force retrieval mode, retrain for abstention

Teams waste months optimizing prompts when the real problem is retrieval, or tuning retrieval when the real problem is generation faithfulness. Decomposition is what prevents that misdiagnosis.

This is a specialized application of Chapter 3's Failure Funnel. The Failure Funnel works for any pipeline. RAG decomposition gives you the specific error categories for retrieval-augmented systems.

Agent-specific diagnostics

Techniques for systems that choose their own actions: reading the trajectory before it compounds.

Reasoning and action traces

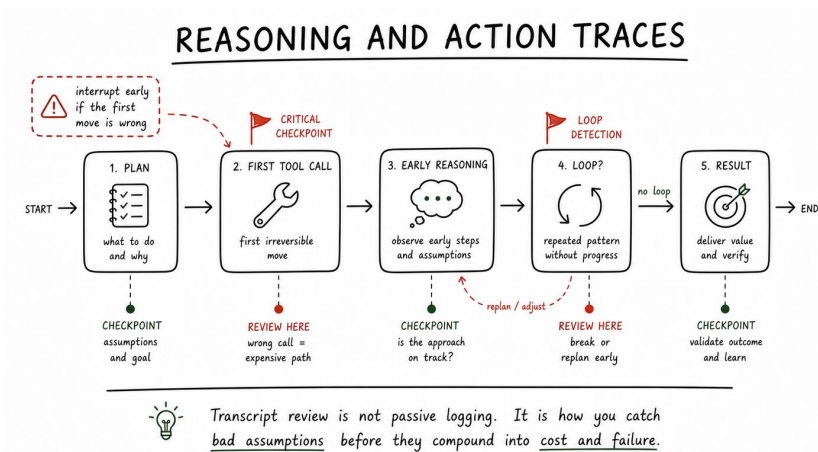


Figure: Transcript review is how you catch bad assumptions before they compound into cost and failure.

Adapted from Balic, "The Agentic AI Handbook"

Agent evaluation tells you how to measure success. Execution traces tell you **why failures happen** and where to intervene before they compound.

The First Tool Call Rule:

"Have your finger on the trigger to escape and interrupt any bad behavior."

When running agents, monitor the first tool call closely. If it's wrong, interrupt immediately rather than waiting for completion. Wrong first steps rarely self-correct. They compound into wasted tokens, loops, or wrong outputs.

What to Monitor:

Signal	What to Watch	When to Intervene
Plan quality	Is the agent's plan reasonable before execution?	Before any tool calls

Signal	What to Watch	When to Intervene
First tool choice	Did it pick the right tool for the first action?	Immediately if wrong
Early reasoning	Is the run heading somewhere sensible?	Before expensive operations
Loop detection	Is it retrying the same action?	After 2-3 identical attempts

The Diagnostic Insight: Transcript review is active diagnosis with a chance to intervene. The goal is to catch wrong assumptions early, not just explain the failure afterward.

The Reflection Pattern:

The difference between mediocre and excellent agent output is often just 2-3 iterations. If your agent produces poor results:

1. **Don't just re-run.** Add a reflection step.
2. **Generate, Evaluate, Refine.** Build this loop into the agent.
3. **Use the transcript.** Where did the agent's mental model diverge from yours?

Practical Guidance from Anthropic:

1. **Prototype with Claude Code first.** Before writing SDK code, try to get the task working locally with an AI coding assistant. If it can do it by writing scripts and using bash, it's a good candidate for automation. If it struggles, you've learned something about the task's complexity before investing in architecture.
2. **Think like a human in a box.** If you were locked in a room and given this task, what tools would you want? A computer, a calculator, a way to search files? Give those same primitives to your agent. Don't over-engineer. Start with the basics a human would need.
3. **Iterate on the transcript.** The best way to improve an agent is to read its execution transcripts. Look at where it gets stuck or confused. Then provide better primitives, clearer instructions, or hints in your system prompt.

Note: This technique complements Chapter 3's Single-Step Test. The Single-Step Test tells you whether the problem is in a step or in the handoff. Traces tell you exactly where the reasoning went wrong within a step.

The agent failure taxonomy: where runtime breaks

The LIFE-Harness paper makes the shape of agent failure explicit by categorizing 393 failed agent episodes across seven environments and finding the same four buckets show up everywhere. (Xu, Wen, and Li, "Adapting the Interface, Not the Model," arXiv 2605.22166; summary by Cobus Greyling)

Action realisation is where the agent knew what to do and got the output format wrong. The JSON is broken. A required field is missing. The tool name is misspelled or invented. The reasoning was there. The format was wrong.

Environment contract is where the agent picked the wrong tool, or called the right tool the wrong way. It used search when the rules said lookup. It passed a value the environment rejects. The decision was wrong, not the format.

Trajectory degeneration is where individual steps are fine and the episode is not. The agent repeats the same step. Loops between two states. Burns its step budget without finishing. These failures only appear at the trajectory level, not the action level.

General reasoning is the residual bucket. The format is correct, the tool is right, the trajectory makes sense, and the answer is still wrong. The model thought wrong. Across the paper's sample, this accounts for only **9.9% of failures**.

Category	Symptom	Where the fix usually lives
Action realisation	Broken JSON, missing args, hallucinated tool name	The wrapper: schema validation before execution
Environment contract	Right intent, wrong tool or wrong call	The wrapper: tool selection guardrails
Trajectory degeneration	Loops, repeats, runs out of steps	Trajectory monitoring, step budgets, early stop
General reasoning	Format and tools correct, answer wrong	The model itself

The failure mix is not the same everywhere. In a Telecom benchmark, ninety percent of failures were contract and action-realisation problems. In ALFWorld, a text-based household simulator, trajectory degeneration dominated. In WebShop, an e-commerce simulator, the four buckets split roughly evenly. No single dominant failure pattern, so no single fix.

The practical consequence is that only about one in ten failures is the agent thinking wrong. The other nine are at the seams: between the model and the tool, between the model and the environment, between this step and the next. Most of these failures get fixed by changing the wrapper, not the weights.

Failure modes by modality

Chapter 4 names seven failure modes that show up in every stack. This section lists the modality-specific shapes underneath them, each with the fix that usually matches. Use it after you've named the failure mode and want the move for your stack. An aggregate metric hides the truth, so look at where the errors concentrate.

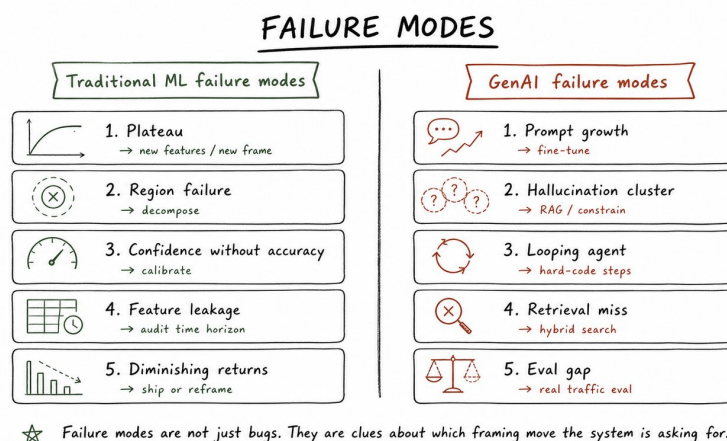


Figure: A failure mode is usually a clue about which architectural move to make next.

Traditional ML failure modes

Plateau (the asymptote). You add more data, tune hyperparameters, make the model deeper, and accuracy stays flat at 82%. The reading to test first is an information ceiling: the signal you need does not exist in your current features. It is not the only reading. Noisy labels, an eval too small to show a two-point gain, and optimization that never ran wide enough produce the same flat line, so confirm the ceiling before acting on it. Chapter 4's check is the one to run: hand an expert the exact inputs the model sees and measure them. If the expert also struggles with those inputs, investigate missing information before spending another cycle on tuning, and identify what evidence would let the expert answer reliably. That doesn't prove the inputs are exhausted, but it's the lead to follow, and to move past the ceiling you need to add information. That can mean engineering better features from the data you already have (interaction terms, graph features, anything that pulls more signal from existing rows), acquiring new data sources (the canonical example is weather added to a sales model), or reframing the target itself (if you can't predict the price, predict the volatility).

Region failure (multimodality). The model is 90% accurate on urban users and 60% on rural. Before reading that as two populations one model can't fit, rule out the cheaper causes: a rural sample small enough that 60% is a dozen examples, labels that were guessed for the rural cases, too few rural rows in training, or features that are empty for rural users. Each of those is a tuning fix. If the segment is well represented, cleanly labeled, and still failing, the "single model" assumption is wrong and you decompose: separate models per segment, mixture-of-experts, or add the segment as a feature so the model can split internally.

Confidence without accuracy. Model says 95% confident, wrong 30% of the time. That is calibration failure. You cannot trust the probabilities, which means you cannot threshold off them, route on them, or show them to a user. Temperature scaling or Platt scaling fix this in most cases. If they don't, the training data is the problem, not the head. The calibration section earlier in this module has the diagnostics.

Feature leakage. Removing your top feature *improves* held-out performance. That is a reason to suspect the feature, not a verdict on it. Three things produce the result: the feature encodes the label through information that won't exist at prediction time (leakage), it correlates with the label in your data for a reason that won't travel (a shortcut), or the model overfit to it. Confirm leakage with the time-horizon check: would this feature's value exist at the moment the decision is made? If not, fix the horizon. If it would, you're looking at a shortcut or overfitting, and the fix is different. The fact that your most important feature is hurting the model should be loud, but teams miss this because the leakage looks like signal until the production data lacks it.

Diminishing returns. Each iteration delivers smaller improvements for growing cost. You are approaching the ceiling for this approach, and the choice narrows to two: ship what you have, or change the frame. The trap is to keep iterating because each step still moves the metric a little, even when the cumulative work no longer justifies the lift.

GenAI failure modes

Prompt growth. Your prompt is five pages long because you keep adding "Do not do X" rules. Read the rules before you reach for a fix. If most of them are style and format, the model is fighting its training, and fine-tuning bakes the behavior into the weights so the prompt can focus on the task. If the rules are patching wrong answers, the problem is upstream, in retrieval or in a task the prompt can't specify, and a fine-tune would bake in the patch rather than the fix. Fifty rules is the symptom. Which kind of rule they are decides the move.

Hallucination cluster. The model confidently states facts that don't exist. The weights contain outdated or noisy information about the topic, and probabilistic generation cannot be coerced into deterministic accuracy. Switch to retrieval (RAG) or constrain generation with citation requirements. Same pattern Klarna hit with policy answers. Separate the facts from the reasoning.

Retrieval miss. "I don't know" comes back even though the relevant document is in the corpus. The query vector does not match the document vector. Add keyword matching alongside the embedding (hybrid search), rechunk, or expand the query before retrieving.

Eval gap. Playground performance does not match production. Your test set is not your real users. Evaluate on real traffic, not curated test cases. If you optimized for the happy path, you will fail on the edge cases that define production reality.

Agent failure modes

These show up when the model is choosing its own actions.

Looping agent. An autonomous agent gets stuck trying to fix its own errors. The task needs too many reasoning hops for a single inference pass, or the scope was too broad. Constrain the output. Hard-code the steps. Don't let the agent decide the plan, force it.

The Lethal Trifecta (agent security)

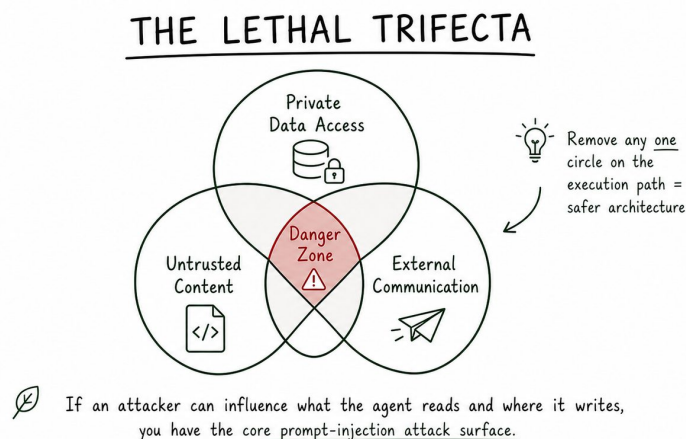


Figure: If an attacker can shape what the agent reads and where it can write, the architecture has the core exfiltration pattern built in.

The term and the argument are Simon Willison's, from his 2025 writing on prompt injection; Balic's "The Agentic AI Handbook" carries the same framing.

Your agent has all three: access to private data, exposure to untrusted content, and the ability to externally communicate. That is the attack surface for prompt injection and data exfiltration. An attacker injects instructions through the untrusted content, the agent reads private data, and the agent ships it out through its external channel.

The fix is to guarantee that at least one of the three circles is missing on every execution path, and the guarantee has to be enforced by the system, not by cleaning the text. Sanitizing inputs doesn't remove untrusted content. There is no filter that reliably tells an instruction hidden in a web page from the page's content, which is why Willison's point is structural: take away one leg of the trifecta rather than trying to make the untrusted leg safe.

Remove this	How	Tradeoff
External comms	Block outbound network from the agent, or allow-list the destinations it can write to	Can't call arbitrary APIs or webhooks
Private data access	Deny direct file and database reads; pass only the data the task needs, explicitly	Needs explicit data passing
Untrusted content	Don't let the agent read it. If it must, run that read in a separate context with no private data and no outbound channel, and pass back only structured results	Limits what the agent can do with the open web

The diagnostic question is short. Can an attacker control what my agent reads AND where it writes? If yes, you have the Lethal Trifecta, and no amount of input cleaning closes it.

Operational red flags

"If the unit economics don't work, the architecture doesn't work."

These failure modes show up when the system works functionally but fails operationally. Teams miss them because attention sits on accuracy while infrastructure burns.

Cost explosion. A \$47K API bill. Costs growing faster than value. No circuit breakers, no cost monitoring, autonomous systems running unchecked. Hard limits, monitoring, kill switches. And consider whether a smaller model gets you most of the way there at a fraction of the cost.

Latency blow-up. P99 jumps from 200ms to 2s and users are waiting. Per-request compute is too heavy for the architecture you have. Pre-compute. Move inference from read time to write time. Cache aggressively. Whether the architecture scales decides more than the model's accuracy on benchmarks.

Scale ceiling. Works at 1K QPS, dies at 10K. Per-request inference is too expensive and the model cannot scale horizontally. Batching, caching, edge deployment. Two-tower architectures or teacher-student distillation are the structural moves when adding hardware stops helping.

Diagnostic techniques summary

Question	Technique	What It Reveals	When to Use
Is There Signal?	Random Probe	Whether your features are better than noise	After feature engineering, before trusting importance rankings
Is There Signal?	RAG Decomposition	Whether the failure is retrieval, reasoning, or hallucination	When diagnosing any retrieval-augmented system
Is There Signal?	Reasoning and Action Traces	Why agent failures happen and where to intervene	When debugging agent behavior and tool selection
Is There Cheating?	Dead Salmon	Whether your interpretability method or agent explanation is trustworthy	Before trusting SHAP, saliency maps, probing results, or self-explanations
Is the Metric Real?	Calibration	Whether model confidence matches actual accuracy	When confidence drives downstream decisions

Sanity checks summary

Question	Check	Run Before	Catches
Is There Signal?	Expert With the Same Inputs	Any build	Signal that isn't in the inputs (any system)

Question	Check	Run Before	Catches
Is There Signal?	Input Removal	Model evaluation	Inputs the system isn't using (any system)
Is There Signal?	Overfit Test	Any training	Broken pipelines, noisy labels (ML, needs training access)
Is There Signal?	Random Labels	Any training	Whether train loss means anything (ML, needs training access)
Is There Signal?	Baseline Sanity Check	Model evaluation	Unnecessary complexity
Is It Cheating?	Feature Ablation	Feature engineering	Leakage, spurious correlation
Is It Cheating?	Time Split	Final evaluation	Look-ahead bias, concept drift
Is It Cheating?	Data Contamination	LLM/benchmark eval	Test data in training, inflated scores
Is Your Metric Real?	Direct Experience Test	Metric definition	Wrong metric, measurement at wrong point

If any sanity check fails, fix the setup before running anything more sophisticated. You cannot diagnose a broken experiment.

Afterword: The arc, end to end

The policy bot ran the whole length of this book. In Chapter 2 a support team framed it: one policy question from one agent as the unit, one-shot retrieval with a cited answer as the choice, and a stop criterion their support director agreed to in writing, accuracy under seventy percent after two tuning cycles means revisiting the approach. In Chapter 3 the offline eval said eighty-four and the agents said the bot was wrong all the time, and an afternoon of diagnosis found the leak: seventy percent of the failures were questions whose answers span two documents, exactly the case the team had judged rare at kickoff. In Chapter 4 the signal they had pre-committed to fired, the plateau test confirmed it, and the call was a pivot to iterative retrieval, prototyped in a two-week time box and judged on fresh multi-hop questions before anyone committed.

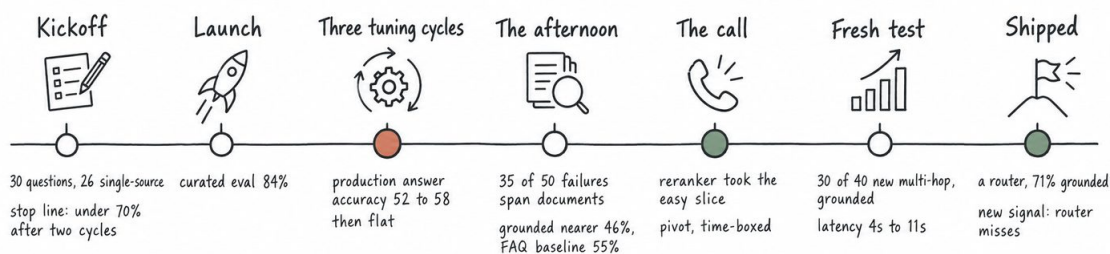
Here is how that time box came out. The numbers are composites like everything else about this project, made specific so the next decision is real. One definition first, because the trace audit changed it. The kickoff thresholds, eighty to succeed and seventy to revisit, were written on answer accuracy: two reviewers grade whether the answer is correct. After the audit found a fifth of correct answers unsupported by their citations, the team kept the thresholds and tightened the measure to grounded accuracy, correct and supported by the cited policy. Every number from here on is grounded accuracy, scored by the same two reviewers.

Measure	Set	One-shot bot	Prototype
Grounded accuracy, multi-hop questions	40 fresh questions from the two weeks after the prototype was frozen	11 of 40 (28%)	30 of 40 (75%)
Grounded accuracy, single-document questions	100 questions from the same weeks	79 of 100	78 of 100
Retrieval coverage, every needed document in the top five (explains the row above, not a quality score)	the same 40 multi-hop questions	12 of 40	35 of 40
Cost per query		1x	about 3x
p95 latency		4 seconds	11 seconds

The prototype cleared the bar on the fresh set. On the single-document questions it was never meant to touch, it came in one case lower, seventy-eight against seventy-nine, inside the two-point tolerance the condition had named before the test ran. The team read the paired cases anyway. The one it lost was a question where the second retrieval pulled in a superseded version of the same policy and the synthesis preferred it, which went into the known-failures list for the iterative path and later became one of the

reasons single-document questions don't get routed there. It also tripled the cost per query and nearly tripled the latency, because every question now paid for a second retrieval and a synthesis pass whether it needed them or not. Most questions didn't. So the team didn't ship the prototype as the new bot. They shipped a router with three paths. When a question matches an FAQ template, the template answers, because it's free and grounded by construction, and the template's fifty-five in Chapter 3 came entirely from those questions. When no template matches and the question looks single-document, one-shot retrieval answers, because on exactly those questions the template has nothing and the one-shot bot's grounded rate was in the high seventies. When the question looks multi-hop, it goes down the iterative path. The standalone one-shot endpoint was retired at cutover; its retrieval path lives on as the middle route. Over the following month the weekly production samples averaged seventy-one, grounded, with cost up by about a third over the old bot instead of threefold. That clears the seventy the team agreed to revisit at and falls short of the eighty the kickoff set, so the next sixty days are about the gap, and the signal to watch is the router's own miss rate, which is a new assumption and a new Step 5. Nothing about that story required a better model. All of it required a frame, a way to check it, and the discipline to act when the check came back.

THE POLICY BOT, END TO END



Every decision names the evidence that earned it.

Figure: Every decision in the thread names the evidence that earned it, and the evidence changes shape as the project goes: a thirty-question test, a curated eval, fifty failures, a fresh holdout, a month of production samples.

Four chapters, one through-line. GOATS is the method. The diagnosis is the evidence. The decision is the action.

When you re-enter the GOATS Loop after a pivot, you start with evidence. Every assumption you tested, every failure you cataloged, every signal you measured carries forward. The new frame is better because the old frame taught you something.

That is also where this book started. Agents keep getting better at building and no better at knowing whether what they built is worth shipping. The loop, the diagnostics, and the decision are how you turn your judgment into evidence other people can act on, and then check that evidence against your judgment again. Run it on the next project before anyone writes code.

Appendix: The four artifacts

This appendix is the one piece of hands-on material in the book. The chapters teach the judgment. These four one-page templates are where it lands, and they are the minimum you need to run the method on your own project without the course. The course goes further: exercises with a quality bar, worked examples across other domains, live review of your project. None of that is here, on purpose.

Each chapter produces one document, and each document is written for someone at your company rather than for you. The Framing Brief goes to your manager. The GOATS One-Pager anchors your kickoff. The Diagnostic Readout goes to your team. The Decision Memo goes to your sponsor. One page each. The budget is the discipline: if it doesn't fit, the thinking isn't done.

Two ways to fill them in. The `frame-use-case` skill produces the first two by interviewing you, and `diagnose-use-case` produces the last two the same way; both are free at github.com/rajshah4/ai-framing-skills, with install steps at aiframer.dev/setup. Or work the templates by hand. The skill asks you the questions. The template leaves you to ask them yourself, which is harder and worth doing at least once.

Each template below is followed by the policy bot's version, the project the chapters followed. Those are composites, with the numbers made specific, and every line in them was earned somewhere in the four chapters.

Artifact 1: The Framing Brief

After Chapter 1. Audience: your manager.

The problem, one sentence, no AI words. What is the underlying problem and who has it? If the sentence needs an AI word to make sense, keep working.

What it costs today. Time, money, errors, or churn. A number if you have one, an honest estimate if you don't.

Top three production failure modes. Specific enough to write a test for. "Cites a retired policy," not "could be inaccurate."

The simplest non-AI solution, and how much it covers. Regex, lookup table, template, a human with a checklist. Estimate the percentage. What's left over is the AI system's actual job.

The system's actual job. One sentence, after de-escalation. This is usually smaller than the original ask, and that's the finding.

Design controls this framing implies. Numbers and rules, not intentions. A confidence threshold, a budget ceiling, a max-iterations cap, an escalation rule. At least one. If no control falls out of your framing, you have a discussion point, not a frame.

The stakeholder and the decision. Who is the one person whose problem this is, and what do they decide differently if it ships?

Quality bar: could your manager read this in three minutes and explain the project to their boss without you in the room?

The policy bot: Framing Brief.

The problem. Support agents can't find the right policy answer fast enough, so they interrupt the policy team or guess.

What it costs today. The policy team spends about a third of its week answering the same questions over chat. Agents' handle time on policy questions runs roughly double the handle time on everything else.

Top three failure modes. It cites a superseded version of a policy. It answers from the base policy when an addendum overrides it. Agents send the answer without reading the citation.

Simplest non-AI solution. Keyword search over the policy library plus the FAQ template the team already uses. It covers the questions the FAQ anticipated, roughly half of what agents ask.

The system's actual job. Answer the policy questions the FAQ doesn't cover, with a citation an agent can check before sending.

Design controls. Every answer carries its citation. The bot is internal only, never customer-facing. It reads no account data. It never decides an exception; those route to the policy team.

The stakeholder and the decision. The support director. The decision is what an agent tells a customer about a policy, and if the bot works, routine policy questions stop landing on the policy team.

Artifact 2: The GOATS One-Pager

After Chapter 2. Audience: your kickoff meeting.

Goal Statement. The sentence from Step 1 with every blank filled: owner, decision, metric, non-goals, the Oracle Test answer, the ROI.

The metric, and the one you rejected. The number that defines success, and why you picked it over the tempting alternative.

Atomic unit. What the system decides about, one at a time. One input, one decision, and the stages inside it.

Top three operating assumptions. Each with a test you could run in a day, and what you'd do if it fails.

Assumption	Test	If false

Alternatives considered. Where each candidate sits on the automation spectrum, which you chose, what you rejected, and the date you'll revisit the rejected ones.

The trade-off, in stakeholder language. What you're giving up and why, in words someone outside the team would understand.

The three-signal dashboard. Success, stop, and leading, each with a threshold, the fallback the stop criterion names, and the stakeholder who agreed to it, with the date.

Quality bar: could two people read this and independently agree on whether the project is succeeding six months from now?

The policy bot: GOATS One-Pager.

Goal Statement. We help support agents decide what to tell a customer about a policy, and the support director owns the number. We'll know it's working when answer accuracy on the weekly production sample is above eighty percent and routine policy questions stop landing on the policy team. We are not building a customer-facing bot, not touching account data, and not deciding exceptions. If the system were perfect tomorrow, it would still fail unless agents trust the answer enough to send it, so every answer carries its citation. It's worth building because the policy team spends a third of its week on these questions and the error cost is bounded by an agent reading the citation first.

The metric, and the one rejected. Answer accuracy on a hundred graded production answers a week, two reviewers against the policy text. Rejected: answers drafted per day, which a bot nobody trusts can max out.

Atomic unit. One policy question from one agent. Two stages inside it, retrieve and generate. The outcome graded: a correct answer whose citation supports it.

Assumption	Test	If false
The answer lives in one document	Pull thirty recent questions, find the sources	Iterative retrieval, the rejected candidate
The corpus is current	Check the last twenty policy updates against the index	A freshness pipeline before launch
Agents read the citation before sending	One week shadowing the pilot team	A hard gate: no send without opening the citation

Alternatives. Keyword search plus the FAQ template (rules, the baseline). One-shot retrieval with a cited draft (constrained output, chosen). Iterative retrieval that searches again after reading (guided agent, rejected for cost and build time, revisit at day sixty or when multi-document questions stop looking rare).

The trade-off. One-shot retrieval ships in six weeks instead of a quarter and costs a fraction per query. The price is a thinner answer on questions whose answer spans two documents, which the thirty-question test says are rare.

Signals. Success: accuracy above eighty percent by day sixty, guarded by escalations to the policy team falling. Stop: accuracy under seventy percent after two tuning cycles, fallback to keyword search plus the FAQ template. Leading: retrieval recall on fifty questions in week one. Agreed in writing with the support director at kickoff.

Artifact 3: The Diagnostic Readout

After Chapter 3. Audience: your team.

Dashboard snapshot. Your signals from the one-pager, targets versus actuals, with the denominator and the sampling method next to every number.

The fifty-row sample. Pull fifty real failures at random, categorize, count. Don't pre-decide the buckets.

Cluster	Count	Shape of the failure

Oracle Test on the dominant cluster. If the model were perfect at the current architecture, does the dominant cluster go away? If no, the model isn't the problem.

Trace audit verdict. Of N passing traces read, how many were legitimate solves, how many were shortcuts or ungrounded, and what the headline score becomes if that rate holds.

Where the frame broke. Which GOATS step the dominant finding maps to, and the assumption it names.

The call. Tune or reframe, in one sentence a non-technical stakeholder would understand, and the repair test that settles it: can a specific change inside the current design fix the clustered cases?

Quality bar: does the readout name a specific thing to fix, or does it just say the system needs work?

The policy bot: Diagnostic Readout.

Dashboard. Offline eval: eighty-four percent, on three hundred question-answer pairs written from the FAQ page. Production: fifty-eight percent, the average of the last three weekly samples of a hundred graded answers. Target: eighty. Stop line: seventy.

Cluster	Count	Shape of the failure
Multi-document	35	Answer spans the base policy and an addendum; the bot retrieved one and answered confidently
Superseded version	9	Cited a policy version no longer in force
Scattered	6	No shared shape

Oracle Test on the cluster. A perfect model handed the same five retrieved documents still fails these, because the document it needs isn't in the five. Not a model problem.

Trace audit. Twenty-five of fifty-eight passes read. Twenty legitimate. Five answered correctly from training data after retrieval missed, ungrounded. If that rate holds, the grounded score is nearer forty-six than fifty-eight.

Where the frame broke. Step 2. The single-document assumption was tested on questions agents asked a human, one narrow question at a time. Agents typing into a bot paste the whole situation.

The call. The cluster leans reframe, pending the repair test: a reranker recovers the cases where both documents share the question's vocabulary, and nothing inside one-shot retrieval reaches the rest.

Artifact 4: The Decision Memo

After Chapter 4. Audience: your sponsor. The first paragraph is the whole memo.

The evidence. What you're seeing, with data. No "I think," no "we're worried."

The pre-commitment. Quote the threshold you agreed on in the one-pager, and where the number stands against it.

The diagnosis. Framing or capability, and which GOATS step.

The options. Persist, pivot, or stop, with the cost of each.

The recommendation. One of the three. If pivot, which of the five moves, what it changes, and the time box, the fresh-data bar, and the fallback if it fails.

The change plan. Who owns the rollout, whose dashboards or workflows the change touches, and who retires the old thing, by when, and what retiring it means.

Quality bar: could your sponsor forward this memo unedited?

The policy bot: Decision Memo.

The evidence. Answer accuracy is fifty-eight percent across the last three weeks of production traffic, a hundred graded answers a week. The grounded rate, answers the citation supports, is nearer forty-six.

The pre-commitment. At kickoff we agreed that below seventy percent after two tuning cycles we'd revisit the approach. We've run three.

The diagnosis. Framing, Step 2. Seventy percent of failures are questions whose second document needs a query the first document supplies. A reranker recovered the slice that shared vocabulary. Nothing inside one-shot retrieval reaches the rest.

The options. Persist: another tuning cycle, two weeks, and no hypothesis for why it ends differently. Pivot: iterative retrieval, four weeks. Stop: ship the FAQ template, which already beats us on grounded answers, and return the team to the roadmap.

The recommendation. Pivot. Decompose the question into sub-queries and retrieve for each. Two-week prototype, judged on forty fresh multi-hop questions from the next two weekly samples, with no drop beyond the two-point eval spread on a hundred single-document questions, and cost and latency recorded. Bar: seventy percent on the fresh set. If it misses, we stop and ship the template.

The change plan. The support team lead owns the rollout. The policy team's interrupt count moves to the shared dashboard so the outer-loop number is visible. Shadow mode for two weeks after the prototype clears. On the cutover date the team lead retires the standalone one-shot endpoint; its retrieval path lives on as one route inside the new system, not as a second bot running alongside.